Appendix

Daniël Lakens

Detailed record of every Metacheck defect found and fixed during this project

This appendix documents, in full detail and in chronological order, every software defect in Metacheck that this project's validation effort identified, the evidence that surfaced each one, the exact code change that fixed it, and the corpus-level numbers before and after that fix. The body of the manuscript reports only the final, current numbers; this appendix exists so that every intermediate claim made in the course of arriving at those numbers remains checkable, without requiring the main text to carry a running commentary on its own revision history. Seven rounds of validation are documented below: (A.1) the original empirical-comparison review against Cooper et al.'s coding, which produced issues #378–#384; (A.2) a focused re-examination of `code_check`'s parse-error and package-version-pinning checks; (A.3) a systematic validation of five further `code_check` checks (missing referenced files, hardcoded absolute paths, `setwd()` calls, comment density, and library/import scattering) against real corpus files, which produced five further fixes; (A.4) a file-count gate added after one outlier repository's processing time exposed a gap the existing byte-size caps did not cover; (A.5) a full re-check of every remaining disagreement against Metacheck after five more fixes (Zenodo reliability, repository-platform coverage, and a rate-limit reliability defect found live during the re-check itself); (A.6) a systemic under-sampling bug found in A.5's own Dataverse DOI-prefix allowlist, two further platform-coverage fixes (DataONE, institutional Figshare instances) and two matching bugs in this project's own comparison code, and a complete, evidence-based categorization of every "repository not detected" disagreement these fixes did not resolve; and (A.7) a full-corpus run of Metacheck's `reproducibility_check` module — a different question from A.1–A.6 (does a paper's own shared code actually run and reproduce its own reported results, not merely whether a repository was detected) — which surfaced five further defects specific to running many papers' Docker containers concurrently at corpus scale and to extracting a paper's own declared dependencies from its code. Fixes described in A.1–A.4 were committed to `scienceverse/metacheck`, branch `repocheck_improvements` (pull request #385); A.5 and A.6's fixes were committed directly to that package's `dev` branch (A.6 as pull requests #410, #412, and #414); A.7's five defects were filed as issues #417–#421, two of which (#419, #420) are confirmed fixed on `dev` as of this writing. Every fix from A.1–A.6 is

covered by a new or updated automated regression test in the package's own test suite; A.7's two confirmed-fixed defects were fixed and committed by the package's maintainer rather than by this project directly, so this manuscript does not independently confirm test coverage for them.

A.1 Defects found during the original empirical comparison

Six defects were identified while individually reviewing every disagreement between Metacheck and Cooper et al.'s human coding (Empirical Comparison, above): five in Metacheck itself, and one in this project's own comparison code. All six are described here with the same level of detail as in the body text; they are repeated in this appendix only for completeness, so that Appendix A is a single, complete record of every fix made during this project.

#378 — `zip_peek()`'s unauthenticated requests. `zip_peek()` (`R/zip-peek.R`) reads a remote zip archive's central directory via an HTTP Range request, without downloading the archive, so a downloader can decide whether the archive is worth fetching before spending the bandwidth. This function's HEAD and Range requests were unauthenticated. Dryad's file-download endpoint answers 401 Unauthorized to any request that touches file bytes, including a bare HTTP Range request — confirmed live against a real Dryad file-download URL — so every Dryad-hosted archive's peek failed structurally, before any question of the archive's actual contents. Fix: both requests now route through `.auth_for_url()`, the same host-based authentication dispatcher every other Metacheck download path already uses, so Dryad's token is attached automatically without requiring each call site to opt in individually (an earlier, per-call-site `req_func` opt-in mechanism had only been adopted by 1 of 7 call sites that needed it).

#379 — `data_check`'s multi-sheet Excel limitation. `data_check`'s tabular-usability check called `readxl::read_excel()` with no `sheet` argument, so it only ever read a workbook's first sheet. When the first sheet was a cover, metadata, or "read me" sheet — a convention confirmed across many real corpus files, with sheet names including "metadata," "Meta-data," "MetaData," "read me," "ReadMe," "User guide," "LEGEND," and "Description of variables" — a later sheet holding genuine, well-structured tabular data was never inspected, and the file was misclassified as unusable ("100% free text, not variables"). One confirmed case: sheet 2 of a real downloaded workbook held 1,532 rows by 58 real variable columns, never read before the fix. Fix: `data_read_head()` now accepts a `sheet` argument, and `data_check` tries each remaining sheet in a workbook before giving up on the file. This was the single largest driver of `data_download` disagreement: before the fix, `mc_data_download` and Cooper et al.'s `data_download` disagreed on 51 of 967 comparable papers (94.7% agreement); after it, agreement rose to 99.1% (908/916 papers, 8 disagreements remaining), and the resolved cases accounted for 29 of the original 51 disagreements (57%).

#380 — Dryad's authentication gap on other download paths. The same unauthenticated-request problem identified for `zip_peek()` (#378) also affected `.remote_`

`size()` and `.remote_content_length()`, two other functions in the download path that check a remote file's size via HTTP HEAD before deciding whether and how to download it. `.remote_content_length()` already had a `req_func` opt-in mechanism for per-host authentication, but only 1 of 7 call sites across the codebase had actually adopted it, so the other 6 still received a 401 from Dryad. Fix: both functions now route through `.auth_for_url()` unconditionally, the same fix applied to #378, closing every remaining unauthenticated-request gap in the download path at once rather than patching call sites one at a time.

#383 — `code_check` could not see code bundled inside a zip archive. This was a compound architectural gap with three separate causes, discovered while investigating why `code_check` found no code files at all for several papers whose repository, on manual inspection, clearly contained real analysis scripts bundled inside a `.zip`: (1) `code_check()` never read `data_check`'s already-expanded file structure, so any code file `data_check` had already discovered inside a zip (in the course of looking for data files) was invisible to `code_check` regardless; (2) `code_check` had no zip-peek capability of its own for the case where `data_check` had not already run first; (3) the shared zip-expansion helper `.archive_rows()` discarded code-typed files outright even when it did expand a zip, keeping only `"data"` and `"documentation"`-typed files. Fix: `code_check()` now prefers `data_check`'s structure when available; it gets its own zip-peek fallback (`.code_expand_zip()`) for when `data_check` did not run first; and `.archive_rows()` now keeps `"code"` alongside `"data"`/`"documentation"`. A real, concrete example verified live: a Zenodo record (record 3694205, repository `SharpRT/CMV_Spread`) bundled 4 real MATLAB `.m` files inside a zip that were entirely invisible to `code_check` before this fix. This fix, together with the archive-member content-fetch wiring described immediately below, moved `code_download` agreement from 66.2% (355 comparable papers, 120 disagreements) to 89.7% (358 comparable papers, 37 disagreements) — the first of two stages in this construct's improvement (the second is #384, below).

As part of the same round of work, `repo_check`'s zip-peek expansion was extended to carry not just a discovered code file's name (already possible after the fixes above) but its content: each expanded row now also carries `archive_url` (the parent zip's own URL) and `archive_member` (the file's exact name inside it), and `download_repo_files()` (`R/repo-download.R`) uses those two fields to fetch that one member's bytes directly, via a byte-range request against the parent archive, without downloading the archive's other contents. This mechanism was verified working in every isolated unit test at the time, but a full end-to-end pipeline run against a real paper still returned no file content for reasons not yet diagnosed at that point — this was filed and tracked as issue #384, described next, rather than merged as "fixed" while a known gap remained open.

#384 — Two distinct causes behind the archive-member byte-fetch failing intermittently. Issue #384 was filed as an open question ("archive-member content fetch works in isolation but returned NA once in a full pipeline run") rather than a diagnosed bug, and its investigation spanned two separate validation efforts and uncovered two independent causes.

Cause 1: silent truncation over 32,768 bytes. `.zip_inflate_member()` (`R/zip-peek.R`) decompresses one zip member's raw bytes by calling `zip::inflate()`. Passing the member's exact known uncompressed size to that function raised an unrelated "Failed to inflate data" error (a quirk internal to the `zip` package), so the original code left the `size` argument unset instead. `zip::inflate()`'s own documentation states that leaving `size` unset triggers automatic buffer resizing as needed — but, confirmed live against a real Zenodo-hosted zip member, this documented behaviour does not actually happen in practice: any member larger than the default 32,768-byte output buffer came back silently truncated at exactly that length, with no error, no warning, and a `bytes_written` field the calling code never inspected. A concrete confirmed case: a real 44,669-byte code file came back as exactly 32,768 bytes. `.zip_member_fetch()`'s own subsequent length check (comparing the decompressed length against the size recorded in the archive's own central directory) correctly caught this mismatch and discarded the truncated bytes — but the practical, user-visible effect was that every code or data file over 32KB inside a zip silently failed to fetch, indistinguishable from a genuine network or host-compatibility failure. Fix: passing `size + 1` (one byte over the true size) avoids both failure modes — it is no longer the exact size that triggers the package's internal error, and it is large enough that the real output always fits in the up-front buffer rather than depending on the documented-but-non-functional resize path. Verified against both a real archive member under 32KB and one over it, and covered by a new offline regression test using a locally constructed zip file whose one member compresses to roughly 95KB uncompressed. A targeted rerun of the 36 papers affected by this pattern found 27 moved from zero downloadable code files to full or near-full recovery (23 fully, 4 partially), which is what raised `code_download` agreement from 89.7% (37 disagreements) to 97.2% (10 disagreements remaining) — the second of the two stages in this construct's improvement, and the largest single agreement-rate improvement of any construct in the empirical comparison.

Cause 2: an intermittent, silently-swallowed failure not explained by the truncation bug alone. Independently of the truncation bug, a second cause was found while investigating why 40 of the archives affected by issue #384 failed entirely (every member missing) despite containing no member anywhere near the 32,768-byte threshold. Tracing the actual corpus data (`res_code_check.RData` and its 38 per-batch constituent files, in `cooper_validation_metacheck/manuscript/data/`) showed that a single pipeline run calls `zip_peek()` on the exact same archive URL twice — once inside `repo_check` to discover a zip's member names, and again inside `download_repo_files()`'s archive-member-fetch step (added as part of #383/#384's own fix) to retrieve one member's bytes — with no throttling and, critically, no retry logic at all on either call: `zip_peek()`'s own HEAD request and its two Range-request helper functions (`.http_range_tail()`, `.http_range_bytes()`) called `httr2::req_perform()` directly, with no `req_retry()`, and treated any non-206/200 HTTP status (including a 429 burst-rate-limit refusal) as an unconditional, silent `NULL` — indistinguishable, by design, from "this host does not support range requests at all." Two back-to-back, unthrottled requests to the same archive URL within one process is exactly the pattern most likely to trip a host's burst-rate limiter, and the second call (the byte-fetch, inside `download_repo_files()`) was the one exposed to it. Fix, applied in two parts: (1) all three of `zip_peek()`'s network calls now

retry through the same `is_transient`/`backoff`/rate-limit-wait helper functions (`.storage_is_transient_factory()`, `.storage_backoff()`, `.storage_retry_after_factory()`) that every other Metacheck download path already uses, so a 429 is now retried with backoff rather than immediately and silently treated as "unsupported"; (2) a new same-session, in-memory cache (`.zip_peek_cache`, keyed by URL) means the second call to `zip_peek()` on an archive already peeked earlier in the same run reuses the first call's result instead of repeating the network request at all, removing the redundant call — and the rate-limit exposure it caused — entirely rather than merely tolerating it. Verified by a new regression test exercising the cache directly (`test-zip-peek.R`) and by the full package test suite (463 tests passing, 3 pre-existing unrelated skips, 0 failures) after both changes.

Verdict corrections found by the post-rerun double-check pass. After the full corpus rerun incorporating #378–#384, every disagreement was independently double-checked a second time (see Empirical Comparison, above, for the check's method). This pass corrected 7 verdict-labeling errors from the original review: 2 in `data_availability` (one moved to `COOPER_RIGHT` after finding the "data" files actually belonged to a cited third-party software dependency's repository, not the paper's own; one moved to `METACHECK_RIGHT` after finding a methods paper's repository, originally read as containing only package example files, in fact holds real trait data and model output), 1 in `data_download` (moved to `COOPER_RIGHT` after confirming a Zenodo-hosted "Functional Trait Database" file had, in fact, already downloaded and still failed the tabular-usability check, rather than remaining `UNCLEAR` on the assumption it had not yet downloaded), and 4 in `code_download` (three rows originally recorded `METACHECK_RIGHT` whose own written reasoning already concluded the opposite, corrected to `COOPER_RIGHT`; one row originally left `UNCLEAR` where the code files were, on direct inspection, confirmed genuinely downloaded, corrected to `METACHECK_RIGHT`). The pass also found one substantial scope correction rather than a single-row relabeling: `code_download`'s dominant post-#383 disagreement pattern was originally tagged in only 13 of its 37 disagreements at the time of the initial qualitative review; re-deriving the pattern systematically against every disagreement's saved `file_location` value found the identical signature (a discovered code file with no resolvable download location) in 35 of the 37, which is what led directly to tracing and fixing #384's second cause, above. Every one of `code_archived`'s 164 disagreements was also individually re-derived against the current `repo_check`/`code_check` file listing during this pass, including the 8 originally resolved via an automated keyword search rather than an individual hand read; this found zero verdict corrections in that column.

A sixth issue, not in Metacheck but in this project's own comparison code. `code_data_preparation/03_recreate_cooper_columns.R` derives `mc_data_availability`, `mc_code_archived`, `mc_data_README`, and `mc_code_README` for a paper from `repo_check`'s per-file tables (`res_repo_check$table`, `res_data_check$structure`, `res_code_check$table`). A paper for which `repo_check` found zero files anywhere has no row at all in any of those three tables, even though `repo_check`'s own per-paper `summary_table` — which covers every one of the 1861 papers — correctly records `files_n = 0` for it. Because the script built its paper list from the three per-file tables rather than from `summary_table`, every such paper was silently absent from the comparison entirely: its `mc_*` value was not `FALSE`, it simply never existed as

a row to compare. Confirmed directly: 495 of 1861 papers were missing this way, including 404 papers Cooper et al. coded `data_availability Yes` — 404 real disagreements (cases where Cooper et al.'s coders found a repository Metacheck's own detection had missed) that were never entering the comparison at all, rather than being counted as a genuine miss. Fix: the paper list is now built from `summary_table$paper_id` (already the full 1861 by construction), and `mc_data_availability`, `mc_code_archived`, `mc_data_README`, and `mc_code_README` are explicitly set to `FALSE` for a paper `repo_check` examined and found nothing in; every other `mc_*` column (license, DOI, download, format, language) is correctly left `NA` for such a paper, since those questions remain genuinely inapplicable without a repository to ask about. Every number reported in the Empirical Comparison section above reflects this correction.

A.2 Re-examining `code_check`'s parse-error and package-version-pinning numbers

A separate line of inquiry, prompted directly by a sentence in an earlier manuscript draft stating that "138 files (1.3%) failed to parse as valid code in their declared language at all," asked two questions: is this the right denominator, and is the file list behind it available for inspection at all.

Locating the underlying data. `code_check()`'s per-run output already carries everything needed: a `table` with one row per code file (`file_name`, `language`, `parse_error`, `parse_error_msg`) and a `summary_table` with one row per paper (`code_version_pinned`, `code_parse_errors`), both already computed by the module — no new code was needed to make this data available, only to locate and aggregate the 38 saved per-batch corpus-run files (`cooper_validation_metacheck/manuscript/data/batches/res_code_check_batch001.RData` through `batch038.RData`) that had already been produced by the original corpus run.

Package version pinning: 34.2% was a batch-level upper bound, not a per-paper rate. An earlier manuscript draft reported that "only 13 of 38 batches (34.2%) contain at least one paper with any form of version pinning," explicitly flagged in that draft's own text as an upper bound on the true per-paper rate because a batch of ~50 papers needs only one pinned paper to count. Checking directly against the saved per-batch `summary_table$code_version_pinned` column (present in every batch, keyed by `paper_id`) confirmed the per-paper detail this draft believed was not retained had, in fact, been retained all along: aggregating it across all 38 batches gives 18 of 400 code-bearing papers (4.5%) with version pinning present — substantially lower than the 34.2% upper bound, as expected, and now a true per-paper number rather than an aggregate-of-aggregates. No code fix was needed here; this was a reporting correction (using data that already existed but had not yet been extracted at the right level of granularity), not a software defect.

Parse errors: 1.3% used the wrong denominator, and undercounted for a subtler reason. The originally reported 1.3% divided 138 parse failures by 10,461, the total code-file count across every language in the corpus — but parsing is only ever attempted on R-type files (`.R`,

`.Rmd`, `.qmd`; `code_parse_r()` is R-only), so the other 3,259 SAS/SPSS/Stata/Python/MATLAB files in the corpus were never eligible to fail this check and should not have been in the denominator at all. Restricting to the corpus's 7,202 R-type files, a further complication emerged: 1,651 of those were never actually downloaded or read at all (an inaccessible repository or a failed download), so parsing was never attempted on them either — they, too, do not belong in the denominator. A further 15 files reached a local copy but errored during extraction (before `parse()` itself was ever called), splitting into two genuinely different causes on closer inspection: 11 were confirmed to be genuinely empty (0-byte) files — fetched directly from their original GitHub repositories to confirm the emptiness is a fact about the authors' own repository, not an artefact of Metacheck's download step (e.g. `github.com/olgaviedma/LadderFuelsR`'s `R/foofy.R` and `github.com/dpritchard/frair`'s `tests/testthat/test_emdII.R`, both independently confirmed 0 bytes via a direct `curl` fetch from `raw.githubusercontent.com`) — and 3 were `.Rmd` files that failed during Quarto/knitr's own chunk-extraction step with a genuine, structural fence-mismatch error (a closing code-fence with a different number of backticks or indentation than its opening fence), which is the same underlying "not valid in its declared format" problem a parse error represents, just caught one step earlier in the pipeline than `parse()` itself. Restricting to the 5,536 files that were both locally available and actually reached a call to `parse()`, 138 failed (2.5%) — not 1.3%, and not the 1.92% that a denominator of "all 7,202 R-type files regardless of whether they were ever read" would give either. Including the 3 fence-mismatch failures as the same kind of problem gives 141 of 5,539 (2.5%).

This investigation also surfaced a genuine, previously unknown software defect: **`code_read()` (`R/code_check.R`) crashes on a genuinely empty file.** `readr::guess_encoding()` returns a zero-row result for a 0-byte file, so `enc <- readr::guess_encoding(file_path)$encoding[[1]]` silently produces `NA` rather than a valid encoding string, and that `NA` then crashes deep inside `readr`'s own internals (`if (stringi::stri_enc_isascii(lines))`, which requires a length-one logical and receives `NA` instead) with the unhelpful, generic message `"missing value where TRUE/FALSE needed"` — this is exactly the message recorded for all 11 of the empty-file cases above, rather than a clear "this file is empty" signal. This defect affected a small fraction of files (11 of 7,202 R-type files in this corpus) and was not fixed during the review described here; it has since been fixed: in Metacheck 0.3.3.11, `code_read()` returns an empty result for a 0-byte file instead of failing (checked directly on 18 October 2026).

A.3 Systematic validation of five further `code_check` checks

A third, more systematic validation exercise checked five of `code_check`'s remaining automated checks — missing referenced files, hardcoded absolute paths, `setwd()` calls, comment density, and scattered library/import lines — against real files from the same corpus, reading every flagged file that had already been downloaded locally (8,218 of 10,461 code-file rows) and sampling up to 100 of the remainder that would have required a fresh download. The full method, every sampled file's judgment, and every quoted example are recorded in

a dedicated, reproducible validation report (`code_check_validation.qmd`, with its accompanying script `validate_code_check.R` and judgment tables, all in `cooper_validation_metacheck/manuscript/`) with a fixed random seed (`set.seed(42)`) at every sampling step; this appendix summarizes its findings and the resulting code fixes. Two checks not covered by this validation pass (parse errors, package version pinning) are covered separately in A.2 above.

Missing referenced files — case-sensitivity bug (fixed). `code_check`'s check for files a script tries to open but that are absent from the repository (`inst/modules/code_check.R`, `missing_files <- setdiff(base_ref, base_repo)`) compared filenames case-sensitively. 13 confirmed false positives across the sampled files were all the same mechanism: the referenced file exists in the repository under a different letter case, which is not a real error on the author's part on the case-insensitive filesystems (Windows, macOS by default) most authors develop on. Concrete examples, each confirmed directly against the paper's real repository file listing: paper `10_1002_2688_8319_12229` references `REM_tools.r` where the repository holds `REM_tools.R`; paper `10_1111_2041_210x_13923` references `diagnostic_fcns.r/boot_glmm.r` where the repository holds `diagnostic_fcns.R/boot_glmm.R`; paper `10_1111_2041_210x_13125` references `FoodWeb_Functions.R` where the repository holds `Foodweb_functions.R`; paper `10_1002_pan3_10736` references two scripts whose repository copies differ only in the first letter's case. Fix: the comparison now lowercases both sides before comparing (`missing_files <- base_ref[!tolower(base_ref) %in% tolower(base_repo)]`), removing all 13 confirmed false positives found in this sample and any of the same kind not yet sampled.

Missing referenced files — `vroom::vroom()` not recognised as a file-reading call (fixed). The same check's file-reference extractor (`code_file_refs()`, `R/code_check.R`) recognises a fixed list of R reader-function names (`read.*/read_*`, `import`, `fread`, `readRDS`, `load`, `readLines`, `fromJSON`, `readtext`, `source`); `vroom::vroom()` does not begin with "read" and was absent from this list, so a file loaded exclusively via `vroom::vroom()` was invisible to the missing-file check regardless of whether it actually existed in the repository. Confirmed false negative: paper `10_1111_2041_210x_13440`'s `R/ncbi.R` calls `vroom::vroom(file.path(dir, "nodes.dmp"), ...)` and a second `vroom::vroom()` call for `names.dmp`; neither file exists in the repository, but the check never flagged either, since `vroom` was not on its keyword list. Fix: `"vroom"` added to the reader-keyword list.

Missing referenced files — remaining known limitations, not yet fixed. Two further false-negative mechanisms were confirmed but not fixed, since each would require a more substantial change than a keyword addition or a comparison tweak: (1) `code_file_refs()` only captures a quoted filename appearing on the same line as a reader-call keyword, so a file assigned to a variable on one line and read via that variable on a later line (`liz_file <- "example_lizard_data.csv"`, then later `read.csv(liz_file)`) is invisible to the check even when the file genuinely does not exist (confirmed: paper `10_1111_1365_2435_13993`'s `tests/testthat/test-m_run_biophysical.R` does exactly this for three files, none of which exist in the repository). (2) When a referenced filename is built as `paste0(some_variable, "...literal_suffix.csv")`, the check's regex only captures the literal trailing fragment (e.g. `_`

M_D_abundance_interactions.csv) rather than the true, variable-interpolated filename — the resulting "missing file" flag is still technically correct (nothing by that literal fragment name exists either), but the name shown to a paper's author is a confusing partial match rather than the file they would recognise; across the corpus, 128 of 2,599 (4.9%) genuinely-missing entries have a name starting with punctuation, the signature of this dropped-prefix pattern.

Hardcoded absolute paths — three false-positive mechanisms fixed, one left open. code_abs_path() (R/code_check.R) regex-matches quoted strings shaped like an absolute filesystem path. Three confirmed false-positive mechanisms were fixed: (1) a single-segment string beginning with / (e.g. self.fetch('/login') in a vendored web-app test suite, github.com_ccrisan_motioneye/tests/test_handlers/test_login.py) is an HTTP route, not a filesystem path — 322 of 4,182 originally matched strings were 6 characters or fewer, a class dominated by exactly this pattern; the regex's bare-/-prefixed branch now requires a second /-separated segment before matching, which a real absolute path (e.g. /Users/name) always has and a bare route never does. (2) An all-digit UNC "host" is a sub()/gsub() regex backreference string, not a network path — confirmed twice in real code: osf.io_5fter/IDcallR_sameN_withIDcallPP.R line 183, sub('group (\d+)_(\d+)', "\2\1", colnames(clupo)), where "\2\1" was matched as \\host\share because a bare digit satisfied the old hostname character class; and a second, independent instance in a Pyrenees-mapping script's gsub('(.{1,10})(\s|$)', '\1\n', ...) call, matched for the same reason. The UNC branch's hostname character class now requires at least one letter, which excludes both confirmed cases without excluding any real hostname (no genuine UNC host is all-digit-and-punctuation with no letter at all). (3) The pattern's own pre-existing docstring already flagged a known risk — a variable-built path like paste0(dir, '/file.csv') triggering a false match on the literal fragment — and this was confirmed for real: github.com_YvesBas_Tadarida-C/Sonotypes/write_tabase3HF_Sonotype.R line 40, list.files(paste(ListDate[[i]],"/eti",sep=""), ...), where "/eti" is a relative fragment concatenated onto a loop variable. This third mechanism is addressed by the same "require a second path segment" fix as (1), since /eti is itself single-segment. A fourth mechanism, confirmed but **not fixed**: a Python string built by +-concatenation from a relative literal ('data\\' + tile_name + '\\run\\' + model_name + ..., from a Zenodo-hosted model_test.py) is a relative path whose middle segments (\run\, \output\tile\) still match the UNC branch on their own; no fully static regex-only fix exists for this case, since it would require tracking whether the enclosing expression is a string concatenation starting with a relative literal, which is beyond what a line-based regex can determine. Of the true positives sampled for comparison, 15 of 15 further-checked "real username-shaped" matches (e.g. C:/Users/reddin/Documents/..., /Users/farrer/Dropbox/..., /home/michael/Dropbox/...) were confirmed genuine, machine-specific absolute paths — the large majority of this check's flags are correct.

setwd() calls — no false positives found in this corpus; one confirmed-but-unobserved risk noted. All 194 distinct setwd() call strings found across the sampled files are genuine, live, executable calls, including calls whose argument is itself relative or dynamic (setwd("./data"), setwd(dir), setwd(tempdir())) — these correctly count as

flags under this check's own stated rule, since the anti-pattern being flagged is changing the working directory at all, not specifically a hardcoded absolute-path argument. A latent false-positive mechanism was confirmed by construction (a synthetic test, not a real corpus case): `code_setwd()` has no awareness of string-literal context, so a line like `message("Please run setwd(your_path) before continuing")` would be matched as if it were a live call. Scanning all 194 real call strings in the corpus for instructional-text markers ("please," "before," "instructions," "note:") found zero real occurrences of this pattern — a real, demonstrated risk with no confirmed instance in this corpus, and left unfixed for that reason, though `code_setwd()` could straightforwardly reuse the same quote-context-tracking logic already implemented for `.code_strip_inline_comment()` if a future corpus surfaces a real case.

Comment density — a Python-specific docstring signal added (not a bug fix). `code_remove_comments()`'s Python branch deliberately never treats a triple-quoted string as a comment, since doing so would also destroy real docstrings (themselves ordinary string literals, syntactically). The consequence, quantified directly: of 280 zero-comment-flagged Python files in the corpus, 200 (71.4%) contain at least one complete triple-quoted docstring block — real documentation that the mechanical `#`-only comment count cannot see, and so reports as "0% comments" regardless. Five were read directly to confirm these are genuine module/function docstrings rather than incidental triple-quoted data, e.g. `model_runs.py`'s `"""Filters for Model Runs."""` and `features.py`'s 16 separate triple-quote markers (8 docstrings) opening with `"""API functions to interact with feature names."""`. This is a deliberate, correctly-reasoned design choice in `code_remove_comments()`, not a bug — stripping docstrings to count them as comments would also delete them from every downstream check that reads code text — so the fix made is additive rather than corrective: a new function, `.code_has_docstring()`, detects whether a file contains at least one complete triple-quoted block, and `code_line_stats()` now returns this as a separate `has_docstring` field (`NA` for every non-Python language) alongside the unchanged `percent_comments` metric. `inst/modules/code_check.R`'s report text now adds an explanatory note whenever a zero-comment Python file in fact has a docstring, so a reader is told the file is documented via docstrings rather than concluding it has no documentation at all.

Scattered library/import lines — Python import-anchor tightened (fixed). `code_library_lines()` (`R/code_check.R`) previously anchored its Python import-detection regex on the bare keyword plus a following word (`^\s*(import|from)\s+[A-Za-z_.]`), with no requirement that the rest of the line actually form a valid import statement. Because Python's comment-removal never strips triple-quoted strings (the same mechanism behind the comment-density finding immediately above), docstring prose that happens to start with "from" or "import" survives into this scan and was matched as if it were a real import line. Concrete, exact example: `github.com_openvinotoolkit_cvat/site/process_sdk_docs.py`, line 213, inside a docstring spanning lines 211–214 — the prose sentence " from the generated model and api descriptions." was matched as an import line at line 154 of the comment-stripped text, while the file's 10 real imports are all genuinely contiguous at lines 1–10; the spurious match alone widened the reported gap between imports to 144 lines, crossing the "scattered" threshold (more than 3 non-comment lines between library statements) on a file whose real imports are, in

fact, perfectly grouped. Quantified across all 525 Python files flagged as scattered in the corpus: 155 (29.5%) have at least one spurious, non-import match of this kind; of those, 68 (13.0% of all 525 flagged files) would no longer be flagged as scattered at all once the spurious matches are excluded — pure false positives caused entirely by this mechanism (the remaining 87 files with a spurious match are still genuinely scattered even after excluding it). R showed essentially no equivalent problem in a 60-file sample (the one apparent candidate, on manual inspection, was a genuine conditional `require()` call, not a false positive — R's regex requires an opening parenthesis directly after the keyword, structurally harder to spuriously trigger than Python's bare whitespace-plus-word anchor). Fix: the Python pattern now requires the full import shape rather than just the leading keyword — a `from` line must also contain its own `import` keyword later on the same line (`^\s*from\s+[A-Za-z_][A-Za-z0-9_.]*\s+import\s`), and a bare `import` line must be followed by a comma-separated identifier list with an optional `as name` and nothing else (`^\s*import\s+[A-Za-z_][A-Za-z0-9_.]*(\s*,\s*[A-Za-z_][A-Za-z0-9_.]*)*(\s+as\s+\w+)?\s*$`) — verified against the docstring false-positive example above (now correctly unmatched) and against every real import shape found in the corpus sample, including multi-line `from x import (a, b, c)` continuations (matched on their opening line) and indented imports, with no loss of any real detection.

Status of the fixes in this appendix. All Metacheck fixes described in A.1 and A.3 (nine in total: #378, #379, #380, #383, both causes of #384, and the five `code_check` fixes from the systematic validation) have been committed to `scienceverse/metacheck`'s `repocheck_improvements` branch (pull request #385) and are covered by the package's automated test suite (463 tests passing at time of writing, 0 failures, 3 pre-existing skips unrelated to these changes). The sixth fix described above (the comparison script's missing-repository default) is not a Metacheck fix and has been committed instead to this project's own `code_data_preparation/03_recreate_cooper_columns.R`, in the `cooper_validation_metacheck` repository. The one defect identified but not yet fixed (`code_read()`'s crash on an empty file, A.2) and the two `code_file_refs()` limitations noted but not yet fixed (A.3) remain open. The further full corpus rerun incorporating the five A.3 fixes, planned when this appendix was first written, has since been performed (confirmed directly: the current corpus run's `code_check` output carries the `has_docstring` field A.3's docstring fix introduces, which cannot exist in a pre-fix run) — the numbers reported in "Results you get for free alongside Metacheck" above reflect that post-fix corpus, not the pre-fix state described earlier in this sentence's original drafting. This was a distinct item from the rerun described in Appendix A.5 below, which re-checked only the papers with a remaining disagreement (not the full corpus) and tested a different set of fixes (Zenodo reliability, repository-platform coverage, and rate-limit handling), not the five A.3 `code_check` fixes.

A.4 A repository excluded from processing by file count

During the corpus rerun incorporating the A.3 fixes, one paper's data repository — Zenodo record 10.5281/zenodo.3971689, SAPFLUXNET (a global database of sap flow measurements:

202 datasets from 121 locations, sap flow and environmental time series for 2,714 individual plants) — was found to hold approximately 4,000 individual files. Downloading and, in particular, `data_check`'s per-file classification and full-content read of every one of those files (`data_check_head(..., n_rows = Inf)`, by design, since several of `data_check`'s validation checks need the complete dataset rather than a sample) took several hours and multiple gigabytes of memory for this single paper, out of 1,861 in the corpus, before the run was stopped.

This exposed a real gap in Metacheck: `download_repo_files()` already gates a repository outright by total byte size (`max_download_size`) or per-file byte size (`max_file_size`), but had no gate on file count — a repository holding thousands of individually small files was not caught by either existing cap. A new parameter, `max_files_per_repo` (`R/repo-download.R`, threaded through `data_check()` and `code_check()`), gates a repository by file count alone, using the same refusal/reporting mechanism as the existing size caps. This has been committed to the `repocheck_improvements` branch alongside a regression test, and the corpus rerun was configured with `max_files_per_repo = 1000`.

Under this cap, the SAPFLUXNET repository was refused outright in that run. The cap exists so that one exceptionally large repository cannot block or degrade processing of the other 1,860 papers in a corpus run. In the results reported in this manuscript, however, the repository was not refused: `data_check` lists all 3983 of its files, every one of them classified as data.

A.5 A targeted rerun on every remaining disagreement, after further platform-detection and reliability fixes

After Appendix A.1–A.4's fixes and the resulting full corpus rerun, every remaining disagreement between Metacheck and Cooper et al.'s coding (991 rows across 989 papers, the same set individually adjudicated for the Empirical Comparison above) was re-checked a further time against Metacheck as it stood after seven more commits landed on `scienceverse/metacheck`'s `dev` branch: two fixing crashes and false negatives in Metacheck's Zenodo integration, three widening which repository platforms `repo_check` can locate a repository on at all (targeting "repository not detected," the single largest recurring disagreement cause identified in the original review), and two fixing a rate-limit reliability defect surfaced live during this rerun itself. Rather than reprocess the full 1,861-paper corpus, only the 989 papers with at least one open disagreement were rerun, one paper at a time (so that a single paper's slow host response could not block any other), reusing the shared on-disk repository-file cache from the original corpus run so that a repository already fetched and unchanged cost no new download.

Zenodo reliability. Two fixes addressed real crashes and false negatives in Metacheck's Zenodo integration, confirmed against Zenodo records already present in this corpus. `online()`, the function `repo_check` calls to confirm a host is reachable before querying it, treated a single transient DNS lookup miss as "Zenodo is offline" and abandoned that paper's entire Zenodo retrieval rather than retrying; it now retries. Separately, a JSON metadata field

arriving as a length-zero value (an empty array) rather than `NULL` defeated the `%||%` operator used throughout Metacheck's metadata-extraction code (which only guards against `NULL`, not length-zero), producing a `data.frame()` "arguments imply differing number of rows" error that was silently swallowed inside a `tryCatch`, discarding the whole record; a new `%empty_or%` operator, applied everywhere the same gap existed (Dataverse, Dryad, DSpace7, Figshare, GitHub, GitLab, OSF, PsychArchives, and ReShare, not just Zenodo), closes this. A further, closely related pair of crashes was found and fixed the same way: a Zenodo record Metacheck could not read at all previously returned a `data.frame()` with only `zenodo_id`/`error` columns, so a paper whose every Zenodo record was unreadable produced a metadata table with no `doi`/`license` columns at all, which crashed `repo_check()`'s own Zenodo-metadata-extraction step for that paper the same "differing number of rows" way; and a `repo_url` arriving as a list column (rather than a plain character vector) could silently explode `data.frame(zenodo_url = <list>)` into one column per list element instead of the expected single column. Both are now guarded.

Repository platform coverage. Three commits substantially widened the set of hosts Metacheck's `repo_check` can locate a repository on at all, directly targeting the "repository not detected" cause responsible for the largest share of the original disagreement (the original review named DataSTORRE, DIGITAL.CSIC, AADC, SRA/GenBank, KNB, Neotoma, CIRAD Dataverse, and ForestPlots.net as example platforms with no backend at the time). Mendeley Data (detected via `10.17632/` DOIs and `data.mendeley.com` URLs, queried through the public, unauthenticated `data.mendeley.com/public-api/datasets/<id>` endpoint) and DataONE (a federation of independently-run repositories rather than one hosted service; four member nodes verified live and wired in — Arctic Data Center, KNB, Taiwan Forestry Research Institute, and the Smithsonian Tropical Data Portal, each mounting the same underlying REST API at a different base path) were added as new platforms entirely. Separately, Dataverse's recognised-installation list was expanded from 50 to 121 hosts, 16 institutional Figshare white-label front ends were added, and DSpace support was generalized from PsychArchives specifically to any host running the same legacy (6) API (5 more verified live) plus a new implementation covering 64 DSpace 7 installations (e.g. Cornell's eCommons, Georgia Tech's Digital Repository, ETH Zurich's Research Collection) — every one of these 121 + 16 + 5 + 64 hosts confirmed live via its own version/info endpoint before being added, not merely listed from a registry. A further, smaller fix excludes an R package's own source tree (`DESCRIPTION`/`NAMESPACE`/its `R//man//tests/` subdirectories) from a repository's data/code file listing, on the reasoning that reusable software infrastructure bundled alongside a study's own materials is not itself the study's data or analysis code and should not inflate `files_n`/`files_code` or appear in downstream reports.

A reliability defect found live during this rerun. While this rerun was in progress, a single paper stalled repeatedly at the same point across two separate attempts: `zip_peek()`'s HEAD request and its Range-request follow-up (`.http_range_tail()`, `R/zip-peek.R`), used to read a remote zip archive's file listing without downloading it, each independently hit Zenodo's rate limit and each, independently, paid the full announced wait for the host's confirmed reset — three consecutive confirmed-rate-limit waits back to back for what was, from Zenodo's own

side, a single rate-limit window, since neither request had any way to know the other had already discovered and waited out the same limit. (The rate-limit-wait message itself was also, at the same time, made to name the specific host responsible — Zenodo, Dryad, GitHub, or GitLab, the only hosts that expose a machine-readable reset time — rather than reporting only "the host's own reset," a smaller, secondary fix made in the course of diagnosing this one.) Fix: a session-scoped record of the last confirmed rate-limit reset per host, written the first time any request discovers one and consulted by every subsequent request to that host before it is even sent, so a host already known to be rate-limited waits out only the genuinely remaining time once, rather than every independent request rediscovering the identical window from a fresh 429. Verified with new regression tests covering the record/lookup logic directly (round-tripping a wait, keeping the later of two recorded resets, clearing an expired window, and the pre-flight wait actually sleeping the correct duration) and confirmed live: the same paper that previously stalled for over 40 minutes across three stacked waits resolved in roughly 3 minutes with zero confirmed-rate-limit waits after the fix, the remaining time attributable to Zenodo's own server-side response latency for that specific record (confirmed independently: a single isolated request to the same record, made outside Metacheck entirely, itself took 30 seconds and returned a 504 on a second attempt).

Results of the rerun. Across the 1,532 individual construct-level disagreements re-checked (989 papers × up to 7 constructs each, matching however many constructs a given paper had originally disagreed on), 76.8% (1,176 of 1,532) are now resolved — Metacheck's fresh result agrees with Cooper et al.'s original human coding where it did not before — with 348 still disagreeing and 1 now returning a value that could not be classified as agreeing or disagreeing (`data_license`, a repository whose licence field this rerun could not read for reasons not yet diagnosed). By construct: `any_readme` 548/625 resolved (87.7%), `data_availability` 339/424 (80.0%), `code_archived` 252/360 (70.0%), `data_archive` 12/56 (21.4%, evaluated by set membership — a paper's platform coded by Cooper now appearing among Metacheck's own, possibly multi-platform, detected set, since a paper can genuinely link more than one repository), `data_license` 16/57 (28.1%), `data_download` 9/9 (100%), and `code_download` 0/1 (the sole remaining `code_download` disagreement in this corpus). The comparatively modest improvement in `data_archive` and `data_license` is consistent with the original review's own finding that these two constructs' disagreement was, unlike `data_availability`/`code_archived`/`any_readme`, not dominated by the repository-detection cause these fixes targeted (`data_license`'s largest cause was cases where Metacheck was already right and Cooper's own coding had missed a real licence field) — a fix aimed at finding more repositories naturally does less for a construct whose disagreement was rarely about a missing repository in the first place. This rerun's full row-level detail (every paper, every re-checked construct, the value before and after, and whether it resolved) is recorded in `cooper_validation_metacheck/manuscript/data/rerun_disagreements_comparison.csv`, produced by `rerun_disagreements.R`; the underlying per-paper `repo_check`/`data_check`/`code_check` results are cached per paper in `data/rerun_batches/` for inspection or a further rerun without recomputing what has not changed.

Checking the rest of the corpus for regressions. This rerun was scoped to the 989 previously-disagreeing papers only — a deliberate choice at the time, to get a fast answer

to "did these fixes help" without the cost of a full 1,861-paper rerun (see the rerun's own header comment for the reasoning). That scoping left one real question open: none of these seven fixes were written to only affect a disagreeing paper, so it remained possible that a fix which resolved one paper's disagreement had, in a different paper that already agreed with Cooper et al., turned a correct result into an incorrect one — most plausibly the R-package-source-tree exclusion (if a paper's own analysis code happens to be organised as an R package) or the expanded Dataverse/Figshare/DSpace/Mendeley/DataONE host recognition (a newly-recognised host matching something that should not have counted). This was checked directly: the remaining 868 papers (every paper not in the 989-paper disagreement set) were also rerun, using the identical per-paper, resumable approach, and the full corpus's `cooper_vs_recreated.RData`/disagreement worklist were then rebuilt from the combined result (all 1,861 papers) rather than just the 989. At the level of aggregate agreement rates, this full-corpus check was reassuring: the three constructs this comparison's own disagreement was concentrated in — `data_availability`, `code_archived`, `any_readme` — all showed a higher, not lower, agreement rate once the previously-agreeing papers were included, and `data_download`/`code_download` (both already above 99%) moved by less than a percentage point in either direction. But an aggregate rate rising is consistent with a handful of individual papers having genuinely regressed, provided enough other papers improved to more than offset them — and that is exactly what the case-by-case review below found.

A confirmed regression, found and fixed within this same cycle. The suspicion named above turned out to be correct: the R-package-source-tree exclusion did break papers it should not have touched. The exclusion was written to detect an R package's fingerprint (a `DESCRIPTION` and `NAMESPACE` file in the same folder) and drop that folder's contents — the package's `R/`, `man/`, `tests/`, `vignettes/`, `data/`, `inst/`, and `src/` subdirectories — from a repository's file listing entirely, on the reasoning that a bundled, reusable software dependency is not itself a study's data or analysis code. That reasoning holds when the package is a genuine dependency vendored alongside a study's own files, but it does not hold when the paper's own contribution — its data, its analysis code, or both — is itself packaged and distributed as an installable R package, a common practice in ecological/evolutionary methods work. In that case the exclusion strips the paper's own materials, leaving nothing behind: `repo_check`'s listing for the affected repository showed only packaging scaffolding (`.Rbuildignore`, `.gitignore`, `NEWS.md`, `README.md`, `cran-comments.md`, a bare `.Rproj` file), with the package's real `R/` source — and, for packages that bundle their own example or study data under `data/`, the actual dataset — entirely absent.

A corpus-wide scan for this exact symptom (a repository whose only surviving files, after the exclusion ran, were scaffolding, with zero files classified as data or code) surfaced candidates across the full 1,861-paper corpus, each individually verified against the repository's real GitHub listing (via `gh api`) or Dryad/Zenodo record before being counted: 16 confirmed true instances (`RevGadgets`, `BaBA`, `m2b`, `RRPP`, `actel`, `scampr`, `MINOTAUR`, `LadderFuelsR`, `biomonitoR`, `EcoEnsemble`, `fossilsim`, `frair`, `tRophicPosition`, `sabinaNSDM`, `bef`, and `phylomice`), plus several candidates from the same scan that turned out to be unrelated causes once checked directly (a repository with no `DESCRIPTION`/`NAMESPACE` at all, and several Dryad/Zenodo-hosted

papers whose `code_archived` disagreement traced to an unconnected, pre-existing detection gap — code cited in a journal's own Supporting Information, or a repository never located by `repo_check` at all). Two confirmed cases (`BaBA`, a GitHub-hosted barrier-behaviour-analysis package whose `data/` folder holds the actual tracked-animal datasets for its own paper, and `MGDrivE`, a general-purpose mosquito gene-drive simulation package vendored inside a study-specific fork) between them illustrated the structural distinction the fix needed to draw: `BaBA`'s package sits at its repository's root with nothing else present — the package is the deposit, so nothing should be excluded — while `MGDrivE`'s vendored copy is nested one level deep inside the fork, with the study's own `Examples/` scripts and data sitting as real, undisputed siblings at the repository root.

The corrected rule preserves the original exclusion's logic for a nested package (still always excluded, on the assumption that a nested package always has the study's own files beside it; the `good` package, which sits alone in a subfolder, shows that this can fail, scienceverse/metacheck#505) but adds one check for a package detected at a repository's root: before excluding it, check whether anything outside the package folder classifies as real data or code, using `data_classify_files()` — the same rules-based classifier `data_check` itself uses elsewhere in the pipeline, rather than a separate, bespoke list of what counts as "real content." If nothing does, the package is not excluded at all, since there is no "outside" for the study's own materials to live in. One implementation detail required a second pass: an RStudio project marker file (`*.Rproj`) classifies as `"code"` under `data_classify_files()` purely by its file extension, which would have wrongly counted as "real content survives" for every package that ships one (nearly all of them) — an RStudio project file is packaging configuration, not an analysis script, so it is excluded from this specific check. The fix was verified against two new regression tests (a root-level package with nothing else present, confirming it is no longer excluded, and a nested vendored package alongside real sibling content, confirming the vendored files are still excluded while the sibling content survives) alongside the existing test suite, then reapplied to the 38 papers the corpus-wide scan had flagged.

Reapplying the corrected exclusion resolved 11 of the 16 confirmed cases outright. The remaining 5 continued to show `code_archived` disagreement. An earlier version of this manuscript attributed these to the package's `R/` directory never having been downloaded; a later direct check showed that explanation was wrong. The file listing was complete, but the corrected rule itself misfired: the rule made an exception for RStudio project files, as described above, but not for continuous-integration configuration files at the repository root (such as `.travis.yml` and `codecov.yml`), which are classified as code by their extension. The rule therefore concluded that real code survived outside the package and kept excluding the package's own source files. This was confirmed in detail for `biomonitoR` (238 files retrieved, 229 of them excluded) and reported as scienceverse/metacheck#469. Metacheck 0.3.3.3 fixed it by making the same exception for continuous-integration files; after a rerun of the affected papers with Metacheck 0.3.3.11, 16 of the 16 have their code found. Two of the 16, `phylomice` and `morton` — cases surfaced by the same corpus-wide scan but outside the original 13-paper set found during the initial regression check — resolved to, respectively, `METACHECK_RIGHT` (`phylomice`'s six R source files are a genuine methods contribution the paper's own text never explicitly

claims as "archived code," but which Metacheck is nonetheless correct to report) and `COOPER_RIGHT` (`morton`'s 37 R source files are unambiguously this paper's own phylogenetic-analysis code, which Cooper et al.'s coders had missed).

A.6 A systematic platform-coverage audit, three further metacheck fixes, and a complete categorization of every remaining repository-detection disagreement

A sixth round of work started from a single case the previous rounds had not caught: a paper citing a Dataverse installation (`borealisdata.ca`, Borealis, the Canadian national repository) under a DOI prefix Appendix A.5's Dataverse allowlist did not recognise. Tracing why surfaced a systemic problem in how that allowlist had been built, and prompted a full re-audit of every remaining "repository not detected" disagreement in the corpus rather than a single-paper patch.

A systemic under-sampling bug in the Dataverse DOI-prefix allowlist. Appendix A.5's per-host Dataverse DOI-prefix allowlist (`.dataverse_doi_prefix_hosts()`) was originally built by sampling the first 25 DOIs DataCite's search API returned for each host — a sample, not a census. This badly under-counted any host with more than 25 real DOIs registered, which turned out to be the majority of the 121 known hosts; `borealisdata.ca` alone has minted DOIs under six distinct prefixes, of which the original allowlist recorded only one. Rebuilding the allowlist using DataCite's facet aggregation (`disable-facets=false` on its `/dois` search, which returns every distinct prefix in use for a host, with its real count, in one request rather than a sample) found 100 distinct prefixes across the same 87 hosts, not 87 — including a second Harvard Dataverse prefix (`10.26300`, alongside the dominant `10.7910`) that the original sample had also missed. Verified live before and after: the DOI that surfaced this bug (`10.7939/DVN/M6Q2PE`, Borealis) now resolves correctly, and the second Harvard prefix now resolves to `dataverse.harvard.edu`. This was reported and merged upstream as scienceverse/metacheck#410.

Two further platform-coverage gaps found and fixed the same way. `.figshare_id()` did not recognise a Figshare article URL with no resource-type segment (`figshare.com/articles/<slug>/<id>`, as opposed to `figshare.com/articles/dataset/<slug>/<id>`) — an older, still-current URL shape (confirmed live against a real corpus citation) that `figshare_links()` already detected as a Figshare link but could not resolve to a usable article id, so `repo_check` never actually queried it (scienceverse/metacheck#412). Separately, `dataone_links()` had no entry for `data.piscoweb.org` (PISCO, a UC Santa Barbara-run DataONE member node), and `.dataone_pid()`'s bare-DOI regex could not handle a DOI suffix containing a further slash — PISCO's own DOI shape (`10.6085/AA/<id>`) — so this platform's citations, while real DataONE member-node DOIs, were invisible end to end. Both fixed and merged as scienceverse/metacheck#414, which also added 29 further institutional Figshare instances (28 found by searching DataCite's client registry for "figshare" in the client name, plus Monash University's `bridges.monash.edu`, found separately since its DataCite client is registered under the institution's own name) — each confirmed live, the same way as every host already

in `.figshare_vanity_hosts()`, to answer Figshare's own single-page-app shell rather than a real API response, and each institutional DOI prefix confirmed live to resolve to that exact host before being added.

Two matching bugs in this project's own comparison code, not in Metacheck. `03_recreate_cooper_columns.R`'s `.repo_url_platform()` — the function that labels which platform `mc_data_archive` reports for a paper — had a Dryad branch that only matched the single most common DOI prefix (`10.5061`), and a Figshare branch that only matched the literal `figshare.com/10.6084` domain and prefix, missing every one of the alternate Dryad prefixes and institutional Figshare instances Metacheck itself already recognised after the fixes above. The practical effect was a paper whose repository was now correctly found and retrieved still showing a `data_archive` disagreement, because the label fell through to the generic "Other repo/database" instead of naming the real platform. Both branches were rewritten to read directly from Metacheck's own `.dryad_doi_prefixes()`/`.figshare_vanity_hosts()`/`.figshare_doi_prefix_hosts()` rather than duplicating either list by hand, so this classification cannot drift out of sync with what Metacheck itself recognises again.

Reprocessing every paper these fixes could plausibly affect. Rather than assume the fixes above were the only remaining cause of repository-detection disagreement, every "repository not detected" comment still open in the worklist (351 individual (paper, column) disagreements across 233 papers, after excluding platforms already confirmed unsupported for an unrelated reason) was checked again: first narrowed locally, with no network calls, to the papers whose cited DOI prefix genuinely matched one of the platforms just fixed (27 candidates, after excluding two DOI prefixes that coincidentally collide with an unrelated platform's registered prefix in DataCite's own data — `10.6084`, Figshare's own prefix, and `10.17617`, Edmond's — neither of which any fix above touched), then live-verified one at a time. Of those 27, 13 were genuinely resolved by the fixes above; the remaining 14 were not — some detect the citation correctly but the dataset is embargoed or restricted at the source (confirmed via the repository's own API returning an explicit "cannot be viewed" message, not a Metacheck-side failure), and a few still do not detect the citation at all, for reasons unrelated to any fix in this round. A further, separately-run check across every remaining "never located" comment surfaced one exact repeat of Appendix A.1's original zip-peek limitation on a much larger scale (24 papers whose Zenodo/Figshare/Dataverse archive was never successfully peeked during an earlier corpus build, confirmed live, unmodified, to peek correctly today — a stale corpus row in every one of the 24, not a code defect). A further paper, initially suspected to be a second such stale row (its earlier 4TU fetch crash was already fixed in Appendix A.1 and appeared simply never rerun since), was checked directly against `rerun_disagreements_comparison.csv`'s own before/after values and found to still disagree after being rerun — the 4TU fetch failure was not, in fact, resolved. A later, more thorough recheck (Appendix A.9) traced the actual cause: 4TU is a genuinely supported platform, but its DOI-embedded identifier does not match the numeric article ID 4TU's own API requires, a distinct defect from the one Appendix A.1 fixed, so the paper was counted as `SUPPORTED_PLATFORM_BUG`, not `UNSUPPORTED_REPO`, in Appendix A.9's categorization. Since the rerun with Metacheck 0.3.3.11 (17 October 2026), Metacheck lists this 4TU record (7 files, 5 of them data) and finds the

paper's data. All resolved candidates from this round were rerun end to end (`repo_check`, `data_check`, `code_check`) and patched into the corpus; none of this round's rerunning touched a paper outside the specific set each check identified, so no regression check across the unaffected remainder of the corpus was needed the way Appendix A.5's broader platform-coverage changes required.

A.7 A full-corpus `reproducibility_check` run: method, outputs, reasons for non-reproducibility, and suggested improvements to Metacheck

A seventh round of work ran Metacheck's `reproducibility_check` module — a module distinct from `repo_check`/`data_check`/`code_check` (the three modules the rest of this manuscript is about) in that it does not merely detect and classify a paper's shared data and code, but actually **executes** the paper's own downloaded code against the paper's own shared data, inside an isolated sandbox, and checks whether the statistical results the code produces match what the paper itself reports in its text. This appendix documents the method, the files this run produced, a full breakdown of why papers were and were not reproducible, two Metacheck defects this run surfaced (both reported upstream, one fixed on `dev` and used to produce the final result for the one paper it had stopped), and a set of concrete suggestions for further improving `reproducibility_check` itself.

A.7.1 Method

`reproducibility_check` was run with `code/run_metacheck.R` on the 435 papers in which an earlier module run had found runnable R or Python code (the subset of the 1,861-paper corpus with code to execute; the other 1,426 papers have no code for the module to run, and are not discussed further in this section). For each paper, the module: (1) reuses `code_check`'s dependency extraction and `psychds_check`'s path-rewrite plan to materialise a throwaway copy of the paper's own Psych-DS-layout data and code; (2) installs the paper's declared package dependencies (CRAN, GitHub, Bioconductor, or URL sources) into a throwaway library inside a Docker container, starting from a pre-built image (`ghcr.io/scienceverse/metacheck_r_large` for R, `ghcr.io/scienceverse/metacheck_py_large` for Python) in which the most common packages found across a prior corpus scan are already installed, so most papers skip most of the install step; (3) runs each of the paper's scripts, in an automatically-derived dependency order, each in its own Docker container, with the filesystem read-only outside the materialised sandbox — a genuine containment boundary for running downloaded, unvetted third-party code; and (4) extracts any statistical output the executed scripts printed to the console and compares it against the paper's own reported results (`match_reported_output()`), recording how many of the paper's own reported test statistics the executed code actually reproduced.

The run used Metacheck commit `f6aec07c` (version 0.4.3.6), three papers at a time (`--workers 3`), each container limited to 4 GB of memory and a share of 6.6 processors, a 3,000-second

time limit per script and 3,600 seconds for all of a paper's scripts together, a 1,200-second limit on installing a paper's packages, a 1,000 MB limit on any single file downloaded, and internet access allowed during execution (so a script that fetches a file over the network could still run, at the cost that what it downloaded may differ from what the authors had). The scripts could use the internet up to 1,000 MB per file and 2,000 MB for all of a paper's scripts together. Each paper's complete result was written to its own file the moment that paper finished, so a run interrupted partway through — which happened several times, because the drive holding the sandbox and the downloaded data repeatedly filled — did not lose the papers that had already completed; restarting skipped every paper with a saved result and ran only those still outstanding.

Two of the 435 papers were run on a different Docker image or a different Metacheck version than the other 433, for reasons given in A.7.5, and each paper's result records which run and which commit produced it (`run_info$paper_runs`): 58 papers whose code needed a package that would not compile for lack of a system header file were run again on an image with those headers added (`ghcr.io/scienceverse/metacheck_r_large:headers-2026-11-07`), and one paper (10.1111/2041-210X.13928) whose reproducibility check the main version could not complete at all was run on the `dev` branch (commit with the fix for that defect, version 0.4.3.13). The matching of reported results to the code's output (step 4 above) was computed afterwards from the saved console output with `code/match_reported_results.R`, run once with the `dev` version for every paper, without running any script again.

A.7.2 Outputs stored

The run wrote one `.rds` file per paper, each holding that paper's complete result across every module, into a run folder on the drive used for the run. `code/splice_reproducibility.R` combined the main run with the two follow-up runs named above into `data/metacheck_results.rds`, the single file every number in this manuscript is computed from; it records, for each paper, which of the three runs and which Metacheck commit produced its reproducibility result. Each paper's own result carries the per-script table (one row per script attempted, with its `outcome`, the one-word `reason` the module assigned a script that did not run through, the file paths the module rewrote, the `setwd()` calls it removed, and the packages it declared and installed) and the per-paper summary row (`repro_code_n`, the count of scripts that were run as analyses, the per-reason failure counts, and the reported-versus-reproduced statistic counts).

A.7.3 Scope: which papers had code to check at all

Of the 1,861 papers in the corpus, 435 (23.4%) had at least one runnable code file for `reproducibility_check` to attempt (`repro_code_n > 0`); 1,426 (76.6%) had none, and are not further discussed in this section, since there is nothing for the module to execute — this is the same "does the paper share any code" question `code_check/repo_check` already answer

elsewhere in this manuscript, not a new finding. The reproducibility run therefore covered these 435 code-bearing papers.

Note on this count. Every number in this section is computed on the same scheme as the body of the manuscript: a file is counted among the scripts that were run only if the module ran it as an analysis of its own. The files it did not (a file run by another through `source()` or an import, a file that only defines functions, a JAGS or BUGS model file, a file that is not code in a language the module runs, and a file not downloaded) are counted and named apart. The comparison of reported results with the code's output was computed afterwards from the saved output, without running any script again.

A.7.4 Reasons for non-reproducibility: a full breakdown

Across the 435 code-bearing papers, 2,874 scripts were run as analyses, 607 of them Python (`.py` or `.ipynb`) and the rest R. Not every code file is such a script: a further 641 files are run by another script through `source()` or an import and have no result of their own, 217 only define functions, 11 are JAGS or BUGS model files, 2 are not code in a language the module runs, and 10 needed a data file too large to download; these are counted apart and are not included below. Of the 2,874 scripts that were run, 626 (21.8%) ran without an error. Their outcomes:

Outcome	Count	% of 2,874
`ran_ok` (ran without an error)	626	21.8%
`errored` (ran, but raised an error)	1,391	48.4%
`dependency_unavailable` (a package it needs could not be installed, so it was not run)	269	9.4%
`needs_a_person` (the code asks a person to choose a file, so it cannot run unattended)	285	9.9%
`skipped_time_limit` (not started, because the paper's time had run out)	148	5.1%
`not_parsed` (the file is not valid syntax in its language)	52	1.8%
`timed_out` (still running at the 3,000-second cutoff)	49	1.7%
`download_too_large` (a file it needs was over the download limit)	24	0.8%

Outcome	Count	% of 2,874
`out_of_memory` (stopped by the memory limit)	21	0.7%
`needs_internet` (needs network access the sandbox blocks for that host)	9	0.3%

At the paper level, only 41 of the 435 code-bearing papers (9.4%) had every one of their scripts run without error; 222 (51.0%) had zero scripts run successfully at all; the remaining 135 (31.0%) had a mix of successful and failed scripts within the same paper.

Why the 2,248 scripts that did not run through failed. The module records a one-word reason for each; the same reasons are reported, with their counts, in the body of the manuscript. The largest is a file the script needs that was not found (632 scripts, in 160 papers): the authors did not archive it, it is produced by an earlier step that is not archived, or the name in the code could not be matched to a file in the deposit. Next is a catch-all for an error that fits none of the named reasons (452), then a package that could not be installed (376), code that asks a person to choose a file (285), the time limit (197), an object or function that is not defined (178, the signature of a script meant to run as one step of a larger pipeline), a file that is not valid code (56), a difference in the software environment (39), a download too large (24), and a script that needs the internet (9).

Two directly observed patterns are worth naming, because both argue the code was never meant to run unattended: a script that calls `file.choose()` to open an interactive file-picker, which cannot work in a headless container (the module records these as `needs_a_person`), and a script that starts a blocking viewer such as `rstanarm::launch_shinystan()`, which waits for a browser connection that never arrives and is stopped only by the time limit.

Dependency installation. Before running a paper's code, Metacheck installs the R or Python packages that code declares it needs, from a Docker image in which the most common packages are already present. Across the 435 papers, 190 package installs failed, in 75 papers, naming 144 distinct packages (most failing in a single paper). The module records, for each, a classified category: the largest group is a package found on neither CRAN nor Bioconductor and not anywhere else the module looks, because the code does not say where it comes from; the next are a package that fails to compile, and a package no longer available on CRAN for the version of R the run used. The packages that needed a system development library the image did not contain (the GDAL/GEOS files behind `rgdal`, `rgeos` and `gdalUtils`, and the compression headers behind several Bioconductor packages) are not in these counts, because the 58-paper rerun on the image with those headers added (A.7.5) installed them; what remains is the set no added header could fix.

In 40 papers, a package failed to install and no script of the paper ran successfully — the subset where a failed install is most clearly the reason the code did not run. For these, giving

the Docker install backend the CRAN Archive retry the other backend already has (A.7.6) could recover a share of the packages that were removed from CRAN but still have a version in the Archive.

Result matching: did the code's own output match what the paper reports? Beyond running a paper's code, `reproducibility_check` extracts each statistical result the paper's own text reports (a t-test, a correlation, a regression coefficient) and checks whether that value appears in what the executed code printed (`match_reported_output()`). This is the closest the run comes to asking not just "does the code run" but "does running it get the paper's own numbers." It also reads the result tables inside a self-reproducible `.jasp/.omv/.spv` file (JASP, jamovi, or SPSS Viewer), which carry data and analysis together and need no code execution, and feeds them into the same step.

208 papers had at least one statistical result the module could both extract from the paper's text and attempt to match against the code's output; across those papers, 1,323 reported statistics were checked, of which 281 (21.2%) were matched. At the paper level, 6 of the 208 papers (2.9%) had the code reproduce every reported result checked, 158 (76.0%) reproduced none, and the remaining 44 (21.2%) reproduced some but not all. This is best read together with the outcome breakdown above rather than as a claim about the research: a script has to run to completion and print a value before that value can be compared to the text at all, and only 21.8% of scripts, and 9.4% of papers in full, cleared that first bar.

The matching step draws one further distinction worth keeping. Of the 281 matched values, 262 (93.2%) matched at full confidence (every component of the reported test matched together) and 19 (6.8%) only partially; when the code did reproduce a value, it usually reproduced it completely. And of the 1,042 values not matched, 670 were nonetheless traced to a real quantity the code computed under a different grouping than the paper's text suggested — a materially less serious finding than a value the code never produced at all.

A note on scope and what this section does not claim. These numbers describe whether a paper's own downloaded code, as shared, ran and reproduced its own reported results in this execution environment, with its pre-installed package set, its 3,000-second per-script budget, and the internet available during execution. A script failing here is not evidence the paper's research is wrong, or even that the code never worked — it may never have been meant to run outside the authors' own machine, as the `file.choose()` and "designed as an interactive pipeline" patterns above show directly. A full per-paper review of why each failed script failed, in the style of Appendix A.1–A.6's disagreement-by-disagreement review, is listed as a further step in A.7.6 rather than attempted here.

A.7.5 Infrastructure failures, found and fixed during this run

Running `reproducibility_check` on all 435 code-bearing papers surfaced two defects in Metacheck itself — neither related to a paper's own code or data — and two operational

problems with the run's own environment that required part of the corpus to be run again. Both defects were filed as issues against `scienceverse/metacheck`.

scienceverse/metacheck#597 — a file name is not escaped before it is used as a regular expression, which stops the module for a paper. When the module rewrites a script's file paths to point at the downloaded files, it builds a search pattern from each file name, but the code that was meant to escape characters with a special meaning in a regular expression used a form that, without Perl mode, escaped nothing. A file name containing a Windows path with a backslash followed by a letter (for example `\P` or `\s`) therefore produced an invalid regular expression, which stopped the whole reproducibility check for that paper before any script was run. One paper in this corpus hit it (10.1111/2041-210X.13928, whose code names the path `D:\AviaNZ\Sound_Files\...`). The defect was fixed on the `dev` branch (the four affected call sites now use the escape function that works), and this one paper was run again on that version (0.4.3.13) to produce its reproducibility result, which is the result reported here for it. **Confirmed fixed.**

scienceverse/metacheck#598 — an install address in a commented-out line is used instead of CRAN. For one paper (10.1111/1365-2435.13410), the module installed a package from a web address that appears only inside a commented-out line of the paper's own code, rather than from CRAN; the address was dead, so the install failed. The defect is that the dependency extractor reads an install address out of a line the R interpreter would never run. This affects which source a package is installed from, not whether the module runs, so it did not stop any paper. **Filed, open.**

Fifty-eight papers run again on an image with system header files. The pre-built R image used for the main run did not contain the C and C++ header files that some packages need in order to compile from source (for example `zlib.h`, needed by several Bioconductor packages, and the GDAL/GEOS development files needed by the spatial packages `rgdal`, `rgeos` and `gdalUtils`). A package that needs such a header could not install, so the scripts that load it could not run. After the main run, the project built an image with these development files added, confirmed in it the packages that the main run could not install now did, and ran again, on that image, the 58 papers whose code needed one of those packages. Every package the new image was meant to provide then installed; the packages that still failed are ones no added header could fix (a package with no findable source, one removed from CRAN, or one that fails to compile for an unrelated reason). Each of these 58 papers' reproducibility result in the combined output is the result from this second run, recorded as such in `run_info$paper_runs`.

Repeated interruptions from the run drive filling. The drive holding the sandbox, the downloaded repositories and Docker's own data repeatedly ran low on free space during the run, because a paper that was stopped — by its time limit, by a stall, or by the low-space guard itself — left its unpacked copy of the deposit behind, and these accumulated across the run. The run has a built-in guard that starts no new paper below 10 GB of free space and stops the running papers below 5 GB, so each time the drive filled the run stopped itself rather than failing in some uncontrolled way; it was restarted after the leftover copies were

deleted, and because each paper's result is saved the moment it finishes, every restart skipped the papers already done and ran only those outstanding. This is a property of the environment the run was carried out in, not of Metacheck or of any paper's code, but it is the reason the run was completed in several stretches rather than one.

A.7.6 Suggestions for improving `reproducibility_check`

The following suggestions are drawn directly from running the module on all 435 code-bearing papers; each is grounded in a specific, confirmed observation from this run, not a general aspiration.

1. **Ship the common system libraries and header files in the Docker image.** The single largest avoidable cause of a failed install this run found was a package that needs a C or C++ development library the image did not contain: the GDAL/GEOS/PROJ development files behind the spatial packages (`rgdal`, `rgeos`, `gdalUtils`), and the compression and other headers behind several Bioconductor packages. This run had to build a second image with these added and run 58 papers again on it (A.7.5), after which every package the new headers covered installed. Adding these development libraries to the image Metacheck ships by default would let every future spatial or Bioconductor paper install its dependencies on the first attempt, rather than recurring as a per-paper failure.
2. **Give the Docker install backend the same CRAN Archive retry the process backend has.** When a package's current CRAN release fails to install but its last-published version in the CRAN Archive would install cleanly, the process backend (`repro_install_deps()`) retries against the Archive before giving up; the Docker backend (`repro_install_deps_docker()`) does not. Of the 90 packages that failed to install in exactly one paper each this run, 35% of the classified failures were a compilation failure and a further 6% were a package no longer on CRAN — some of which an Archive retry might recover. This would recover a share of the one-off install failures without any change to a paper's own code.
3. **Add a distinct outcome for a script that needs interactive input.** A script that calls `file.choose()` (an interactive file picker) or starts a blocking viewer such as `launch_shinystan()` cannot run unattended, but surfaces as an ordinary error or, worse, a script that sits until its time limit. The module already has an `interactive` reason for the clearest of these; broadening it to recognise a short list of known-interactive calls before execution, and distinguishing a script that produced no output before timing out, would let a report separate "this code cannot run unattended by design" from "this code is simply slow" — two very different findings for an author or reader.
4. **Extend the paper-by-paper qualitative review method of Appendices A.1–A.6 to `reproducibility_check`'s own output.** This appendix reports the rate and category of non-reproducibility across the 435 papers, but does not individually verify a sample of

the failed scripts against their own paper's repository the way A.1–A.6 reviewed each `repo_check`/`data_check`/`code_check` disagreement. A systematic sample of the `other` category in particular (452 of the 2,248 failed scripts, the module's catch-all for an error that fits none of the named reasons) would likely surface further mechanically fixable causes, in the same style A.1–A.6 already found for the other modules.

A.8 Three further reliability fixes to compressed-archive handling, and their effect on `data_archive/code_archived/code_download`

An eighth round of work returned to a structural limitation named repeatedly in earlier appendices but never itself directly fixed: `zip_peek()`'s inability to read a zip's contents without downloading it on Dryad and Figshare, both of which redirect a file's download URL to Amazon S3. Diagnosing and fixing this surfaced two further, closely related defects in the same download path, and reprocessing every paper the three fixes together could plausibly affect closed most, but not all, of the remaining `code_download` disagreement this comparison's own numbers above already reflect.

`zip_peek()` failing on S3-backed hosts (scienceverse/metacheck#424). `zip_peek()` reads a remote zip's table of contents with two requests: a `HEAD` to learn the file's total size, then a byte-range `GET` for the tail bytes holding the archive's central directory. Amazon S3 — the storage backend both Dryad's and Figshare's own download URLs redirect to — answers the size-only `HEAD` request with 403 but answers the byte-range `GET` normally (confirmed live: a `HEAD` against a real Dryad-hosted file returned 403, while a `Range: bytes=0-100 GET` against the identical URL returned 206 with the file's true total size in its own `content-range` header). Because `.http_range_tail()`, the function `zip_peek()` calls to fetch the tail bytes, only sends its range request once `HEAD`'s size is known, a `HEAD` failure meant the byte-range request was never sent at all — not just for Dryad and Figshare but, confirmed the same way, for Harvard's Dataverse installation, another host redirecting to S3. Fix: fall back to a suffix range request (`Range: bytes=-<n>`) when `HEAD` fails, reading the total size from that response's own `content-range` header instead. Confirmed live before and after against nine archives sampled from this corpus (three each on Dryad, Figshare, and Zenodo): all six Dryad/Figshare archives returned `NULL` from `zip_peek()` before the fix and a correct, complete listing after it, while the three Zenodo archives (not S3-backed, and so unaffected by the underlying bug) listed correctly both times.

`zip_peek()` never persisting its result across a restart (scienceverse/metacheck#427). A separate, longstanding defect (`repo_info_cache()`) already exists specifically to persist a repository's listing (which files it has) to disk, since restarting an interrupted corpus run otherwise re-pays every repository's listing cost from scratch — a real problem previously severe enough to exhaust Dryad's own daily request quota (see that fix's own code comment, confirmed live 2026-09-08: 11 restarts of one 50-paper batch cost 440+ re-listing requests against a workload that genuinely needed a handful). `zip_peek()`'s own result — a zip's contents, a separate cost from the repository listing above it — had no equivalent: it was

cached only in memory, for the life of one R process, so the same restart pattern silently re-peeked every zip already peeked, and `repo_check` and `data_check` each peeked the same zip independently (as two separate R processes in this project's own pipeline), doubling the cost again. Confirmed directly during this project's own corpus rerun (below): before this fix, four consecutive restarts each re-hit the same Dryad rate limit at the same early batch, at increasingly long waits (24.5, then 21.5, then 16 minutes remaining); after it, a restart at the same point in the run loaded every already-completed batch from disk in under 20 seconds and progressed past the previously-stuck batch with no rate-limit wait at all. Fix: persist `zip_peek()`'s result to disk the same way `repo_info_cache()` already does for a listing, gated on the same `cache = TRUE` argument.

A silently discarded error in `download_repo_files()`'s archive-member fetch (scienceverse/metacheck#429). Once `repo_check`'s zip-peek expansion lists a file living inside an archive (an "archive member," carrying `archive_url`+`archive_member` rather than its own `file_url`), `download_repo_files()` fetches it via a dedicated byte-range path (`.zip_fetch_members()`/`.zip_member_fetch()`), wrapped in a bare `tryCatch(..., error = function(e) NULL)` with no logging of any kind. A transient failure at this one step — a dropped connection, a timeout, a momentary host error — left `file_location` permanently `NA` with no warning, no log entry, and no record anywhere that a download had even been attempted, indistinguishable after the fact from a member the host genuinely cannot serve. This surfaced directly in this project's own corpus rerun: after #424 above let `repo_check`/`data_check` correctly list many more Dryad/Zenodo archive members than before, 24 papers whose code repository had previously never been detected at all (`mc_code_archived FALSE`) newly showed `mc_code_archived TRUE` but `mc_code_download` still `FALSE` — the code files were now correctly found, but never downloaded. Tracing one case in full (a small `.R` file inside a Dryad-hosted zip) confirmed the underlying fetch mechanism worked perfectly when called directly, with the paper's own exact saved row data and arguments, ruling out a logic bug; the corpus run's own 17,661-line warning log had no entry at all for this archive, this paper, or any of the other 23, confirming the failure really was silently discarded rather than merely unlogged elsewhere. Fix: record the specific error `.zip_fetch_members()` raises, the same way the ordinary `file_url` download path a few lines below already does, rather than discarding it.

Reprocessing the 24 code-download regressions. All 24 papers' `code_check` output was rerun against `dev` after #429's fix (`code_check` alone — `repo_check`'s listing and `data_check`'s own downloads were unaffected, confirmed directly: `mc_data_download` was already correctly `TRUE` for the papers where a data download was expected). 13 of 24 resolved immediately. The remaining 11 were rerun a second time with `skip_on_api_limit` set `FALSE` for this small, targeted rerun (unlike the corpus-wide default `TRUE`, whose own rationale — never blocking an unattended, hours-long, 1,861-paper run on a host's quota reset — does not apply to an 11-paper rerun, and, confirmed directly, was itself the reason these 11 still failed: their Dryad archive fetch had landed during a rate-limit window and been silently skipped rather than waited out, and waiting the quoted remaining time fetched the exact right file on the first attempt); 4 more resolved. The final 7 each printed a `download_repo_files()` retry-exhaustion message

(`Failed to perform HTTP request`, `Re-run to retry: cached files are reused, only the missing files are fetched`) that — a `message()`/`cat()` call, not a `warning()` — this project's own rerun script's `warning =` handler could not catch or log, a message-routing gap distinct from #429's own fix and not itself corrected here; rerunning those 7 a third time, as the message itself instructed, resolved all 7. Every one of the original 24 papers now downloads successfully.

Corpus-wide effect. With these three fixes installed and the full 1,861-paper corpus rerun, sensitivity, specificity, and the number of papers newly entering each comparison (a repository or file that previously could not be evaluated at all now can be) all moved together in the direction these fixes targeted. `code_archived` agreement rose from 84.5% to 87.9% (sensitivity 60.1% to 71.2%), `any_readme` from 69.8% to 72.9% (sensitivity 59.6% to 64.2%), and `data_availability` from 83.3% to 85.5%. `code_download`'s own comparable count grew from 342 to 404 papers (50 papers whose code could not previously be evaluated for a download outcome at all — because it had never been detected as archived in the first place — now can be), and at that point only 2 disagreements remained across that larger, harder set (down from 24 when these three fixes began). After a later targeted rerun of every paper with an archive member that had not been downloaded, this time waiting out host rate limits (see the `code_download` results in the main text), 1 `code_download` disagreement remains (99.8% agreement), a case where Metacheck is right and Cooper et al.'s coding missed the code. `zip_not_peeked` — the disagreement cause tracing directly to #424's own limitation — dropped to 0 as a recorded cause across every construct in this comparison's disagreement-cause categorization. The "compressed archive never opened" category among `data_archive`'s remaining under-coded papers (Appendix's own `data_archive_uncoded_papers.RData` analysis) fell from 70 papers to 12; the remainder now trace to distinct, already-documented causes (a code-only or documentation-only repository, a non-tabular file format `data_check` does not classify as data, or an image/GIS/3D file). Every previously-recorded disagreement in the corpus was re-reviewed the same way as every earlier round: 1,070 of 1,174 prior verdicts carried forward unmodified, because their underlying (paper, column, Cooper value, Metacheck value) triple was unchanged; every disagreement whose Metacheck-derived value changed, or which was newly surfaced, was reviewed from scratch (43 rows immediately after the corpus rerun above; 19 once the 24 papers' `mc_code_download` values were updated by the reprocessing described above and the comparison data rebuilt a second time) against the same evidence-source rule used throughout this manuscript (repository file listing and `repo_metadata` for `data_license`/`code_archived`/`any_readme`/`code_download`, the paper's own text for `data_availability`/`data_archive`). In the current worklist, all 876 disagreeing papers carry a verdict for every column on which they disagree.

Archive downloads that failed during a corpus run, October 2026. At this stage 25 `code_download` disagreements shared one cause: Metacheck found the paper's code files, all inside a zip archive on Dryad or Zenodo, but none of these archive members was downloaded in the corpus run, so `code_check` had no local file to inspect. The run recorded no reason for the failed downloads. A live recheck of these papers on 10 October 2026, with the same Metacheck version, retrieved a code file from each archive at exactly the size listed in the archive, so the

code was downloadable and the failure was specific to that run rather than to the repositories. This is the same archive-member download pattern as scienceverse/metacheck#429. We reran Metacheck's repository, data, and code checks for all 191 papers with at least one archive member that had not been downloaded, this time recording the outcome of every download and waiting out host rate limits instead of skipping the file. The code of all 25 papers then downloaded. Host rate limits are the most plausible cause of the original failures, although that run recorded no reason: in the rerun, the last paper whose code still failed did so because Dryad refused the requests as too many (HTTP 429), and its code downloaded once the limit was waited out.

A.9 A full, paper-by-paper review of every "unsupported repository, not detected" disagreement, and a full recheck of Appendix A.6's original categorization

Appendix A.6 categorized every "repository not detected" disagreement in the corpus, but 40 of the 161 papers whose only Cooper-coded `data_archive` label(s) fell outside Metacheck's supported-platform categories, and for which `repo_check` consequently found no repository at all, had not yet been individually checked one at a time when an earlier version of this manuscript's main text was written. This appendix documents that remaining review in full: each paper's own extracted data-availability text, the specific DOI or URL cited, whether it was live-verified to resolve, whether the cited platform is genuinely supported by Metacheck (checked directly against the vendored source in `code/data/reproducibility_check/vendor/metacheck/R/`, not inferred from this manuscript's own earlier prose, which had itself gone stale for at least one platform – see below), and the final category assigned. All 40 papers received a verdict at the time of this review; 6 have since had their underlying `repo_check` defect fixed by later work elsewhere in this project and no longer appear in the live disagreement set at all (confirmed: `repo_check` now finds a repository for every one of them), so Table 2 below shows the 34 that remain.

This review also surfaced a further risk worth checking directly rather than assuming away: Appendix A.6's original 230-row categorization was itself produced before the platform-support defects this review found (the stale 4TU/DANS assumptions, an incomplete dedicated-platform list, a missing DSpace-legacy host count), so any of those original 230 rows that happened to cite one of these same platforms could be mislabeled the same way. Rather than only check the platforms these specific bugs touch, every one of the original 230 rows was rechecked from scratch: each paper's own text was re-extracted directly from the corpus (paragraph-grouped, not sentence-truncated, avoiding the extraction bug documented earlier in this manuscript), every DOI and URL in that text was identified, and each one's host was checked directly against Metacheck's current supported-platform functions – not against what an earlier comment claimed was or was not supported.

Of the 230, 21 candidate citations resolved to a host now confirmed supported despite the row's original `UNSUPPORTED_REPO` label; after excluding false positives (a truncated GitHub URL with no repository path, a citation that names a genuinely unrelated, unsupported host

despite superficially matching a regex), 9 rows were corrected to `SUPPORTED_PLATFORM_BUG` or `SUPPORTED_PLATFORM_MISSED` at the time of this review. The corrected rows span three recurring patterns: a citation to a genuinely supported host that `repo_check` simply never queried (Dataverse, DataOne/KNB, Zenodo, standard-format Figshare), a supported platform cited via a URL format its matcher does not parse (a Figshare private share-link or collection URL), and a supported platform cited via a non-standard DOI prefix its prefix-matching does not recognise (an alternate Dryad EZID prefix, a non-standard Figshare prefix). None of the corrected rows involve a platform Metacheck genuinely cannot reach at all – every correction is a detection defect on an already-supported platform, consistent with the pattern the original 40-paper review above found. 8 of these 9 papers have since had their underlying defect fixed by later work elsewhere in this project (the same `repo_check`-now-finds-it pattern as above) and no longer appear in the live disagreement set; Table 1 below shows the 1 still remaining in it, with the full original set described narratively here regardless.

Table 1: The rows from Appendix A.6's original 230-paper categorization corrected on full recheck, from `UNSUPPORTED_REPO` to a supported-platform category, still present in the current disagreement set.

Paper	Category	Evidence and verification
10_1111_1365_2435_14709	SUPPORTED_ PLATFORM_BUG	Same root cause: the private Figshare share-link format was not recognized by repo_check. RECHECKED: figshare.com is a fully supported platform; the defect is specifically that a private/anonymous Figshare share-link (figshare.com/s/...) uses a different URL format from the standard figshare.com/articles/... pattern repo_check's Figshare matcher expects, so the host itself is supported but this specific link format is not recognised -- a supported-platform bug, not an unsupported platform.

Two platforms cited in this set turned out to be genuinely supported by Metacheck despite `repo_check` finding nothing, each traced to a specific, distinct defect rather than a missing backend. **4TU** (`data.4tu.nl`) has dedicated support (`archive-4tu.R`), and the DOI cited (`10.4121/uuid:04b25e70-ca3e-489f-9510-36385c1a4104`) live-resolves correctly to a real 4TU record. The ID-extractor correctly pulls the UUID from the DOI, but the live 4TU API lookup queries `data.4tu.nl/v2/articles/<uuid>`, which 404s, because 4TU's own DOI redirect resolves the identical record to a different, numeric article ID that the UUID does not match. This corrects an earlier, incorrect statement elsewhere in this manuscript (Appendix A.6) that this paper's 4TU fetch failure remained unresolved and should count as `UNSUPPORTED_REPO`; it is a supported-platform bug, not an unsupported platform. Since the rerun with Metacheck 0.3.3.11 (17 October 2026), Metacheck lists this record's files and finds the paper's data. **DANS EASY** is one of four separate subject-specific Dataverse installations DANS operates (archaeology/lifesciences/phys-techsciences/ssh), all sharing a single DOI prefix (`10.17026`). Metacheck's `.dataverse_host_from_doi()` resolves any DOI under this shared prefix to `archaeology.datastations.nl`, but the paper in this set cites a dataset actually hosted on `lifesciences.datastations.nl` – a genuine Metacheck defect in DOI-to-host resolution for any DOI prefix shared across multiple Dataverse instances, not specific to DANS. A third

paper cites the Knowledge Network for Biocomplexity (`knb.ecoinformatics.org`), a confirmed DataOne member node whose DOI live-resolves correctly; the specific reason `repo_check` did not detect it was not further root-caused within the scope of this review, so it is recorded as a supported-platform miss without a fully diagnosed cause, distinct from the two fully diagnosed bugs above.

Table 2: Full paper-by-paper review of the 40 previously-unreviewed 'unsupported repository, not detected' disagreements: cited platform, live DOI/URL resolution status, and final category.

Paper	Category	Evidence and verification
10_1002_pan3_10149	NO_REPO_DATA_ RESTRICTED	Paper text: 'Our raw data are not publicly available for confidentiality reasons, as they contain social, cultural and language data that may not be interpreted outside of their Nyikina context. They will be archived with Griffith University and Walalakoo Aboriginal Corporation.' No repository URL or DOI is given anywhere in the paper -- data are explicitly withheld (Indigenous data sovereignty). There is nothing for repo_check to find; this is not a Metacheck gap of any kind.
10_1111_1365_2745_12930	UNSUPPORTED_REPO	Paper text: 'Data associated with this paper have been deposited in the Environmental Climate Data Sweden depository https://ecds.se/dataset/species-distribution-modelling-data-for-23-dead-wood-inhabiting-bryophytes'. Live-verified the URL resolves (redirects to researchdata.se). researchdata.se/ecds.se is not in any of Metacheck's supported host lists (not Dataverse, DSpace7, DataOne, Figshare, and has no dedicated single-platform module).
10_1002_pan3_10307	UNSUPPORTED_REPO	Paper text: 'Data and code used in the analyses are available at ScienceBase (https://doi.org/10.5066/P9KBZW0G and https://doi.org/10.5066/P9CDA1UN, respectively)'. Live-verified both DOIs resolve to www.sciencebase.gov catalog items. ScienceBase (USGS) is not in any Metacheck supported host list and has no dedicated module.
10_1111_1365_2745_13067	TEXT_EXTRACTION_ TRUNCATED	Extracted text is a mid-sentence fragment: '...net online repository (Lopez-Gonzalez, Lewis, Burkitt, & Phillips, 2011; ...)'. No DOI or URL is present in the extracted text at all -- the actual data-availability statement (likely referring to ForestPlots.net, based on the citation) was not captured by GROBID extraction.
10_1111_1365_2435_14226	UNSUPPORTED_REPO	Paper text: 'The data that support the findings of this study are available on the National Tibetan Plateau Data Center https://doi.org/10.11888/Terre.tpdc.272843'. Live check: this DOI request failed to resolve (network/redirect issue querying data.tpdc.ac.cn from outside China is common). Regardless, tpdc.ac.cn is not in any Metacheck supported host list and has no dedicated module.
10_1111_1365_2745_14315	UNSUPPORTED_REPO	Paper text: 'Novel data collected in this study and compiled from FRED are available through FRED: https://doi.org/10.25581/ornlsfa.014/1459186'. FRED (via ORNL's Sustainable Forestry/SFA data archive) is a niche, domain-specific archive with no entry in any Metacheck supported host list and no dedicated module. Live DOI resolution failed (ornlsfa.014 DOI prefix appears not to resolve through doi.org currently), which is a separate, secondary problem on top of the platform simply being unsupported.
10_1111_2041_210x_14453	UNSUPPORTED_REPO	Paper text: "FishScale' software ... can be accessed at https://doi.org/10.5066/P13QVR2R. Data available via https://doi.org/10.5066/P9E1DTYB'. Live-verified: the code DOI resolves to code.usgs.gov (a self-hosted GitLab instance on USGS's own domain -- explicitly out of scope per Metacheck's own GitLab module documentation, which matches only gitlab.com); the data DOI resolves to www.sciencebase.gov (unsupported, as elsewhere in this set). Both citations point to unsupported hosts.

Paper	Category	Evidence and verification
10_1111_1365_2745_13529	NCBI_GENBANK	Paper text: 'Data were deposited in the Sequence Read Archive (https://www.ncbi.nlm.nih.gov/bioproject/PRJNA592420)' and 'Sequence data are deposited at NCBI-SRA under the accession number PRJNA592420.' A domain-specific bioinformatics accession (SRA/BioProject), a distinct category from an ordinary data/code repository and outside repo_check's scope by design.
10_1002_pan3_10743	TEXT_EXTRACTION_TRUNCATED	Extracted text is entirely a philosophical/conceptual discussion of aesthetics and definitions of 'mess' -- no data-availability statement was captured anywhere in the extracted text for this paper.
10_1111_1365_2664_12991	TEXT_EXTRACTION_TRUNCATED	Extraction returned an empty string -- no text was captured for this paper's data-availability section at all.
10_1002_2688_8319_12241	TEXT_EXTRACTION_TRUNCATED	Extraction returned an empty string -- no text was captured for this paper's data-availability section at all.
10_1111_1365_2745_13517	TEXT_EXTRACTION_TRUNCATED	Extracted text covers only methodological discussion of pollen/charcoal source areas -- no data-availability statement with a URL or DOI was captured anywhere in the extracted text for this paper.
10_1002_pan3_10677	TEXT_EXTRACTION_TRUNCATED	Extraction returned an empty string -- no text was captured for this paper's data-availability section at all.
10_1111_1365_2745_13154	UNSUPPORTED_REPO	Paper text: 'Data available through Digital Commons @ Georgia Southern repository https://doi.org/10.20429/data.2019.01'. Live-verified the DOI resolves (HTTP 200) to digitalcommons.georgiasouthern.edu. A generic bepress Digital Commons instance, not in any Metacheck supported host list.
10_1111_1365_2745_12749	TEXT_EXTRACTION_TRUNCATED	Paper text ends mid-sentence: 'Data, including location, growth, and iWUE data are located at ScholarsArchive@OSU' -- the sentence is cut off before any URL or DOI appears, so no identifier was captured for this citation despite a named repository being mentioned.
10_1111_1365_2435_14477	TEXT_EXTRACTION_TRUNCATED	Extraction returned an empty string -- no text was captured for this paper's data-availability section at all.
10_1111_1365_2664_13892	UNSUPPORTED_REPO	Paper text: 'The entire dataset that supports the findings of this study is available via a permanent open access data repository of the University of Vienna http://phaidra.univie.ac.at/o:1168756'. Live-verified the URL resolves (HTTP 200). Phaidra (University of Vienna's repository platform) is not in any Metacheck supported host list and has no dedicated module.
10_1111_2041_210x_13110	TEXT_EXTRACTION_TRUNCATED	Extracted text covers only general methodological discussion of uncertainty modelling -- no data-availability statement with a URL or DOI was captured anywhere in the extracted text for this paper.
10_1111_1365_2745_14252	TEXT_EXTRACTION_TRUNCATED	Extraction returned an empty string -- no text was captured for this paper's data-availability section at all.
10_1111_1365_2664_14700	UNSUPPORTED_REPO	Paper text: 'Data are available from the OFB Digital Repository https://ged.ofb.fr/share/s/XiFQtejFQy28DG6u0TDzoA/folder'. Live-verified the URL returns HTTP 403 (access-restricted share link) on a custom OFB (French biodiversity agency) Nextcloud-style file-share instance. Not in any Metacheck supported host list and has no dedicated module; a bare access-controlled folder link is also not the kind of structured repository API Metacheck's dedicated modules are built to query even where a host is supported.

Paper	Category	Evidence and verification
10_1111_1365_2656_13918	UNSUPPORTED_REPO	Paper text cites four EDI/PASTA data package DOIs (10.6073/PASTA/... and 10.6073/pasta/..., resolving to portal.edirepository.org) for the paper's actual data, plus a properly-formatted Zenodo DOI (10.5281/zenodo.7737746, live-verified HTTP 200) for the analysis code only. EDI/PASTA is not in any Metacheck supported host list and has no dedicated module. Because the data_availability comparison column concerns the data (not the code), and the data are exclusively on EDI/PASTA, this paper is classified as citing an unsupported repository for its data; the Zenodo code citation is correctly formatted and would be expected to be found by Metacheck's code_check separately.
10_1111_2041_210x_13463	UNSUPPORTED_REPO	Paper text: 'The data associated with this manuscript are published on ESS-DIVE, the Department of Energy's data portal...'. No specific DOI is given in the extracted text (the paper states DOIs are 'documented in Supporting Information file 4', not in the main text). ESS-DIVE is not in any Metacheck supported host list and has no dedicated module; Google Earth Engine (also mentioned) is not a structured data repository Metacheck can query as such.
10_1111_1365_2435_13550	NCBI_GENBANK	Paper text: 'The sequences were deposited in NCBI SRA database (accession no. ... SRP218212 and ... SRP068522)'. A domain-specific bioinformatics accession (SRA), outside repo_check's scope by design, consistent with other SRA/GenBank/BioProject cases in this set.
10_1111_1365_2435_12874	TEXT_EXTRACTION_TRUNCATED	Extraction returned an empty string -- no text was captured for this paper's data-availability section at all.
10_1111_1365_2664_14787	UNSUPPORTED_REPO	Paper text: 'Data available from the CSIC repository: http://hdl.handle.net/10261/367361'. Live-verified the URL resolves (HTTP 200) to digital.csic.es. A DSpace-based institutional repository, but not in Metacheck's DSpace7 or DSpace-legacy host lists and has no dedicated module.
10_1111_1365_2656_14227	UNSUPPORTED_REPO	Paper text cites two repositories: 'Guillemot demographic data ... freely available from Heriot Watt research portal https://doi.org/10.17861/8fd6a194-d1a7-469c-8604-e45b0414f8f0' (live-verified HTTP 200, researchportal.hw.ac.uk) and 'Puffin and razorbill demographic data ... available upon request ... from the University of Gloucester online repository https://doi.org/10.46289/8FP68KK9' (live-verified HTTP 404, likely a stale/incorrect DOI). Neither host is in any Metacheck supported host list, and neither has a dedicated module.
10_1111_1365_2745_13324	UNSUPPORTED_REPO	Paper text: 'Data used in this publication are available in the appendices and archived at the USU Digital Commons: https://doi.org/10.26078/99w7-eb58'. Live-verified the DOI resolves (HTTP 200) to digitalcommons.usu.edu. A generic bepress Digital Commons instance, not in any Metacheck supported host list.
10_1111_1365_2656_13272	UNSUPPORTED_REPO	Reference-list citation: 'ScholarWorks Data Repository, https://doi.org/10.18122/bio_data/3/boisestate'. Live-verified the DOI resolves (HTTP 200) to scholarworks.boisestate.edu. A generic bepress ScholarWorks instance, not in any Metacheck supported host list.
10_1111_1365_2435_13791	TEXT_EXTRACTION_TRUNCATED	Extraction returned an empty string -- no text was captured for this paper's data-availability section at all.
10_1111_1365_2745_13849	UNSUPPORTED_REPO	Paper text: 'The data are available as a data package on https://www.forestplots.net/: https://doi.org/10.5521/forestplots.net/2022_1'. Live-verified the DOI resolves (HTTP 200) to forestplots.net. Forestplots.net is a domain-specific forest-inventory data archive, not in any Metacheck supported host list and has no dedicated module (a separate, unrelated Dryad DOI also appears in this paper's reference list, but only as a citation for a prior, different dataset -- not this paper's own data).

Paper	Category	Evidence and verification
10_1111_1365_2435_14514	UNSUPPORTED_REPO	Paper text: 'The data that support the findings of this study are available from UNL data archive: https://doi.org/10.32873/unl.dr.20230705'. Live-verified the DOI resolves (HTTP 202) to sandy.unl.edu. Not in any Metacheck supported host list and has no dedicated module.
10_1111_1365_2435_13936	UNSUPPORTED_REPO	Paper text: 'Data are available from the Illinois Data Bank https://doi.org/10.13012/B2IDB-4547091_V1'. Live-verified the DOI resolves (HTTP 403, access-gated redirect) to databank.illinois.edu. A Hydra/Fedora-based institutional repository, not in any Metacheck supported host list.
10_1111_1365_2664_13845	UNSUPPORTED_REPO	Paper text: 'Data are available in the US Department of the Interior and U.S. Geological Survey's ScienceBase https://doi.org/10.5066/P94B0WUV'. Live-verified the DOI resolves (HTTP 403) to www.sciencebase.gov. ScienceBase is not in any Metacheck supported host list, consistent with the other ScienceBase cases in this set.
10_1111_1365_2664_14229	SOURCE_CITATION_ ERROR	Paper text: 'Data available via Unimi Dataverse http://10.0.51.74/RD_UNIMI/5ZXGIV'. Unimi's real Dataverse host (dataverse.unimi.it) IS a supported Dataverse installation (matches .dataverse_host_regex()), but this paper cites a raw internal/private IP address (10.0.51.74, a non-routable RFC 1918 address) instead of the public hostname -- live-verified this address does not resolve externally at all (request failed outright). The paper's own published citation is unusable as given; this is an error in the source text, not a Metacheck platform-support gap or extraction artefact.

Of these 34 papers, 18 genuinely cite an unsupported platform, 2 cite a domain-specific bioinformatics accession outside `repo_check`'s scope, 0 cite a platform Metacheck genuinely supports but failed to detect for the reasons described above, 12 had no usable citation captured in the paper's extracted text at all (either empty extraction or a citation cut off before any identifier appeared), 1 explicitly withheld their data with no repository to find, and 1 paper's own published citation pointed to an internal, non-public address rather than a reachable repository. These per-paper counts feed directly into the corpus-wide reconciling numbers reported in the main text and into the totals of its table of reasons.

A.10 Full paper-by-paper detail for every technical limitation identified in the full disagreement-population review

Table 3 lists every one of the 48 papers in the main text's "technical limitation" cluster individually: its DOI, which of the five specific limitations (grouped into text extraction and Metacheck's own code, as in the main text) applies, and the evidence-based reasoning for that classification, generated directly from the same per-paper audit file (`manual_repo_findability_audit_data.csv`) the main text's numbers are computed from. Because the table is built programmatically from that file rather than transcribed by hand, it stays reconciled with the main text's counts automatically on every re-render (confirmed by the chunk above: the table's row count is checked against `n_metacheck_limitation` directly).

Table 3: Full paper-by-paper detail for every technical limitation identified in the full disagreement-population review.

Paper (DOI)	Category	Why Metacheck misses it
10.1002/pan3.10185	Text extraction: identifier corrupted by the PDF-to-text step	CHECKED 2026-10-13 against metacheck 109a9aa by comparison, not by a full rerun: metacheck's changes since this row was written (data/code extension list and archive download fix, 4 Oct; GROBID DOI-space removal, 4 Oct; new registered hosts and Figshare institutional ids, 6 Oct; Dryad zip members, 10 Oct) were checked against this paper. 109a9aa's link detectors find the same repository links in the paper as the stored run, and none of the changes applies to this paper's files, so this explanation still holds. metacheck fully supports this exact platform and host: rdr.ucl.ac.uk is explicitly hardcoded in R/archive-figshare.R's .figshare_vanity_hosts() (line 88, labeled "University College London (UK)"), the figshare.com DOI prefix 10.5522 is not even needed here because the underlying numeric article id (13469022) appears directly, and .figshare_id()'s regex patterns are written to match exactly this "10.6084/m9.figshare.N" or vanity-host "/articles/.../N" shape. The reason metacheck_any_detector_hit is false is almost certainly that the GROBID-extracted full_text corrupted all four DOIs identically: every occurrence reads "10.5522/04/13469 022.v1" (and 13469 085, 13469 139, 13469 166) with a literal space inserted between the 5th and 6th digit of the article id, splitting what should be one contiguous numeral "13469022" into "13469" + " " + "022". figshare_links()'s bare-text regex fallback (and .figshare_id()'s digit-capture groups) require an unbroken run of digits ([0-9]+); a stray embedded space breaks that run, so the regex simply does not match this citation as extracted, even though a human reader (and the live DOI resolver) has no trouble with it. This is a text-extraction artifact (GROBID/PDF-to-text splitting digit groups, the same class of bug already fixed elsewhere for effect-size subscripts per the recent commit history on this branch) rather than any gap in metacheck's Figshare/UCL-RDR platform support, which is otherwise a fully correct, already-registered configuration.
10.1002/pan3.10623	Text extraction: identifier corrupted by the PDF-to-text step	CHECKED 2026-10-13 against metacheck 109a9aa by comparison, not by a full rerun: metacheck's changes since this row was written (data/code extension list and archive download fix, 4 Oct; GROBID DOI-space removal, 4 Oct; new registered hosts and Figshare institutional ids, 6 Oct; Dryad zip members, 10 Oct) were checked against this paper. 109a9aa's link detectors find the same repository links in the paper as the stored run, and none of the changes applies to this paper's files, so this explanation still holds. Paper states 'All data used in the analysis can be found freely online on Figshare,' but the adjoining reference-list DOI is corrupted by PDF column-reflow (an unrelated reference interleaved mid-title; no Figshare DOI/URL survives extraction). Independently located the real dataset via Figshare's search API: DOI 10.6084/m9.figshare.25066136.v1, verified live (is_public TRUE, MIT license, 11 files). Metacheck's archive-figshare.R fully supports resolving this DOI via the documented api.figshare.com/v2 REST API -- it never had a chance to act because no identifier survived extraction.

Paper (DOI)	Category	Why Metacheck misses it
10.1002/pan3.10689	Text extraction: identifier corrupted by the PDF-to-text step	RECLASSIFIED 2026-10-13 (EXTRACTION_GAP_DOI_GARBLED, was SUPPORTED_PLATFORM_CONFIG_GAP): The host (VTechData) is registered in metacheck, but the DOI in the extracted text is broken by spaces ('10.7294/ 25914 451. v2'): a text-extraction problem. Earlier explanation: metacheck has a full, working Figshare backend (R/archive-figshare.R) that correctly resolves articles via api.figshare.com, including dozens of institutional "vanity" Figshare instances and their own DOI prefixes (.figshare_vanity_hosts() and .figshare_doi_prefix_hosts()). But this paper's data link is https://doi.org/10.7294/25914451.v2, hosted at VTechData (data.lib.vt.edu) under DOI prefix 10.7294. Neither the host "data.lib.vt.edu" nor the DOI prefix "10.7294" appears in .figshare_vanity_hosts() or .figshare_doi_prefix_hosts(), and the bare-DOI fallback regex in figshare_links()/.figshare_id() only recognizes the literal 10.6084 prefix plus whatever prefixes are registered in that list -- so a bare citation of "10.7294/25914451.v2" (as this paper's availability statement uses, with no "figshare.com" or "data.lib.vt.edu" string anywhere near the DOI) is invisible to figshare_links(), and the resolved redirect host data.lib.vt.edu is also absent from the host regex, so even a real hyperlink to the landing page would not match either. I independently confirmed live via api.figshare.com/v2/articles/25914451 that this is genuinely the same Figshare API backend already supported (is_public: true, matching title, files PAN_MSV_Dataset.xlsx/Survey Instrument.pdf/README.rtf, license CC0 1.0) -- so this is a missing host/DOI-prefix registration for an existing, working platform integration, not a missing platform and not a broken or inaccessible repository. I also note archive-dryad.R's own comments record that DOI prefix 10.7294 (VTechData) was previously checked and confirmed NOT to be Dryad, but it was apparently never cross-registered as Figshare despite that check. Fix: add "10.7294" -> "data.lib.vt.edu" to .figshare_doi_prefix_hosts() and "data.lib.vt.edu" to .figshare_vanity_hosts(). CORRECTION 2026-10-07: a live rerun through the actual installed package (after confirming .figshare_doi_prefix_hosts()/.figshare_vanity_hosts() do now list 10.7294/data.lib.vt.edu via a newer .figshare_doi_lookup() API fallback added in commit 80a7d37e) still produced repo_n = 0, files_n = 0 for this paper -- the registry fix does not resolve it in practice. Tracing why: this paper's own extracted text has the DOI itself whitespace-garbled by GROBID ('https://doi. org/ 10.7294/ 25914 451. v2', with a spurious space inside the numeric id), a DIFFERENT and unrelated defect from the host-registry gap this row originally described. The direct-function-call test that appeared to confirm this fix used a clean, manually-reconstructed DOI string, not this paper's actual corrupted extracted text, so it never actually tested the real failure path. Root cause is EXTRACTION_GAP_DOI_GARBLED-shaped, layered on top of what is otherwise a genuinely-fixed config gap -- the config fix alone is not sufficient. [CONFIG_GAP_TIER: needs_code_change -- rechecked 2026-10-13 against metacheck 109a9aa and the live repository: The host (data.lib.vt.edu / 10.7294, VTechData) is now registered in metacheck, but the paper is still not found: the DOI in the extracted text is broken by spaces ('10.7294/ 25914 451. v2'), so the fix needed is in reading the text, not in the host list.]

Paper (DOI)	Category	Why Metacheck misses it
10.1111/1365-2435.14477	Text extraction: identifier corrupted by the PDF-to-text step	CHECKED 2026-10-13 against metacheck 109a9aa by comparison, not by a full rerun: metacheck's changes since this row was written (data/code extension list and archive download fix, 4 Oct; GROBID DOI-space removal, 4 Oct; new registered hosts and Figshare institutional ids, 6 Oct; Dryad zip members, 10 Oct) were checked against this paper. 109a9aa's link detectors find the same repository links in the paper as the stored run, and none of the changes applies to this paper's files, so this explanation still holds. metacheck already fully supports this exact host and DOI prefix: R/archive-dataverse.R's .dataverse_hosts() explicitly lists "phys-techsciences.datastations.nl" (one of the four DANS Data Stations, which are Dataverse installations, not a generic institutional repository as the coder's label "DANS (Data Archiving and Networked Services, Netherlands)" implies), and .dataverse_doi_prefix_hosts() explicitly maps that host to the "10.17026" DOI prefix used by all DANS Data Stations. So this is not a platform gap or a config gap -- the detector's host/prefix registry already covers it. The reason metacheck's detector did not fire (metacheck_ any_detector_hit = false) is that the GROBID/PDF text extraction garbled the one and only DOI in the paper: it appears as "https://doi. org/ 10.17026/ dans-zqs-g6cn" with spurious internal whitespace splitting "doi.org/", "10.17026/", and "dans-zqs-g6cn" into separate tokens, inside a reference-list citation rather than in the data-availability sentence itself (which is truncated mid-word and has no link at all: "...are stored in DANS (Data archiving and networked..."). A regex expecting a contiguous "10.17026/dans-zqs-g6cn" or "doi.org/10.17026/..." string would not match this whitespace-split form, so the extraction/garbling is the actual failure point, not any lack of platform or host support.
10.1111/1365-2435.14696	Text extraction: identifier corrupted by the PDF-to-text step	CHECKED 2026-10-13 against metacheck 109a9aa by comparison, not by a full rerun: metacheck's changes since this row was written (data/code extension list and archive download fix, 4 Oct; GROBID DOI-space removal, 4 Oct; new registered hosts and Figshare institutional ids, 6 Oct; Dryad zip members, 10 Oct) were checked against this paper. 109a9aa's link detectors find the same repository links in the paper as the stored run, and none of the changes applies to this paper's files, so this explanation still holds. Metacheck fully supports Zenodo via a dedicated R/archive-zenodo.R module, which explicitly anticipates bare "10.5281/zenodo.NNNNNNN" citations without a URL scheme (see the module's own comment: "10.5281/zenodo.1234567 is routinely cited without any URL scheme at all"), and this paper's DOI (10.5281/zenodo.13986711) is exactly that standard Zenodo prefix pattern, resolving live to a public CC-BY-4.0 Zenodo record (zenodo.org/records/13986711) with a CSV dataset and README. So this is not a platform-support gap. The failure is that the in-text data-availability sentence itself is truncated by the PDF text extraction right after "Data reported in the manuscript are available from the Zenodo" -- the DOI never appears in that sentence at all in full_text. The DOI does appear later, in the separate bibliography entry for Lu, Wenglein, Bluhm, Stuckenberg, Potapov & Scheu (2024), but there it is corrupted by stray whitespace injected during extraction: "https://doi. org/ 10.5281/ ZENODO.13986711" (spaces after "doi.", after "org/", and after "10.5281/"). A DOI/URL regex that requires a contiguous "doi.org/10.5281/zenodo.NNNNNNN" string without internal whitespace tolerance would fail to match this specific occurrence, even though the Zenodo backend itself would handle the ID correctly once given a clean DOI. This is best classified as a garbled-DOI extraction gap rather than a missing platform or a genuinely absent link.

Paper (DOI)	Category	Why Metacheck misses it
10.1111/1365-2656.13345	Text extraction: identifier corrupted by the PDF-to-text step	CHECKED 2026-10-13 against metacheck 109a9aa by comparison, not by a full rerun: metacheck's changes since this row was written (data/code extension list and archive download fix, 4 Oct; GROBID DOI-space removal, 4 Oct; new registered hosts and Figshare institutional ids, 6 Oct; Dryad zip members, 10 Oct) were checked against this paper. 109a9aa's link detectors find the same repository links in the paper as the stored run, and none of the changes applies to this paper's files, so this explanation still holds. Metacheck fully supports Dryad via R/archive-dryad.R, whose dryad_links() function matches bare Dryad DOIs with the regex "(?:https?://)?(?:doi\ .org/)?10\ .5061/dryad\ .[A-Za-z0-9]+" (and .dryad_doi() re-applies the equivalent capturing pattern "(10\ .5061/dryad\ .[A-Za-z0-9]+)"). In this paper's extracted full_text, the DOI is corrupted by what looks like a PDF line-wrap or column-break artifact that inserted a literal space inside the Dryad suffix: "https://doi.org/10.5061/dryad.hmgqn k9f3" in the data-availability statement, and again as "https://doi.org/10.5061/ dryad.hmgqn k9f3" in the reference list (La Sorte & Graham, 2020 citation). Because "[A-Za-z0-9]+" cannot match across a whitespace character, the regex engine stops at "hmgqn" and either extracts the truncated/incorrect DOI "10.5061/dryad.hmgqn" or otherwise fails to recover the real, correct suffix "hmgqnk9f3". The live repository itself is a completely ordinary, fully supported, publicly accessible Dryad dataset with files present -- this is not a platform gap or a configuration gap, but a text-extraction corruption of the DOI string that defeats the existing, otherwise-correct Dryad regex.
10.1111/1365-2656.13893	Text extraction: identifier corrupted by the PDF-to-text step	CHECKED 2026-10-13 against metacheck 109a9aa by comparison, not by a full rerun: metacheck's changes since this row was written (data/code extension list and archive download fix, 4 Oct; GROBID DOI-space removal, 4 Oct; new registered hosts and Figshare institutional ids, 6 Oct; Dryad zip members, 10 Oct) were checked against this paper. 109a9aa's link detectors find the same repository links in the paper as the stored run, and none of the changes applies to this paper's files, so this explanation still holds. metacheck fully supports Dryad (R/archive-dryad.R), and its dryad_links() regex (10\ .5061/dryad\ .[A-Za-z0-9]+, plus a doi\ .org/ prefix variant) would ordinarily match this exact DOI (10.5061/dryad.7d7wm3806, under the explicitly allow-listed 10.5061 prefix) without any config change needed. The reason metacheck_any_detector_hit is false is that the PDF-extracted full_text corrupts the DOI at both of its two occurrences with stray whitespace introduced during text extraction, not because the platform or DOI prefix is unsupported. In the data-availability statement, the text reads \ "https://doi.org/10.5061/dryad.7d7wm 3806\ ", with a space inserted between \ "7d7wm\ " and \ "3806\ " that splits the DOI suffix into two tokens; since the regex's [A-Za-z0-9]+ character class does not match whitespace, it would at most capture the truncated \ "dryad.7d7wm\ " and miss the real dataset DOI \ "dryad.7d7wm3806\ ". In the reference-list citation (Riecke, Hegelbach, & Schaub, 2023), the second occurrence instead has the space between \ "doi.\ " and \ "org\ " (\ "https://doi. org/10.5061/dryad.7d7wm3806\ "), which breaks the literal \ "doi\ \ .org\ " substring the regex's doi.org/-prefix alternative requires, though the bare \ "10.5061/dryad\ " alternative in the pattern could still match this one correctly since the DOI digits themselves are intact there. Because the live dataset is confirmed real, public, and directly reachable at exactly the DOI the human coder recorded, this is a text-extraction/OCR-layer corruption gap specific to this PDF's rendering of the DOI (likely a kerning or line-wrap artifact in the original Wiley PDF), not a platform-support gap, a configuration gap, or a detector logic bug against clean input.

Paper (DOI)	Category	Why Metacheck misses it
10.1111/1365-2656.14000	Text extraction: identifier corrupted by the PDF-to-text step	CHECKED 2026-10-13 against metacheck 109a9aa by comparison, not by a full rerun: metacheck's changes since this row was written (data/code extension list and archive download fix, 4 Oct; GROBID DOI-space removal, 4 Oct; new registered hosts and Figshare institutional ids, 6 Oct; Dryad zip members, 10 Oct) were checked against this paper. 109a9aa's link detectors find the same repository links in the paper as the stored run, and none of the changes applies to this paper's files, so this explanation still holds. metacheck fully supports this exact case at the code level: R/archive-figshare.R already registers "10.26180" => "bridges.monash.edu" in .figshare_doi_prefix_hosts(), and .figshare_id() (lines ~307-344) carries a comment explicitly citing this very paper's DOI as a confirmed-working worked example ("confirmed live 2026-09-19: https://doi.org/10.26180/19095317.v1 resolves to Monash article id 19095317"). The failure is not in the detector's logic but in the upstream PDF text extraction: in full_text, the DOI string is rendered with two stray internal spaces -- "https://doi.org/10.26180/ 19095 317.v1" -- splitting the contiguous digit string "19095317" into "19095" and "317" (apparently a line/column-wrap artifact from the two-column PDF layout, identical in both the reference-list occurrence and the implied data-availability-statement occurrence). The very first scanning stage that finds Figshare candidate strings, figshare_links()'s fs_bare_regex (line 220 of archive-figshare.R), requires the DOI suffix to match "[A-Za-z0-9._-]+" contiguously with no whitespace, so this regex never matches the garbled string and no candidate URL is even extracted for .figshare_id() to later resolve. The coder's own recorded repo_link_verbatim has the identical "19095 317" space-split, confirming this is a property of the underlying text extraction itself, not an error either the coder or I introduced. The platform (Figshare via Monash Bridges), its DOI-prefix registration, and its id-resolution regex are all already correctly built and tested for the clean form of this exact DOI; only the whitespace-corrupted rendering of this one citation defeats the first-stage text-search regex, which is a text-extraction/garbling problem, not a missing or buggy detector.
10.1111/1365-2656.14054	Text extraction: identifier corrupted by the PDF-to-text step	CHECKED 2026-10-13 against metacheck 109a9aa by comparison, not by a full rerun: metacheck's changes since this row was written (data/code extension list and archive download fix, 4 Oct; GROBID DOI-space removal, 4 Oct; new registered hosts and Figshare institutional ids, 6 Oct; Dryad zip members, 10 Oct) were checked against this paper. 109a9aa's link detectors find the same repository links in the paper as the stored run, and none of the changes applies to this paper's files, so this explanation still holds. Code citation appears only in the reference list (no DAS exists): 'Amarasekare, P. (2024)... Zenodo https://doi. org/ 10.5281/ ze- nodo.10497888' -- PDF extraction inserted spaces after 'doi.', after 'org/', after '10.5281/', and hyphen-broke the word 'zenodo' itself. Metacheck's zenodo_links() regex (archive-zenodo.R) requires a contiguous string and cannot match text containing embedded spaces/hyphen-breaks. Live-verified the Zenodo record resolves and is open, CC-BY-4.0, linking to the companion GitHub repo.

Paper (DOI)	Category	Why Metacheck misses it
10.1111/1365-2664.13890	Text extraction: identifier corrupted by the PDF-to-text step	CHECKED 2026-10-13 against metacheck 109a9aa by comparison, not by a full rerun: metacheck's changes since this row was written (data/code extension list and archive download fix, 4 Oct; GROBID DOI-space removal, 4 Oct; new registered hosts and Figshare institutional ids, 6 Oct; Dryad zip members, 10 Oct) were checked against this paper. 109a9aa's link detectors find the same repository links in the paper as the stored run, and none of the changes applies to this paper's files, so this explanation still holds. metacheck_any_detector_hit was reported as false, but this looks like it should actually be detectable as a dspace/dspace7 case, so I lean toward SUPPORTED_PLATFORM_BUG/CONFIG_GAP rather than a clean no-gap -- see caveat below. The repository DOI 10.20350/digitalCSIC/13825 resolves to digital.csic.es/handle/10261/236117, and the "/handle/<naan>/<suffix>" path is the canonical DSpace resolver URL pattern, confirming DIGITAL.CSIC is a generic DSpace installation (the coder's platform label "DSpace-based" is correct and matches the URL evidence, independent of the live Anubis block). Since metacheck's supported-platform list explicitly includes "dspace/dspace7 (generic DSpace installations)," the platform family itself is supported. The DOI is also present, unambiguously, in the paper's reference list, though the raw PDF-extracted text has it corrupted by stray inserted spaces ("https://doi.org/10.20350/ digit alCSI C/13825" instead of ".../digitalCSIC/13825") -- a GROBID-style extraction artifact that could defeat a naive strict-pattern link extractor looking for an unbroken "digitalCSIC" token or path segment, even though the DOI prefix "10.20350" and the surrounding citation context are intact and a human (or a more tolerant regex/whitespace-stripping extractor) would still recognize it. Given metacheck_any_detector_hit=false, the most defensible classification is that this is a DOI-level extraction problem (stray whitespace inside the handle/suffix string) rather than a missing-platform gap, so I classify it as EXTRACTION_GAP_DOI_GARBLED: the link is present in the text but corrupted by split/inserted whitespace in a way that likely prevents metacheck's extractor from recognizing it as the clean "10.20350/digitalCSIC/13825" DOI string, even though the DSpace/DIGITAL.CSIC platform itself is on metacheck's supported list and would otherwise be detectable.
10.1111/1365-2664.13961	Text extraction: identifier corrupted by the PDF-to-text step	CHECKED 2026-10-13 against metacheck 109a9aa by comparison, not by a full rerun: metacheck's changes since this row was written (data/code extension list and archive download fix, 4 Oct; GROBID DOI-space removal, 4 Oct; new registered hosts and Figshare institutional ids, 6 Oct; Dryad zip members, 10 Oct) were checked against this paper. 109a9aa's link detectors find the same repository links in the paper as the stored run, and none of the changes applies to this paper's files, so this explanation still holds. metacheck has a dedicated, well-built Dryad detector (R/archive-dryad.R) whose regex matches datadryad.org/dataset(s)/doi: URLs as well as bare "10.5061/dryad.XXXXX" DOIs with or without a URL scheme, so the platform itself is fully supported and this exact DOI shape would normally be caught. The reason metacheck_any_detector_hit is false here is that the real Dryad link is corrupted at both places it appears in the extracted full_text: (1) in the data-availability statement, the URL token has been replaced by the author's ORCID link (https://orcid.org/0000-0002-9304-228X) -- almost certainly a PDF-to-text extraction artifact where a hyperlinked DOI field was resolved to the wrong embedded link target; and (2) in the reference-list citation, the DOI itself is printed one character short as "10.5061/dryad.0pngf20h" (missing the digit "2"), which I verified live returns HTTP 404 rather than resolving to the dataset. Only the correct, non-garbled DOI "10.5061/dryad.0p2ngf20h" -- which does not appear anywhere in the extracted text -- actually resolves to the real public dataset. Since neither occurrence in the text metacheck would see contains a matchable, correct Dryad identifier, no amount of improving the Dryad detector logic would fix this particular case; the gap is upstream, in the source PDF/text extraction corrupting the DOI before metacheck ever sees it.

Paper (DOI)	Category	Why Metacheck misses it
10.1111/1365-2664.14229	Text extraction: identifier corrupted by the PDF-to-text step	CHECKED 2026-10-13 against metacheck 109a9aa by comparison, not by a full rerun: metacheck's changes since this row was written (data/code extension list and archive download fix, 4 Oct; GROBID DOI-space removal, 4 Oct; new registered hosts and Figshare institutional ids, 6 Oct; Dryad zip members, 10 Oct) were checked against this paper. 109a9aa's link detectors find the same repository links in the paper as the stored run, and none of the changes applies to this paper's files, so this explanation still holds. metacheck already fully supports this exact repository: dataverse.unimi.it is explicitly listed in .dataverse_hosts() (R/archive-dataverse.R, line 137, "Università degli Studi di Milano (Italy)"), and its DOI prefix 10.13130 is explicitly mapped to it in .dataverse_doi_prefix_hosts() (line 314: "dataverse.unimi.it" = "10.13130"), individually verified against DataCite per that function's header comment. This is a correctly-configured, supported Dataverse installation with no gap in metacheck's platform or host/prefix registry. The failure is purely in the paper's extracted text: the data-availability sentence gives the link as "http://10.0.51.74/RD_ UNIMI/5ZXGIV" — the real DOI resolver domain was corrupted into what looks like an internal/private IPv4 address during PDF extraction, so it contains neither the "dataverse.unimi.it" hostname string (which dataverse_links()'s host-regex path requires) nor the "10.13130" DOI prefix (which the bare-DOI-prefix regex path requires). The second occurrence, in the reference list ("https://doi.org/10.13130/ RD_UNIMI/ 5ZXGIV"), does carry the correct, verified 10.13130 prefix, but PDF line-wrap extraction inserted literal spaces after each forward slash, splitting the DOI into three fragments ("10.13130/", "RD_UNIMI/", "5ZXGIV"). metacheck's DOI-prefix regex (an unbroken "[A-Za-z0-9/._ -]+" after the prefix) cannot match across that whitespace, so this occurrence also fails to resolve. Both failures are text-extraction corruption of the same underlying, already-registered link, not a gap in platform coverage or host/prefix configuration -- this is a GROBID/PDF-extraction defect (domain substitution in one place, inserted whitespace in another) rather than anything metacheck's Dataverse module needs to be extended to handle.
10.1111/1365-2745.13085	Text extraction: identifier corrupted by the PDF-to-text step	RECLASSIFIED 2026-10-13 (EXTRACTION_GAP_DOI_GARBLED, was SUPPORTED_PLATFORM_BUG): The PDF-to-text step inserted an artefact (file://localhost/opt/grobid/...) into the middle of the Dryad DOI (10.5061/dryad.745g830), so only a truncated DOI reaches metacheck: a text-extraction problem. Earlier explanation: metacheck fully supports Dryad (R/archive-dryad.R), and its bare-DOI regex `10\ .5061/dryad\ .[A-Za-z0-9]+` (dryad_links(), line 78) is written correctly and would match the text as printed, extracting "10.5061/dryad.745" as a Dryad hit -- which is why metacheck_any_detector_hit is true and the sanity check is not a clean pass. The real problem lies upstream of the regex, in GROBID's PDF text extraction: the paper's actual DOI is 10.5061/dryad.745g830, but GROBID inserted a spurious internal artifact, "file://localhost/opt/grobid/grobid-home/tmp/g830", directly between "745" and the true suffix "g830" in the extracted full_text, so the DOI appears split into "10.5061/dryad.745" followed by an unrelated local file path that happens to end in the string "g830" that truly belongs to the DOI. metacheck's regex, which has no way to know a PDF-extraction artifact sits in the middle of a DOI, correctly matches only the truncated "10.5061/dryad.745" -- which independently resolves to an HTTP 404, i.e. a dead DOI. So metacheck's detector registers a syntactic hit, but on a non-resolving link, not on the real, live, public dataset (10.5061/dryad.745g830) that a manual check finds. This is a text-extraction/GROBID artifact bug feeding a correctly-implemented Dryad detector, not a missing platform or missing regex case, and the human coder independently transcribed the exact same garbled DOI from the same corrupted source text, so this is not an extraction-pipeline issue unique to metacheck's own parsing either -- it is inherited directly from the GROBID output both relied on.

Paper (DOI)	Category	Why Metacheck misses it
10.1111/1365-2745.13577	Text extraction: identifier corrupted by the PDF-to-text step	CHECKED 2026-10-13 against metacheck 109a9aa by comparison, not by a full rerun: metacheck's changes since this row was written (data/code extension list and archive download fix, 4 Oct; GROBID DOI-space removal, 4 Oct; new registered hosts and Figshare institutional ids, 6 Oct; Dryad zip members, 10 Oct) were checked against this paper. 109a9aa's link detectors find the same repository links in the paper as the stored run, and none of the changes applies to this paper's files, so this explanation still holds. Dryad is a platform metacheck explicitly supports, and the dataset is genuinely live and public (confirmed by following the DOI redirect to datadryad.org). The reason metacheck's detector did not fire (metacheck_any_detector_hit: false) is that in the extracted full_text, every occurrence of the Dryad DOI -- both in the in-text data-availability statement ("Data on plant phenology are available at the Dryad Digital Repository: https://doi.org/10.5061/dryad.c59zw 3r69") and in the reference-list entry (Bucher & Römermann, 2020a) -- contains a stray internal space that splits "c59zw3r69" into "c59zw" and "3r69". This is almost certainly an artifact of PDF line-wrapping/hyphenation during text extraction (GROBID or similar), not a typo in the original PDF, since a genuinely broken DOI would not be something the original authors or Wiley typesetting would publish twice identically. Because the space breaks the DOI into two whitespace-separated tokens, any regex-based or URL-pattern detector looking for a well-formed `10.5061/dryad.xxxxx` DOI or a contiguous `doi.org/10.5061/dryad...` URL would fail to match it as written in this extracted text, even though the platform (Dryad) is fully supported by metacheck. This is a text-extraction corruption problem upstream of the detector logic, not a gap in platform coverage or a config issue -- the fix would belong in the extraction/normalization step (e.g. stripping/collapsing whitespace inside DOI-shaped strings before pattern matching), not in adding a new platform detector. Separately, a second repository link in the paper (TRY database, Dataset 416, a plant-trait database) is on an unsupported platform and was not something the coder recorded either, but it is incidental to this specific gap -- the primary data-sharing link the coder flagged is the Dryad one, and that one is a garbled-DOI extraction issue.
10.1111/1365-2745.13606	Text extraction: identifier corrupted by the PDF-to-text step	CHECKED 2026-10-13 against metacheck 109a9aa by comparison, not by a full rerun: metacheck's changes since this row was written (data/code extension list and archive download fix, 4 Oct; GROBID DOI-space removal, 4 Oct; new registered hosts and Figshare institutional ids, 6 Oct; Dryad zip members, 10 Oct) were checked against this paper. 109a9aa's link detectors find the same repository links in the paper as the stored run, and none of the changes applies to this paper's files, so this explanation still holds. Dryad is a fully supported platform in metacheck (R/archive-dryad.R), including a specific fallback in dryad_links() for exactly this scenario: a bare, non-hyperlinked DOI mention in body/reference text, matched via the regex `(?:https?://)?(?:doi\ .org/)?10\ .5061/dryad\ .[A-Za-z0-9]+` (plus a found_href branch for real anchor-tag links, which does not apply here since the coder's own note confirms no clickable URL exists in the extracted_urls table). The detector still misses this citation because the text extraction (likely GROBID, given the paper's Wiley/BES PDF source) corrupted the DOI across what was probably a line wrap in the original PDF: the extracted full_text reads "https://doi.org/10.5061/ dryad.f7m0c fxv3" (verified by direct substring inspection of full_text) — note two inserted spaces, one between "10.5061/" and "dryad.", and a second one splitting the identifier itself into "f7m0c" and "fxv3". Because `[A-Za-z0-9]+` cannot match across embedded whitespace, neither the dryad_bare_regex pattern nor the DOI-extraction helper `.dryad_doi()` (which uses the same `10\ .5061/dryad\ .[A-Za-z0-9]+` pattern) can recognize this as a single token, so dryad_links() returns no match for this paper. This is a text-extraction/garbling problem upstream of metacheck's Dryad detector logic, not a missing platform or a misconfigured host: live-checking the DOI confirms it resolves correctly to a real, public, two-file Dryad dataset that exactly matches the paper's subject matter, so the citation itself is intact in the published paper — only the extracted text metacheck works from is corrupted at this one point.

Paper (DOI)	Category	Why Metacheck misses it
10.1111/1365-2745.13694	Text extraction: identifier corrupted by the PDF-to-text step	CHECKED 2026-10-13 against metacheck 109a9aa by comparison, not by a full rerun: metacheck's changes since this row was written (data/code extension list and archive download fix, 4 Oct; GROBID DOI-space removal, 4 Oct; new registered hosts and Figshare institutional ids, 6 Oct; Dryad zip members, 10 Oct) were checked against this paper. 109a9aa's link detectors find the same repository links in the paper as the stored run, and none of the changes applies to this paper's files, so this explanation still holds. Metacheck has a full, working Zenodo detector (R/archive-zenodo.R), and its design is explicitly built to catch bare DOI citations of the exact form "10.5281/zenodo.NNNNNNN" with or without a URL scheme or doi.org prefix (see the regex and comment at R/archive-zenodo.R lines 5 and 28: "10.5281/zenodo.1234567" is routinely cited without any URL scheme at all"). The gap here is not a missing platform but a text-extraction corruption: in the extracted full_text, the citation reads "Zenodo, https://doi.org/10.5281/ zenodo.3756781" with a literal space inserted between "10.5281/" and "zenodo.3756781" (almost certainly a PDF line-wrap artifact from GROBID/PDF-to-text conversion at a hyphenation or column break). Metacheck's R/text-extractors.R extract_urls() function, which is the upstream source of link candidates that archive-zenodo.R and other archive-*.R detectors work from, uses a \ b-bounded regex requiring a contiguous domain-like token ([\ w.-]+\ .[a-z]{2,}) with no internal whitespace. That regex matches "https://doi.org/10.5281/" and "zenodo.3756781" only as two separate, incomplete fragments across the inserted space, neither of which satisfies the Zenodo-ID pattern (10\ .5281/zenodo\ .[0-9]+) that archive-zenodo.R looks for. The result is that no valid Zenodo candidate URL is ever produced for the detector to test, which is consistent with metacheck_any_detector_hit being false even though the paper does cite a fully public, CC-BY-4.0, live Zenodo dataset with three files, and even though Zenodo is a fully supported platform. This is a text-extraction/garbling problem (a split DOI token), not a missing detector and not a dead repository.

Paper (DOI)	Category	Why Metacheck misses it
10.1111/1365-2745.13875	Text extraction: identifier corrupted by the PDF-to-text step	CHECKED 2026-10-13 against metacheck 109a9aa by comparison, not by a full rerun: metacheck's changes since this row was written (data/code extension list and archive download fix, 4 Oct; GROBID DOI-space removal, 4 Oct; new registered hosts and Figshare institutional ids, 6 Oct; Dryad zip members, 10 Oct) were checked against this paper. 109a9aa's link detectors find the same repository links in the paper as the stored run, and none of the changes applies to this paper's files, so this explanation still holds. metacheck fully supports Dryad (R/archive-dryad.R), and the DOI prefix 10.7280 used by this exact dataset is explicitly in .dryad_doi_prefixes() (archive-dryad.R line 37), with dryad_links()'s regex built to match "10.7280/[A-Za-z0-9]+" as a bare in-text DOI even without a scheme. The failure is not a platform gap but a text-extraction corruption: the paper's primary data-availability sentence reads "Data available from the Dryad Data Repository http://10.0.28.112/D1D99J (Campbell et al., 2022)" -- the DOI prefix "10.7280" has been replaced by what looks like a private IP address "10.0.28.112" (almost certainly a PDF/GROBID hyperlink-extraction artifact mangling the embedded doi.org URL), so neither the datadryad.org host string nor the 10.7280 prefix pattern is intact at that location for dryad_bare_regex to match, and it is unlikely the href/url table preserved a clean target either, since the same corrupted string appears in the flattened text. A second, well-formed copy of the same DOI (https://doi.org/10.7280/D1D99J) exists in the reference list under the Campbell et al. (2022) entry; in principle this clean occurrence should satisfy the regex, but the reported metacheck_any_detector_hit=false indicates it did not fire in the live run, most plausibly because GROBID-style extraction of the actual PDF introduces a line-wrap or whitespace split around the DOI (e.g. "doi. org/10.7280/D1D99J") that defeats the regex even though the flattened full_text I was given displays it as one unbroken string. I manually confirmed the DOI itself is completely valid and live: doi.org/10.7280/D1D99J redirects cleanly to a real, public, accessible Dryad dataset page with six CSV files and four READMEs, so this is a genuine false negative caused by extraction-level corruption/formatting of the DOI text, not a missing platform or a dead repository.

Paper (DOI)	Category	Why Metacheck misses it
10.1111/1365-2745.13882	Text extraction: identifier corrupted by the PDF-to-text step	CHECKED 2026-10-13 against metacheck 109a9aa by comparison, not by a full rerun: metacheck's changes since this row was written (data/code extension list and archive download fix, 4 Oct; GROBID DOI-space removal, 4 Oct; new registered hosts and Figshare institutional ids, 6 Oct; Dryad zip members, 10 Oct) were checked against this paper. 109a9aa's link detectors find the same repository links in the paper as the stored run, and none of the changes applies to this paper's files, so this explanation still holds. Metacheck fully supports this exact platform and host: R/archive-dataone.R's .dataone_hosts() already registers host = "arcticdata.io" with doi_prefix = "10.18739" (line 33), which is precisely the DOI prefix all four of this paper's Arctic Data Center citations use, and dataone_links()'s bare-DOI regex (lines 105-109) is built to catch exactly this "https://doi.org/10.18739/<suffix>" citation shape. Live-checking confirmed the dataset is real, public, CC0-licensed, and resolvable through the identical Metacat object endpoint metacheck's own .dataone_info() queries. The actual failure is upstream, in the PDF-to-text extraction that produced full_text: within the data-availability statement itself, three of the four DOI suffixes are split by a spurious whitespace character in the middle of the identifier -- 'https://doi.org/ A2GT5 FG86' instead of 'https://doi.org/A2GT5FG86', 'http://doi. org/10.18739/ A23R0 PV43' instead of '.../A23R0PV43', and 'http://doi. org/10.18739/A2GX44W2M' with a space inserted inside 'doi.org'. Only the second DOI (A2000018G) survived extraction intact. dataone_links()'s matching regex for the bare-DOI fallback is '(?:doi\ \ .org/)?(?:10\ \ .18739)/[A-Za-z0-9._/-]+', which requires the suffix to be one unbroken token of those characters with no whitespace; a space inside 'A2GT5 FG86' terminates the match after 'A2GT5', producing an incomplete, non-resolvable PID rather than no match at all, and likewise breaks the 'doi.org' host token itself in the third and fourth citations ('doi. org'). This is a genuine upstream text-extraction corruption (apparently from the PDF layout, where a line-wrap or font-kerning artifact inserted a space mid-token), not a gap in metacheck's own DataONE detector logic, host list, or regex design for the well-formed case. This matches the EXTRACTION_GAP_DOI_GARBLED category: the link is present in the source text but corrupted in a way that defeats token-based regex matching.

Paper (DOI)	Category	Why Metacheck misses it
10.1111/1365-2745.13999	Text extraction: identifier corrupted by the PDF-to-text step	CHECKED 2026-10-13 against metacheck 109a9aa by comparison, not by a full rerun: metacheck's changes since this row was written (data/code extension list and archive download fix, 4 Oct; GROBID DOI-space removal, 4 Oct; new registered hosts and Figshare institutional ids, 6 Oct; Dryad zip members, 10 Oct) were checked against this paper. 109a9aa's link detectors find the same repository links in the paper as the stored run, and none of the changes applies to this paper's files, so this explanation still holds. Dryad is a fully supported platform in metacheck (R/archive-dryad.R exists and the live link resolves cleanly to a standard public Dryad landing page with no login and a "no restrictions" license), so this is not a platform-support gap. The failure is a text-extraction corruption: in full_text the DOI appears as "https://doi.org/10.5061/dryad.7sqv9 s4sh" -- a literal space has been inserted between "7sqv9" and "s4sh," splitting the Dryad suffix in two. Any DOI/URL-matching regex metacheck runs over the extracted text (whether matching "10.5061/dryad\ \ .\ \ S+" or a general doi.org URL pattern) would stop at the whitespace and either fail to match the full suffix or extract "dryad.7sqv9" as a truncated/invalid DOI, which would not resolve via the Dryad API. The same kind of mid-token space corruption is visible elsewhere in this paper's extracted references (e.g. "foreco.2012.02" rendered as ". foreco.2012.02", "isprs jprs.2020.01.012" instead of "isprsjprs..."), indicating this is a systematic PDF-extraction artifact (likely from line-wrap/hyphenation handling in GROBID or whatever extractor produced full_text) rather than something specific to Dryad's platform support. Additionally, no distinct "Data Availability Statement" section/heading was present anywhere in the extracted full_text at all -- the only occurrence of the link is embedded in the reference list citing the companion Dryad data-descriptor record, suggesting the statement box itself (common in BES/Journal of Ecology articles as a separate panel) may not have been captured by extraction, compounding the problem for any heading-based statement-locator logic upstream of the regex match.
10.1111/2041-210X.12918	Text extraction: identifier corrupted by the PDF-to-text step	RECHECKED 2026-10-17 by a full rerun (metacheck 610cd4f2): still no repository listed, and the explanation below still holds, except that repo_check now recognises the truncated address github.com/frmunoz as a GitHub account and reports it as 'links to a GitHub account, not to a specific repository', without listing the account's repositories. DAS cites two locations: 'The version 1.0.0 of ecolottery is available on CRAN... The development version is available on Github (https://github.com/frmunoz/ [extraction cuts off here, jumping into Authors' Contributions]).' The repo-name path segment ('ecolottery') is entirely absent from the extracted text due to a PDF extraction defect -- Metacheck's github_links() regex requires both org AND repo segments, so with only 'frmunoz' present it cannot match even in principle. The adjacent CRAN URL in the same sentence is independently also garbled, corroborating sentence-level extraction corruption.
10.1111/2041-210X.13350	Text extraction: identifier corrupted by the PDF-to-text step	CHECKED 2026-10-13 against metacheck 109a9aa by comparison, not by a full rerun: metacheck's changes since this row was written (data/code extension list and archive download fix, 4 Oct; GROBID DOI-space removal, 4 Oct; new registered hosts and Figshare institutional ids, 6 Oct; Dryad zip members, 10 Oct) were checked against this paper. 109a9aa's link detectors find the same repository links in the paper as the stored run, and none of the changes applies to this paper's files, so this explanation still holds. Paper cites two links: CRAN (extracted cleanly, twice) and GitHub ('https ://github.com/ achub aty/grain scape', extracted with spaces inserted inside the URL and inside both the owner and repo names). Any regex-based URL matcher fails on the garbled GitHub form even though GitHub has a complete, working backend in Metacheck. CRAN is cited cleanly but is moot since no CRAN backend exists at all.

Paper (DOI)	Category	Why Metacheck misses it
10.1111/2041-210X.13681	Text extraction: identifier corrupted by the PDF-to-text step	CHECKED 2026-10-13 against metacheck 109a9aa by comparison, not by a full rerun: metacheck's changes since this row was written (data/code extension list and archive download fix, 4 Oct; GROBID DOI-space removal, 4 Oct; new registered hosts and Figshare institutional ids, 6 Oct; Dryad zip members, 10 Oct) were checked against this paper. 109a9aa's link detectors find the same repository links in the paper as the stored run, and none of the changes applies to this paper's files, so this explanation still holds. Both repository DOIs ARE present in the extracted full_text, and both platforms (Dataverse, confirmed by the "Powered by The Dataverse Project v6.8.4-SP" footer at borealisdata.ca, and Zenodo, confirmed by the zenodo.org landing page) are on metacheck's supported-platform list, so this is not a coverage gap. The failure is a text-extraction/OCR artifact: the PDF-to-text conversion injected a stray space exactly at the point where each DOI line-wrapped in the original PDF, producing "https://doi. org/10.7939/DVN/QYMAYV" (space between "doi." and "org") for the Dataverse reference and "https://doi.org/10.5281/ zenodo.5070812" (space between "10.5281/" and "zenodo.5070812") for the Zenodo reference. Any DOI-matching regex or URL tokenizer that expects a contiguous string (no embedded whitespace) will fail to recognize either as a valid, resolvable DOI/URL, even though a human reader can see and manually reconstruct both. This matches metacheck_any_detector_hit=false: the detector presumably could not match either garbled string. Separately, the in-text Data Availability Statement itself uses named-citation style ("available on UAL Dataverse Derocher (2021)"; "available on Zenodo Togunov et al. (2021)") with no inline clickable URL -- the resolvable DOIs exist only in the back-matter reference list, so even a perfect-whitespace extraction would still require cross-referencing the in-text citation key to the bibliography entry to recover the link, which is an extra step beyond simple statement-adjacent URL scanning. The coder's recorded link (https://doi.org/10.7939/DVN/QYMAYV, acquired via "a targeted web search" rather than from the text) is confirmed correct and live, but the coder missed the second repository (the Zenodo replication code) entirely.

Paper (DOI)	Category	Why Metacheck misses it
10.1111/2041-210X.13821	Text extraction: identifier corrupted by the PDF-to-text step	CHECKED 2026-10-13 against metacheck 109a9aa by comparison, not by a full rerun: metacheck's changes since this row was written (data/code extension list and archive download fix, 4 Oct; GROBID DOI-space removal, 4 Oct; new registered hosts and Figshare institutional ids, 6 Oct; Dryad zip members, 10 Oct) were checked against this paper. 109a9aa's link detectors find the same repository links in the paper as the stored run, and none of the changes applies to this paper's files, so this explanation still holds. metacheck fully supports this exact platform and host: R/archive-dspace7.R's .dspace7_hosts() explicitly lists "conservancy.umn.edu" (annotated "Data Repository for the University of Minnesota (USA)"), and dspace7_links() already contains code specifically added for this exact failure mode (documented inline as issue #435, citing the near-identical sibling DOI 10.13020/q3y6-h459 on the same host): when a paper cites only a bare DOI with the host domain never appearing in the surrounding text, the function live-resolves every DOI-shaped string found via text_search() and keeps any that redirect to a known DSpace7 host. That mechanism would correctly catch this paper's citation -- except that the DOI as it literally appears in paper's full_text is garbled by a stray inserted space: "https://doi.org/10.13020/ 1f35-rn20" instead of "https://doi.org/10.13020/1f35-rn20" (verified by extracting the exact sentence from full_text). I tested R/archive-dspace7.R's doi_bare_regex directly against both strings: it matches the clean DOI but returns zero matches against the garbled one, because the regex requires an unbroken token after "10.xxx/" and the inserted space breaks it mid-string. This is almost certainly a PDF-text-extraction artifact (the reference list line likely wrapped at that point in the original PDF, and the extraction tool inserted a space at the line break) rather than a metacheck regex design flaw -- the same gap would equally affect any other DOI-based detector in metacheck that depends on this shared doi_bare_regex pattern if a citation's DOI happens to be split by extraction at the same point. I did not attempt to fix this since the task calls for an audit and root-cause classification only.
10.1111/2041-210X.13850	Text extraction: identifier corrupted by the PDF-to-text step	RECHECKED 2026-10-17 by a full rerun (metacheck 610cd4f2): still no repository listed, and the explanation below still holds, except that repo_check now reports the organisation address github.com/ConScape/ as 'links to a GitHub account, not to a specific repository' (before: 'invalid or inaccessible GitHub repository'), without listing the account's repositories. CORRECTION to an earlier, unverified pass that claimed the GitHub repo was 'genuinely inaccessible... not a metacheck defect' -- this is live-verified FALSE; both the GitHub org and the actual repo (ConScape/ConScape.jl) return HTTP 200 via the GitHub API. Primary cause: the reference-list Zenodo citation has a space inserted between '10.5281/' and 'zenodo.5810881' (confirmed via grep on raw full_text), defeating zenodo_links()'s regex (tested directly in R, returns no match). Secondary: the body-text GitHub citation is a bare org URL with no repo segment, so github_repo() correctly returns NULL for that specific citation form -- not itself a bug, since no specific repo was actually named in that URL.

Paper (DOI)	Category	Why Metacheck misses it
10.1111/1365-2656.12794	Text extraction: no trace of the citation in the extracted text	CHECKED 2026-10-13 against metacheck 109a9aa by comparison, not by a full rerun: metacheck's changes since this row was written (data/code extension list and archive download fix, 4 Oct; GROBID DOI-space removal, 4 Oct; new registered hosts and Figshare institutional ids, 6 Oct; Dryad zip members, 10 Oct) were checked against this paper. 109a9aa's link detectors find the same repository links in the paper as the stored run, and none of the changes applies to this paper's files, so this explanation still holds. Dryad is one of metacheck's supported platforms (dataone, dataverse, dryad, dspace/dspace7, figshare, github, gitlab, mendeley, osf, psycharchives, rbox, researchdata4tu, reshare, zenodo, aspredicted), so this is not a platform-support gap. The actual problem is upstream of detection: the full_text metacheck extracted for this paper (apparently a PMC manuscript version, judging by the trailing "Refer to Web version on PubMed Central for supplementary material" and "J Anim Ecol. Author manuscript; available in PMC 2018 June01" notices embedded in the body) simply does not contain the journal's "Data Accessibility" statement or the Dryad DOI (10.5061/dryad.95486) anywhere in the extracted string — I read the entire full_text field end to end and confirmed no occurrence of "Dryad," "data availability," "data accessibility," or any doi.org/5061 string. Since metacheck's Dryad detector can only act on text it is given, and the availability statement text was never captured by the upstream extraction pipeline (likely because the PMC author-manuscript rendering omits or relocates the data-accessibility statement relative to the original journal HTML/PDF), the detector correctly reports metacheck_ any_detector_hit = false through no fault of the Dryad-detection logic itself. The repository itself is independently verified live and public (confirmed via WebFetch to datadryad.org), so this is a text-extraction/input gap, not a detector-capability or configuration gap.
10.1111/1365-2656.12900	Text extraction: no trace of the citation in the extracted text	CHECKED 2026-10-13 against metacheck 109a9aa by comparison, not by a full rerun: metacheck's changes since this row was written (data/code extension list and archive download fix, 4 Oct; GROBID DOI-space removal, 4 Oct; new registered hosts and Figshare institutional ids, 6 Oct; Dryad zip members, 10 Oct) were checked against this paper. 109a9aa's link detectors find the same repository links in the paper as the stored run, and none of the changes applies to this paper's files, so this explanation still holds. metacheck does have a dedicated Dryad detector (R/archive-dryad.R in the metacheck-new source tree), and the cited repository — https://doi.org/10.5061/dryad.mb64cr8, redirecting live to datadryad.org/dataset/doi:10.5061/dryad.mb64cr8 — is a real, public, directly-matching Dryad dataset for this exact paper, with no login barrier. The detector nonetheless could not have fired because the actual DOI/URL string is simply absent from the full_text that was extracted from the PDF: the data-availability sentence reads only "Data for this publication are available online at the Dryad Digital" and is then cut off, running straight into the unrelated author-contributions paragraph ("S.P. conceived the ideas..."). No "Repository" word, no "10.5061" string, and no "doi.org" string appear anywhere in full_text. This is a pure text-extraction gap (likely the original PDF rendered the DOI as a hyperlink or footnote-style reference that the extraction pipeline dropped or failed to inline as visible text) — not a missing platform, not a configuration gap for an unrecognized host, and not a bug in the Dryad-matching logic itself, since that logic never receives the string it needs to match against.

Paper (DOI)	Category	Why Metacheck misses it
10.1111/1365-2656.13052	Text extraction: no trace of the citation in the extracted text	CHECKED 2026-10-13 against metacheck 109a9aa by comparison, not by a full rerun: metacheck's changes since this row was written (data/code extension list and archive download fix, 4 Oct; GROBID DOI-space removal, 4 Oct; new registered hosts and Figshare institutional ids, 6 Oct; Dryad zip members, 10 Oct) were checked against this paper. 109a9aa's link detectors find the same repository links in the paper as the stored run, and none of the changes applies to this paper's files, so this explanation still holds. metacheck has a dedicated, working detector for Dryad (R/archive-dryad.R exists alongside the other platform modules such as archive-figshare.R, archive-dataverse.R, etc.), so this is not a platform-support gap. The failure is purely at the text-extraction stage: the DOI string "10.5061/dryad.vc80r13" that identifies this specific Dryad record is absent from the full_text metacheck extracted from the PDF. It appears the source PDF (a Wiley/Journal of Animal Ecology article) had the data-availability sentence and the corresponding reference-list entry each truncated mid-string during text extraction -- the data-availability statement stops at "...available from the Dryad Digital" (no "Repository", no URL), and the reference-list citation "Gibb, H., Grossman, B. F., Dickman, C. R., Decker, D., & Wardle, G. (2019). Data from... Dryad Digital Repository, https://doi.org/10.5061/dryad." is cut off immediately after "dryad." before the record suffix "vc80r13" that makes the DOI resolvable. Since no detector, regex, or URL-matcher can find a string that isn't present in the text it is given, metacheck's extracted_urls list correctly came up empty for this paper even though its Dryad backend would very likely have correctly identified and verified doi:10.5061/dryad.vc80r13 as a live, public dataset had the complete string been present in the extracted text.
10.1111/1365-2656.14093	Text extraction: no trace of the citation in the extracted text	CHECKED 2026-10-13 against metacheck 109a9aa by comparison, not by a full rerun: metacheck's changes since this row was written (data/code extension list and archive download fix, 4 Oct; GROBID DOI-space removal, 4 Oct; new registered hosts and Figshare institutional ids, 6 Oct; Dryad zip members, 10 Oct) were checked against this paper. 109a9aa's link detectors find the same repository links in the paper as the stored run, and none of the changes applies to this paper's files, so this explanation still holds. metacheck does support Figshare as a platform, so if a resolvable Figshare URL or DOI (e.g. a "10.6084/m9.figshare.XXXXXXX" string or a figshare.com/articles/... link) were present in the text, the existing Figshare detector should be able to find and validate it. The actual problem here is upstream of detection: the paper's extracted full_text never contains such a string. The data-availability sentence reads only "Data are archived on Figshare(Mahoney & Pasch, 2024)," citing a reference-list entry that is itself corrupted by the same GROBID-style extraction/OCR process that produced this full_text -- "Mahoney, S. M., & Pasch, B. (2024). , 55(6), 1483-1487." is missing the dataset title and the words "Figshare" or any repository name, leaving only a volume/issue/page triplet. I independently searched Figshare and the open web for a Mahoney & Pasch 2024 dataset matching this paper's topic (mammalian food-caching evolution / Mt. Graham red squirrels) and could not locate one; the only Figshare DOI associated with these two authors (10.6084/m9.figshare.5285635) belongs to their unrelated 2017 "Caching for where and what" paper. Because no machine-resolvable identifier for the actual 2024 dataset exists in the extracted text (and I could not find one by searching, so I cannot even confirm the live dataset is reachable), metacheck's Figshare detector has nothing to act on; this is a text-extraction gap (the citation/DOI was either never present in the PDF's data statement as a hyperlink, or was stripped during PDF-to-text conversion), not a platform-coverage or detector-logic failure. metacheck_any_detector_hit=false is consistent with this: there is no link in the text for any detector, Figshare or otherwise, to find.

Paper (DOI)	Category	Why Metacheck misses it
10.1111/1365-2664.12927	Text extraction: no trace of the citation in the extracted text	CHECKED 2026-10-13 against metacheck 109a9aa by comparison, not by a full rerun: metacheck's changes since this row was written (data/code extension list and archive download fix, 4 Oct; GROBID DOI-space removal, 4 Oct; new registered hosts and Figshare institutional ids, 6 Oct; Dryad zip members, 10 Oct) were checked against this paper. 109a9aa's link detectors find the same repository links in the paper as the stored run, and none of the changes applies to this paper's files, so this explanation still holds. Metacheck's figshare.R has full detector support for this exact institutional instance -- "ore.exeter.ac.uk" (Open Research Exeter's branded Figshare front end) is explicitly listed in .figshare_vanity_hosts(), and the figshare detection regex (.figshare_host_regex()) also matches any literal "figshare.com" substring, which covers the newer "uoe.figshare.com" subdomain this item now lives under. So this is not a platform gap: both the old branded host and the current host are things metacheck's figshare backend already knows how to query via api.figshare.com. The actual gap is at the extraction stage. The paper's data-availability sentence cites only a bare hdl.handle.net/10871/27138 URL -- no figshare.com domain, no figshare DOI (10.6084/m9.figshare... or an institutional DOI prefix), and no bare numeric article ID appear anywhere in the full text. figshare_links() works by regex-matching the paper's own hyperlink table and body text for figshare-looking strings (a figshare host in the URL, or a figshare DOI pattern); it does not follow live HTTP redirects to discover that a handle.net link currently resolves to Figshare. Separately, metacheck has no generic handle.net resolver at all -- the only handle.net handling in the codebase is in archive-psycharchives.R, scoped to PsychArchives' own handle prefix (20.500.12034), which has no bearing on Exeter's prefix (10871). Because the citation text itself never names figshare or exposes a figshare-recognizable identifier, no existing detector -- correctly implemented as each one is for the identifiers it is given -- has anything to match against, so detection fails for a text-extraction reason, not a missing-platform-support reason or a bug in the figshare detector itself.
10.1111/1365-2664.14138	Text extraction: no trace of the citation in the extracted text	CHECKED 2026-10-13 against metacheck 109a9aa by comparison, not by a full rerun: metacheck's changes since this row was written (data/code extension list and archive download fix, 4 Oct; GROBID DOI-space removal, 4 Oct; new registered hosts and Figshare institutional ids, 6 Oct; Dryad zip members, 10 Oct) were checked against this paper. 109a9aa's link detectors find the same repository links in the paper as the stored run, and none of the changes applies to this paper's files, so this explanation still holds. Figshare DOI (10.6084/m9.figshare.16436379.v2) is cited exactly once, inside a formatted 'Data from:' entry in the References list; no DAS section exists in the extracted text at all. Metacheck's repo_check calls text_search() with default include_refs=FALSE, which filters out all rows tagged section_type=='references' (text_search.R lines 104-110) before pattern-matching -- so the citation is structurally invisible to the default search regardless of host support. Metacheck's Figshare backend is otherwise fully functional and would resolve this DOI if the text reached it (confirmed live via api.figshare.com/v2/articles/16436379, HTTP 200, CC BY 4.0).

Paper (DOI)	Category	Why Metacheck misses it
10.1002/pan3.10756	Metacheck code: a defect in detecting or retrieving a supported repository	RECHECKED 2026-10-13 (per-paper rerun, metacheck 109a9aa): still no files. The cause is deterministic, not intermittent and not a batch artifact. The DOI 10.23634/MMU.00636962 resolves to Figshare article 32616504 on repository.mmu.ac.uk, but .figshare_id() takes the DOI suffix 00636962 as the article id; api.figshare.com/v2/articles/32616504 answers 200 and the id metacheck uses answers 404, so figshare_info() returns 'unfound' (the same class of problem as metacheck issue #439, institutional Figshare DOI shapes). The 'arguments imply differing number of rows' message in gated_repos only hides that 'unfound' (metacheck issue #476). Earlier explanation (superseded): figshare_links() correctly detects the DOI citation via the institutional vanity-host/DOI-prefix table (repository.mmu.ac.uk / 10.23634, both explicitly registered in .figshare_vanity_hosts() and .figshare_doi_prefix_hosts()), and live, direct invocation of the installed package's repo_check() on this single paper today DOES detect the repo (repo_n = 2, two duplicate entries differing only by a trailing citation-punctuation period) -- directly contradicting the repo_n = 0 stored in res_repo_check.RData. The live run's gated_repos shows repo_error = 'arguments imply differing number of rows' for this paper's own figshare id lookup (one live attempt reported '00636962 could not be found on api.figshare.com'), suggesting the same batch-level tryCatch/data.frame-construction fragility identified for 10.1111/1365-2745.13728 is also implicated here, compounded by apparent intermittency in api.figshare.com's own response reliability for this specific institutional-DOI lookup path (a second live attempt via a different code path, querying the article id 32616504 directly, succeeded). The stored repo_n = 0 is best explained as a stale/incorrect batch-run artifact layered on top of a real but intermittent figshare-institutional-DOI lookup fragility, not a permanent inability to detect or resolve this repository.
10.1111/1365-2435.13103	Metacheck code: a defect in detecting or retrieving a supported repository	RECHECKED 2026-10-13 (per-paper rerun, metacheck 109a9aa): still no files. The data statement gives two links: a private Figshare share link (figshare.com/s/d1b7fd290c32cfac593b), which cannot be resolved by design, and a public DOI, 10.4225/03/5aa1d20d4e544, which resolves to Figshare article 5932342 on bridges.monash.edu (Monash's Figshare). Metacheck detects only the share link. It misses the public DOI because 10.4225 is the shared prefix of the Australian National Data Service, not a Figshare prefix, and metacheck does not resolve an unregistered DOI prefix to a host, although bridges.monash.edu itself is a registered Figshare host: a metacheck code limitation. The 'corpus-wide figshare-DOI batch crash' in the earlier explanation is wrong: that error message only replaces the real error when every Figshare link on the paper fails, and here both detected links are the unresolvable share link (metacheck issue #476). Earlier explanation (superseded): files_n = 0 directly contradicts the prior audit's assumption that detection succeeding meant the gap was closed. The resolvable DOI (10.4225/...) is a genuine Bridges/Figshare DOI that metacheck's dataverse/figshare DOI-prefix tables should in principle be able to query for file listings, but because the concatenated /s/ share-link token was also extracted as a second repo_url and is unresolvable by design (opaque token, no API endpoint for third-party resolution -- same limitation as .figshare_id() documents elsewhere), and because the batch-level figshare DOI retrieval call that would otherwise fetch files for the whole corpus is itself broken by a separate code defect (see 10.1111/1365-2664.14575 below, which shares the identical 'arguments imply differing number of rows' failure signature across the entire figshare-DOI batch), no files were enumerated for either repo_url belonging to this paper. This combines two real, separate code-level gaps -- the /s/ token's unresolvable-by-design limitation, and the corpus-wide figshare-DOI batch crash -- both of which are genuine platform bugs, not a non-issue.

Paper (DOI)	Category	Why Metacheck misses it
10.1111/1365-2435.13360	Metacheck code: a defect in detecting or retrieving a supported repository	metacheck fully supports Dataverse as a platform, and dataverse.nl specifically is one of its individually-verified known hosts (R/archive-dataverse.R, .dataverse_hosts() line 64), with its DOI prefix 10.34894 even already mapped to it in .dataverse_doi_prefix_hosts() (line 303). The live chain confirms this paper's dataset genuinely resolves to dataverse.nl under DOI 10.34894/UUNBQQ, which is exactly the registered prefix. The gap is that the paper's text never contains either the host string "dataverse.nl" or the DOI string "10.34894" — it cites the dataset only as the bare handle-resolver URL "https://hdl.handle.net/10411/6Q1YDC". dataverse_links() has three matching passes (real-hyperlink host match, bare-text host match, bare-DOI-prefix match), and all three require either the dataverse.nl hostname or a 10.34894-prefixed DOI to literally appear in the extracted text/href -- neither does, since handle.net URLs use their own handle-prefix namespace (here 10411, DANS's generic handle prefix) which is unrelated to the target repository's DOI prefix. Metacheck does have precedent for resolving a specific hdl.handle.net prefix to a platform (archive-psycharchives.R maps handle prefix 20.500.12034 to PsychArchives), but no equivalent mapping exists for DANS's 10411 prefix pointing at DataverseNL (or at DANS's other dataverse-based Data Stations). Resolving this would require either (a) adding 10411 as a recognized handle prefix that redirects to a Dataverse-family host, or (b) following the live HTTP redirect chain for any hdl.handle.net citation before applying the existing host/DOI-prefix matchers -- neither of which the current static-text extraction does for this handle prefix. [RECLASSIFIED from SUPPORTED_PLATFORM_CONFIG_GAP: not a host/DOI-prefix table entry -- requires a new resolution mechanism (handle-prefix redirect, shortlink expansion, or DOI-to-host resolution) or the host runs an API shape metacheck does not support.]
10.1111/1365-2435.13523	Metacheck code: a defect in detecting or retrieving a supported repository	RECLASSIFIED 2026-10-13 (SUPPORTED_PLATFORM_BUG, was SUPPORTED_PLATFORM_CONFIG_GAP): UNSWorks is supported, but the paper cites it through UNSW's Handle redirector (handle.unsw.edu.au), which metacheck does not follow: a metacheck code defect. (The redirector answered 503 on 2026-10-13.) Earlier explanation: metacheck has a working DSpace 7 detector (R/archive-dspace7.R) and its host allowlist, .dspace7_hosts(), already includes "unsworks.unsw.edu.au" (line 106) -- so if the paper had cited the unsworks.unsw.edu.au URL directly, dspace7_links() would find it. The actual problem is that this paper's data-availability statement cites a different, un-registered domain: "http://handle.unsw.edu.au/1959.4/resource/collection/resdatac_1003/1". UNSW operates handle.unsw.edu.au as a separate CNRI Handle-System redirector in front of UNSWorks; it 302-redirects to the unsworks.unsw.edu.au item page but is a distinct hostname that never appears in .dspace7_hosts(). dspace7_links() checks three paths: (1) real hyperlink hrefs matching a known host -- fails, handle.unsw.edu.au isn't in the list; (2) a bare-text mention of a known host string -- fails, same reason, the text only contains "handle.unsw.edu.au", never "unsworks.unsw.edu.au"; (3) a bare DOI-shaped string ("10.xxxx/yyyy") resolved live and checked against the known-host list -- also fails, because the citation in the paper's text is a Handle-resolver URL with no "10." DOI prefix anywhere in it (the item's real DOI, 10.26190/5e1ce7350483e, only appears on the landing page itself, never in the citing paper's text). So all three of dspace7_links()'s detection paths miss this specific citation even though the underlying repository (unsworks.unsw.edu.au, DSpace 7) is fully supported. This is a configuration gap, not a missing platform or a code bug: the fix would be adding "handle.unsw.edu.au" (or a generic handle.* / hdl.handle.net resolver pattern) to the host recognition logic, analogous to how issue #435 already added live DOI-redirect resolution for the no-host-string-at-all case. [CONFIG_GAP_TIER: needs_code_change -- rechecked 2026-10-13 against metacheck 109a9aa and the live repository: Needs a Handle redirect (handle.unsw.edu.au) resolved before matching UNSWorks, which metacheck supports. The redirector returned 503 Service Unavailable on 2026-10-13, and the cited path is a collection, not a single item.]

Paper (DOI)	Category	Why Metacheck misses it
10.1111/1365-2435.13850	Metacheck code: a defect in detecting or retrieving a supported repository	metacheck fully supports Figshare and even has explicit, dedicated support for exactly this host and DOI prefix: R/archive-figshare.R's .figshare_vanity_hosts() lists "smithsonian.figshare.com" by name (line 97), and .figshare_doi_prefix_hosts() maps DOI prefix "10.25573" to "smithsonian.figshare.com" (line 136). The citation text is also cleanly extractable with no garbling. The actual bug is in ID-type resolution: the cited DOI, https://doi.org/10.25573/data.c.5073065.v1, is a Figshare COLLECTION DOI (identifiable by the literal ".c." segment after the DOI prefix, the same marker .figshare_collection_id() uses for the generic 10.6084/m9.figshare.c.<id> case), but .figshare_id()'s institutional-prefix pattern (archive-figshare.R lines 327-344) does not exclude or recognize this ".c." collection marker for institutional prefixes -- it simply strips any non-digit-containing sub-segment and returns the trailing digits, "5073065", as if it were an ARTICLE id. I confirmed this live: devtools::load_all() then .figshare_id(url) returns "5073065", but .figshare_collection_id(url) returns NA (its own pattern only checks the literal 10.6084 prefix, not institutional ones like 10.25573, per the comment at lines 460-463 of the same file: "Only the literal 10.6084 prefix is checked here... no institutional-collection-DOI citation has been found yet"). Because figshare_info() checks ids$figshare_id first and only falls through to collection-expansion logic when figshare_id is NA (line 591: "unresolved <- is.na(ids$figshare_id)"), it never tries the collection path for this URL -- it instead calls GET https://api.figshare.com/v2/articles/5073065 directly, which I confirmed live returns HTTP 404 (while GET https://api.figshare.com/v2/collections/5073065 returns HTTP 200). Running figshare_info() on this exact URL in the live package reproduces the failure: it returns a row with error = "unfound" and a warning "5073065 could not be found on api.figshare.com". This directly contradicts the payload's metacheck_any_detector_hit = true for this paper: the one link actually cited in the text is not resolvable by the current code, because institutional-prefix DOIs for Figshare collections are not yet recognized as collections the way bare 10.6084/m9.figshare.c.<id> DOIs are.
10.1111/1365-2435.14284	Metacheck code: a defect in detecting or retrieving a supported repository	Zenodo is a fully supported platform (R/archive-zenodo.R) and the Zenodo DOI is extractable: I located the exact string "https://doi.org/10.5281/zenodo.7573829" in full_text, clean and un-garbled, matching the coder's recorded link exactly. The data-availability sentence itself reads "All data and associated scripts used in this study are available on" with the actual link/citation marker stripped out (a GROBID artifact from resolving an inline citation) -- the literal DOI string only survives later, embedded in the formatted bibliography as the self-citation entry "Shah, A. A., Hotaling, S., ... (2023). ... Zenodo. https://doi.org/10.5281/zenodo.7573829". This is why metacheck's own detector misses it despite the platform and pattern being fully supported: zenodo_links() in R/archive-zenodo.R calls text_search(paper, zen_bare_regex, return = "match", perl = TRUE) without passing include_refs = TRUE, and text_search() in R/text_search.R defaults include_refs to FALSE, which explicitly filters out any text row where section_type == "references" (lines 104-109). Since the only textual occurrence of this DOI in the paper is inside a reference-list entry (and the paper's extracted url/href table apparently has no corresponding hyperlink either, since found_href would otherwise have caught it regardless of section), the bare-DOI body-text fallback in zenodo_links() never sees it, and the detector returns no hit even though both the platform support and the raw text content are present and intact. This is a genuine config/argument gap in a supported detector, not a missing platform or a garbled citation.

Paper (DOI)	Category	Why Metacheck misses it
10.1111/1365-2656.13005	Metacheck code: a defect in detecting or retrieving a supported repository	RECHECKED 2026-10-13 (per-paper rerun, metacheck 109a9aa): still no files. The cause is now known. The PDF-to-text step split the data statement: 'The data used in this study are available from the Dryad Digital' stays in the Data Accessibility section, but its continuation 'Repository: https ://doi.org/10.5061/dryad.m20hf10 (Naniwadekar et al., 2019)' was placed in the References section. dryad_links() searches body text (text_search() default include_ refs = FALSE) and the link table (no hyperlink for this DOI), so the DOI, although intact and matched by the Dryad pattern, is never seen. Metacheck's code limitation is not searching the reference list. Earlier explanation (pattern tested and works, cause not found): The data-availability sentence in the paper's full text reads: "The data used in this study are available from the Dryad Digital [Repository]... Repository: https ://doi.org/10.5061/dryad.m20hf10 (Naniwadekar, Chaplod, Datta, Rathore, & Sridhar, 2019)." metacheck has a dedicated Dryad detector (R/archive-dryad.R), and 10.5061 is the FIRST and primary entry in .dryad_doi_prefixes(), with its own purpose-built body-text regex "(?:https?://)?(?:doi\ \ .org/)?10\ \ .5061/dryad\ \ .[A-Za-z0-9]+" in dryad_bare_regex. I ran this exact regex in R against the exact text string from the paper (including the stray "https ://" space artifact from PDF extraction) and it matched successfully, returning "doi.org/10.5061/dryad.m20hf10" -- the leading optional "https?://" group simply fails to consume the malformed "https ://" and the engine falls through to matching starting at "doi.org/10.5061/...", which is literally present with no intervening whitespace. Since the regex is not the point of failure on the text I was given, yet metacheck_any_detector_hit is reported as False for this paper, the gap must lie elsewhere in the pipeline between GROBID text extraction and the detector call (e.g. the real GROBID output differing from this harness's full_text in whitespace/hyphenation around the DOI, or a failure in the live dryad_info() lookup/repo_check step rather than in link detection itself). This is a supported-platform bug rather than a missing-text or platform-gap issue: the correct, resolvable DOI is present in the text in a form the detector's own regex is proven to match, and the live Dryad page is fully public with the exact matching dataset.

Paper (DOI)	Category	Why Metacheck misses it
10.1111/1365-2664.13035	Metacheck code: a defect in detecting or retrieving a supported repository	RECLASSIFIED 2026-10-13 (SUPPORTED_PLATFORM_BUG, was SUPPORTED_PLATFORM_CONFIG_GAP): Supported, FAIR repository (dataSTORRE, host registered in metacheck) cited through an hdl.handle.net link, which metacheck does not follow to the host (metacheck issue #466): a metacheck code defect. Earlier explanation: CONFIRMED CONFIG GAP, still present. The citation ('Data in this study are available from DataSTORRE... http://hdl.handle.net/11667/102') uses a bare Handle System URL, not a DOI, so it can only be recognised via a host match. datastorre.stir.ac.uk is absent from BOTH .dspace7_hosts() (R/archive-dspace7.R, 63 verified installations) and .dspace_legacy_hosts() (R/archive-psycharchives.R, 6 installations) -- confirmed by direct search, zero matches for 'datastorre' or 'stir.ac.uk' in either file. dspace7_links()'s DOI-resolution fallback (which CAN catch a bare DOI whose host isn't separately listed, by live-resolving it and checking where it redirects) only matches DOI-shaped strings ('10.xxxx/...'); a hdl.handle.net URL is not DOI-shaped, so even that fallback never fires. The repository platform (DSpace) is generally supported, but this specific host was simply never added to either allowlist -- a straightforward config/allowlist omission, not a logic defect. CORRECTION 2026-10-07: a live rerun confirmed datastorre.stir.ac.uk IS now present in .dspace_legacy_hosts(), but still produced repo_n = 0, files_n = 0 for this paper. This is metacheck issue #466 (hdl.handle.net redirect resolution for DSpace legacy), a separate, still-open bug explicitly distinguished from the host-registry gap this row originally described -- confirmed by 01_run_metacheck/08_rerun_issue_461_registry_papers.R's own header comment, which excluded this exact paper from that rerun for this exact reason. The host addition alone does not resolve a hdl.handle.net-style citation. [CONFIG_GAP_TIER: needs_code_change -- rechecked 2026-10-13 against metacheck 109a9aa and the live repository: The host (datastorre.stir.ac.uk) is now registered in metacheck, but the paper cites it through an hdl.handle.net link, which metacheck does not resolve to the host (metacheck issue #466, open on 2026-10-13).]

Paper (DOI)	Category	Why Metacheck misses it
10.1111/1365-2664.14198	Metacheck code: a defect in detecting or retrieving a supported repository	RECLASSIFIED 2026-10-13 (SUPPORTED_PLATFORM_BUG, was SUPPORTED_PLATFORM_CONFIG_GAP): Supported, FAIR repository (dataSTORRE, host registered in metacheck) cited through an hdl.handle.net link, which metacheck does not follow to the host (metacheck issue #466): a metacheck code defect. Earlier explanation: metacheck has real, working DSpace support split across two code paths: R/archive-dspace7.R (DSpace 7+, HAL-JSON /server/api/) and R/archive-psycharchives.R's .dspace_legacy_hosts() (DSpace <=6, legacy /rest/ API). Both paths work only against an explicit allowlist of installation hostnames rather than detecting DSpace generically from the resolved URL. I read both host lists directly from source: .dspace7_hosts() in archive-dspace7.R (64 hosts, e.g. repository.gatech.edu, aura.abdn.ac.uk) and .dspace_legacy_hosts() in archive-psycharchives.R (6 hosts: www.psycharchives.org, bonndoc.ulb.uni-bonn.de, dspace.ut.ee, qsardb.org, repositorio-digital.cide.edu, ses.library.usyd.edu.au). The paper's repository link resolves to datastorre.stir.ac.uk (University of Stirling's dataSTORRE, confirmed live to be running DSpace), which appears in neither list. Because both dspace7_links() and dspace_links() match only against their respective host allowlists (via .dspace7_host_regex()/.dspace_legacy_host_regex()), a citation to http://hdl.handle.net/11667/195 -- which does carry a resolvable handle and does redirect to a real, public, CC-BY-licensed DSpace item with a listed data file -- is invisible to metacheck simply because datastorre.stir.ac.uk was never added to either registry. This is exactly the SUPPORTED_PLATFORM_CONFIG_GAP case: the platform family (DSpace) is supported, extraction of the citation is trivial (a single unambiguous handle link appears twice in the text), but this specific host is unregistered. CORRECTION 2026-10-07: same as 10.1111/1365-2664.13035 -- a live rerun confirmed the datastorre.stir.ac.uk host addition did not resolve this paper (repo_n = 0, files_n = 0), because this citation is also a bare hdl.handle.net URL, blocked by the same open issue #466 (handle-redirect resolution), not by the host registry gap this row originally described. [CONFIG_GAP_TIER: needs_code_change -- rechecked 2026-10-13 against metacheck 109a9aa and the live repository: The host (datastorre.stir.ac.uk) is now registered in metacheck, but the paper cites it through an hdl.handle.net link, which metacheck does not resolve to the host (metacheck issue #466, open on 2026-10-13).]

Paper (DOI)	Category	Why Metacheck misses it
10.1111/1365-2745.12882	Metacheck code: a defect in detecting or retrieving a supported repository	RECHECKED 2026-10-13 (per-paper rerun, metacheck 109a9aa): still no files. The cause is the urn:uuid identifier alone: .dataone_pid() extracts only doi: identifiers and returns NA for this link, so dataone_info() returns 'unfound', although the Arctic Data Center API answers 200 for meta/urn:uuid:3dc91064-7b8b-4308-b949-a361b7146baa. The batch-crash part of the earlier explanation is wrong: the 'arguments imply differing number of rows' message only replaces this paper's own 'unfound' (metacheck issue #476) and did not come from another paper. Earlier explanation (superseded): Two distinct issues compound here. First, a narrower, previously-identified limitation (documented in the prior audit and still accurate): the citation uses a bare urn:uuid identifier with no DOI anywhere in the data-availability statement, and metacheck's .dataone_pid() (R/archive-dataone.R) only has regex branches for doi:10.x/... and bare-DOI forms, not for a top-level urn:uuid citation PID -- so dataone_info()'s validity filter (requiring both a non-NA host AND a non-NA pid) would exclude this specific citation from its live per-record API lookup regardless of any other bug. Second, and more decisively given that files_n = 0 for every one of these 8 papers: res_repo_check$gated_repos shows the SAME 'arguments imply differing number of rows' error (with varying counts) attached to every dataone-flagged repo_url across the corpus (4 rows total, from only 2-3 papers using DataONE), exactly mirroring the dataverse and figshare-DOI batch-poisoning defect in inst/modules/repo_check.R's DataONE block (dataone_info() called once per corpus batch inside one tryCatch; one bad record anywhere in the batch crashes the shared call, and the error message is stamped onto every dataone repo_url via repos$repo_error[repos$repo_url %in% dataone_urls] <<- conditionMessage(e)). Because this paper's own citation would already be filtered out of the 'valid' (pid-resolvable) set before ever reaching the API call, the specific batch crash could in principle have originated from one of the OTHER dataone-citing papers' (the two 10.5063/f1... DOIs) records. Either way, files_n = 0 is confirmed, and the correct classification is a genuine code-level platform bug (both the urn:uuid PID-extraction gap and the batch-wide crash handling), not a non-issue.

Paper (DOI)	Category	Why Metacheck misses it
10.1111/2041-210X.13073	Metacheck code: a defect in detecting or retrieving a supported repository	RECHECKED 2026-10-13 (per-paper rerun, metacheck 109a9aa): still no files. The cause is now known. In the extracted text the data statement 'Data and MATLAB script available from the Dryad Digital' is cut off and followed by unrelated text, and the DOI 10.5061/dryad.g66247g appears only in the reference-list entry (Endler, Cole & Kranz, 2018), with no hyperlink. dryad_links() searches body text (text_search() default include_refs = FALSE) and the link table, so it never sees the DOI. Metacheck's code limitation is not searching the reference list. Earlier explanation (pattern tested and works, cause not found): Metacheck has a dedicated Dryad module (R/archive-dryad.R) with dryad_links()/dryad_info(), wired into inst/modules/repo_check.R, and the module's own bare-DOI regex ("(?:https?://)?(?:doi\ \ .org/)?10\ \ .5061/dryad\ \ .[A-Za-z0-9]+") is a perfect, standard match for this citation -- I ran the exact regex from the source file against both the short body-text fragment ("Dryad Digital Repository, https://doi.org/10.5061/dryad.g66247g") and the full reference-list sentence from this paper, and in both cases it correctly extracted "https://doi.org/10.5061/dryad.g66247g" verbatim. The live DOI resolves (302) to a fully public, non-login datadryad.org dataset page with three files listed, so the repository itself is genuinely live and accessible -- this is not a dead link. Since the regex match succeeds in isolation but metacheck_any_detector_hit was reported false for the real pipeline run on this paper, the gap is not that Dryad is unsupported or that this DOI prefix/pattern is unregistered (10.5061/dryad. is exactly the pattern archive-dryad.R is built around); it must instead be that dryad_ links()'s two input sources -- paper_table(paper, "url") (real anchor-tag hrefs extracted from the source PDF/XML) and text_search(paper, ...) over metacheck's own parsed-paper text representation -- did not surface this particular sentence to the regex in the actual extraction pipeline (e.g., a GROBID/PDF-parsing segmentation artifact placing the citation in a block text_search does not scan, or the real paper's extracted text differing from the flat full_text string used here). Because the detector logic is demonstrably correct and the platform is supported, but the real run still missed a citation its own regex can parse, I classify this as a SUPPORTED_PLATFORM_BUG (an upstream text/URL-extraction issue feeding the Dryad detector) rather than a missing config or missing platform.
10.1111/2041-210X.13428	Metacheck code: a defect in detecting or retrieving a supported repository	CONFIRMED CODE BUG, still present. The paper's data-availability sentence ('...has been made publicly available on http://Zenodo.org') cites only the generic Zenodo homepage; the ACTUAL resolvable citation, the Zenodo DOI (10.5281/zenodo.3826426), appears only in the reference list entry for Arje et al. (2020). zenodo_links() DOES include a working regex for exactly this DOI shape (confirmed by direct test: .zenodo_id() correctly resolves this exact DOI string to id 3826426 when tested directly), but it calls text_search(paper, zen_bare_regex, return='match', perl=TRUE) WITHOUT passing include_refs=TRUE. text_search()'s default (include_refs=FALSE, R/text_search.R) explicitly filters out any text row whose section_type is 'references' before searching -- confirmed directly: the row containing the DOI has section_type=='references', and calling text_search() on this paper with the same regex returns 0 rows, while testing the identical regex directly against the raw row string (bypassing text_search()'s section filter) matches it immediately. So the one genuinely resolvable Zenodo citation in this paper is systematically invisible to the detector because it sits in the reference list rather than the body text, and only the unresolvable bare homepage mention is ever found instead.

Paper (DOI)	Category	Why Metacheck misses it
10.1111/2041-210X.13547	Metacheck code: a defect in detecting or retrieving a supported repository	RECLASSIFIED 2026-10-13 (SUPPORTED_PLATFORM_BUG, was SUPPORTED_PLATFORM_CONFIG_GAP): Zenodo is supported and the DOIs are clean, but they appear only in the reference list, which metacheck's Zenodo detector does not search: a metacheck code defect. Earlier explanation: Zenodo is a fully supported platform in metacheck (R/archive-zenodo.R, zenodo_links()). That function already implements a deliberate two-tier search: real hyperlinks from the paper's url table, plus a body-text fallback regex for bare DOIs like "10.5281/zenodo.NNNNNNN" that appear without a hyperlink. The problem for this specific paper is that the fallback regex is run through text_search(paper, zen_bare_regex, return = "match", perl = TRUE) without passing include_refs = TRUE, and text_search() defaults to include_refs = FALSE, which explicitly filters out any text row whose section_type is "references" (see text_search.R, the `section_filter <- !text$section_type %in% "references"` line, used whenever include_refs is not set to TRUE). In this paper, the inline data-availability sentence is a bare author-year citation with no DOI or URL at all ("The Paris 12 dataset is available on Zenodo (Machon, 2020)."; similarly for the code, "available on Zenodo (Louvet, 2020), as well as on a GitHub repository"), and the only place the actual resolvable Zenodo DOIs (10.5281/zenodo.3770339 and 10.5281/zenodo.4314518) appear anywhere in the extracted text is inside the References section's bibliography entries. Since zenodo_links() never searches the references section for its bare-DOI fallback, it finds no hyperlink and no in-body bare DOI, so it returns zero Zenodo URLs for this paper even though both repositories are live, public Zenodo records with files. This is not a missing backend (Zenodo is fully implemented) and not a garbled DOI (the DOIs are intact and correctly formatted) -- it is a configuration/search-scope gap in how the existing Zenodo detector decides which part of the document to scan for bare identifiers, which this specific paper's citation style (name-year in text, DOI only in the reference list) exposes. [CONFIG_GAP_TIER: needs_code_change -- rechecked 2026-10-13 against metacheck 109a9aa and the live repository: The Zenodo DOIs are only in the reference list, which metacheck's Zenodo detector does not search.]
10.1111/2041-210X.13768	Metacheck code: a defect in detecting or retrieving a supported repository	RECHECKED 2026-10-13 (per-paper rerun, metacheck 109a9aa): still no files. The cause is now known. The extracted Data Availability Statement section contains methods text instead of the statement (PDF reflow), and the code deposit DOI 10.5281/zenodo.5562809 appears only in the reference-list entry (Elhamod et al., 2021), with no hyperlink. zenodo_links() searches body text (text_search() default include_refs = FALSE) and the link table, so it never sees the DOI. Metacheck's code limitation is not searching the reference list. Earlier explanation (cause not found): Paper's formal DAS text is scrambled by PDF-extraction reflow, but the code deposit self-citation (DOI 10.5281/zenodo.5562809) survives intact in the reference list. Metacheck's zenodo_links() has a documented bare-DOI fallback regex specifically built for exactly this citation style, and the DOI is neither garbled nor truncated. Live-verified the deposit is public, the zip supports HTTP range requests, and data_check_helpers.R explicitly classifies .py files as 'code' -- every mechanism needed should work. No specific bug could be pinpointed from static source review alone; this would need an actual pipeline rerun to confirm where/if it diverges.

Paper (DOI)	Category	Why Metacheck misses it
10.1111/2041-210X.13785	Metacheck code: a defect in detecting or retrieving a supported repository	RECHECKED 2026-10-13 (per-paper rerun, metacheck 109a9aa): still no files. The cause is known and is not a pipeline bug: the availability statement cites the deposits only as '(Xirocostas et al., 2021)' and '(Debono, 2021)', and both Zenodo DOIs (10.5281/zenodo.5746183 and 10.5281/zenodo.5746655) appear only in the reference list, with no hyperlink. zenodo_links() searches body text (text_search() default include_refs = FALSE) and the link table, so it never sees them, as for 10.1111/2041-210X.13547. Earlier explanation (superseded): Zenodo is a fully supported platform in metacheck, implemented in R/archive-zenodo.R. The availability statement reads "Code and data for all analyses (Xirocostas et al., 2021) and the ZAX Herbivory Trainer (Debono, 2021) are archived on Zenodo," and the reference list spells out both DOIs cleanly and completely: "https://doi.org/10.5281/zenodo.5746183" and "https://doi.org/10.5281/zenodo.5746655" -- neither is garbled, split, or missing whitespace. metacheck's own zenodo_links() bare-text fallback regex is `(?:https?://)?(?:doi\ .org/)?10\ .5281/zenodo\ .[0-9]+`, which matches these two strings exactly as they appear in the extracted text (I checked this against the regex by inspection: "https://doi.org/10.5281/zenodo.5746183" satisfies every token of the pattern with no mismatch). Both DOIs resolve live to real, public, open Zenodo records with listed files. Since the platform is supported, the regex should match the citation form actually used, and the repositories are genuinely live and accessible, yet metacheck_any_detector_hit is reported as false, the most likely explanation is a bug somewhere in the detection pipeline between text/URL extraction and the regex match or the live Zenodo API lookup (e.g., the GROBID-extracted paper object's `url` table or body text segmentation for this specific PDF, or a failure in the zenodo_info() API call for these two specific records) rather than a missing platform or a textual extraction gap, since the text I was handed contains the DOIs in good, unambiguous condition.
10.1111/2041-210X.13865	Metacheck code: a defect in detecting or retrieving a supported repository	RECHECKED 2026-10-13 (per-paper rerun, metacheck 109a9aa): still no files. The cause is now known. The paper points to its deposit only by citation ('available at Ariza et al., 2022'; 'Raw sequences ... and bash and R codes are available in Ariza et al. (2022)'), and the Zenodo DOI 10.5281/zenodo.6400498 appears only in the reference list, with no hyperlink. zenodo_links() searches body text (text_search() default include_refs = FALSE) and the link table, so it never sees the DOI. Metacheck's code limitation is not searching the reference list. Earlier explanation (cause not found): No DAS exists in the extracted text; the only repository mention is a self-citation reference-list entry for Zenodo DOI 10.5281/zenodo.6400498, which appears verbatim and unmangled. Metacheck's zenodo_links() bare-text fallback regex is explicitly designed to catch exactly this citation style. Live-verified the Zenodo API record returns a complete file listing normally. Given the DOI is clean, the regex matches it, the API record is live, and the backend code exists and works for hundreds of other Zenodo papers in this corpus, this is best explained as an execution-time pipeline bug for this specific paper rather than a missing capability or config/URL-pattern gap.

Paper (DOI)	Category	Why Metacheck misses it
10.1111/1365-2435.14350	Metacheck code: archive listed but never downloaded or opened	RECHECKED 2026-10-13 (per-paper rerun, metacheck 109a9aa): still no files classified. Metacheck tried to open the .tar.gz, and tar.exe again reported 'Archive entry has empty or unreadable filename', as in the 2026-10-07 correction below. The repo's Wieczynski_etal_2023_FE_Master.tar.gz genuinely contains a Mathematica model-code notebook plus 30 .csv data files (confirmed by downloading and unpacking it directly). .tar.gz IS recognized as a readable archive by .is_tar_archive(), but under the production download='data' mode, the baseline want formula never sets want=TRUE for an 'unknown'-typed row, and the rescue line 'want <- want & (!(data_type %in% never_fetch) | keep_ archive)' is an AND that can only narrow an already-true want, never promote a false one to true -- confirmed by re-executing the formula in R. So the archive is never downloaded or expanded. CORRECTION 2026-10-07: a live rerun confirmed the archive-promotion AND/OR fix (commit e4b74e1) does make metacheck attempt to open this .tar.gz, but the actual extraction still fails -- the rerun logged 'tar.exe: Archive entry has empty or unreadable filename ... skipping' and the resulting file list contains only a README.md and one other entry, data_type/files_data/files_ code = 0/0/0, not the 30 .csv data files and Mathematica notebook this row originally described. The want-formula bug is fixed, but a second, separate defect in tar extraction itself (likely a long-path, special-character, or non-ustar-header issue in this specific .tar.gz) still prevents the real contents from being enumerated.
10.1111/2041-210X.12820	Metacheck code: archive listed but never downloaded or opened	ADDED 2026-10-12 after the archive-download rerun (script 13, then script 14 with every rate limit waited out). repo_ check finds the Dryad dataset (doi:10.5061/dryad.j2v6q) and lists its README plus two members of the dataset archive, 'A_Simulated Data.zip' and 'B_Empirical data.zip' -- archives inside the dataset archive. Metacheck reads only one level of archive (intended behaviour, confirmed by the Metacheck developer), so these inner archives are neither opened nor counted as data, and data_check classifies no file as data. The earlier corpus run (older metacheck) had counted the two inner .zip files themselves as data files, which is why this paper was not in the population before.

Table 4 lists, in the same way, every one of the 26 papers in the main text's cluster of repositories that contain data or code Metacheck did not classify: 16 where the data or code were not shared in a FAIR format, and 10 where they are in a format Metacheck does not yet support.

Table 4: Full paper-by-paper detail for every paper whose repository contains data or code that Metacheck did not classify.

Paper (DOI)	Category	Why Metacheck misses it
10.1002/2688-8319.12149	Data or code not shared in a FAIR format	CHECKED 2026-10-13 against metacheck 109a9aa by comparison, not by a full rerun: metacheck's changes since this row was written (data/code extension list and archive download fix, 4 Oct; GROBID DOI-space removal, 4 Oct; new registered hosts and Figshare institutional ids, 6 Oct; Dryad zip members, 10 Oct) were checked against this paper. 109a9aa's link detectors find the same repository links in the paper as the stored run, and none of the changes applies to this paper's files, so this explanation still holds. RECLASSIFIED 2026-10-08 (DATA_OR_CODE_NOT_IN_FAIR_FORMAT): The data file is a rich-text (.rtf) document rather than a machine-readable data file. Earlier explanation: The deposit's two files (both .rtf) are the genuine data file (moose density estimates) and its README; both land on 'documentation' because .rtf has no entry in .ext_registry. file_category() returns NA for the data file so Tier 2 doesn't fire either; Tier 3's coarse crosswalk maps .rtf's coarse type 'text' to 'documentation', which correctly identifies the README but incorrectly also catches the actual data file saved in the same format.
10.1002/pan3.10334	Data or code not shared in a FAIR format	CHECKED 2026-10-13 against metacheck 109a9aa by comparison, not by a full rerun: metacheck's changes since this row was written (data/code extension list and archive download fix, 4 Oct; GROBID DOI-space removal, 4 Oct; new registered hosts and Figshare institutional ids, 6 Oct; Dryad zip members, 10 Oct) were checked against this paper. 109a9aa's link detectors find the same repository links in the paper as the stored run, and none of the changes applies to this paper's files, so this explanation still holds. RECLASSIFIED 2026-10-08 (DATA_OR_CODE_NOT_IN_FAIR_FORMAT): The data (letter transcripts) are .docx files named by anonymised codes, not labelled as data. Earlier explanation: All 39 files in the OSF repo are .docx anonymized individual letter transcripts, confirmed by the OSF project's own title and description to be the actual dataset. .docx has no row in .ext_registry, file_category() returns NA for these filenames, and none contain a Tier-2 keyword token (anonymized site codes, e.g. '37yam_3_1.docx'). Tier 3's coarse crosswalk resolves .docx to coarse type 'text', mapped to 'documentation'. Directly reproduced empirically: data_classify_files() on three real filenames returned 'documentation' for all three.
10.1002/pan3.6	Data or code not shared in a FAIR format	CHECKED 2026-10-13 against metacheck 109a9aa by comparison, not by a full rerun: metacheck's changes since this row was written (data/code extension list and archive download fix, 4 Oct; GROBID DOI-space removal, 4 Oct; new registered hosts and Figshare institutional ids, 6 Oct; Dryad zip members, 10 Oct) were checked against this paper. 109a9aa's link detectors find the same repository links in the paper as the stored run, and none of the changes applies to this paper's files, so this explanation still holds. RECLASSIFIED 2026-10-08 (DATA_OR_CODE_NOT_IN_FAIR_FORMAT): The data (interview transcripts) are shared as PDF files whose names (CP01.pdf ...) do not label them as data. Earlier explanation: All 33 genuine interview-transcript files (CP01.pdf...CP35.pdf, redacted/anonymized, exactly matching the paper's stated raw data) are classified 'documentation'. .pdf has no row in .ext_registry (deliberately left unlocked, since a PDF can be materials/documentation/preprint depending on content). Filenames like 'CP01.pdf' contain no Tier-2 keyword token, so Tier 3's coarse crosswalk fires: metacheck::file_types maps .pdf to coarse type 'text', and the crosswalk maps text->'documentation'. This is a real miss on genuinely primary raw data (anonymized qualitative transcripts), not materials/questionnaires/preprints, with no keyword signal to rescue it.

Paper (DOI)	Category	Why Metacheck misses it
10.1111/1365-2435.13500	Data or code not shared in a FAIR format	CHECKED 2026-10-13 against metacheck 109a9aa by comparison, not by a full rerun: metacheck's changes since this row was written (data/code extension list and archive download fix, 4 Oct; GROBID DOI-space removal, 4 Oct; new registered hosts and Figshare institutional ids, 6 Oct; Dryad zip members, 10 Oct) were checked against this paper. 109a9aa's link detectors find the same repository links in the paper as the stored run, and none of the changes applies to this paper's files, so this explanation still holds. RECLASSIFIED 2026-10-08 (DATA_OR_CODE_NOT_IN_FAIR_FORMAT): The data are serialized R objects saved with the .Rd extension, which is R's documentation-file extension, so the file name mislabels the content. Earlier explanation: NOTE: this paper does not exactly fit the stated premise of 'classified none of the files as data or code' -- 5 of 6 files WERE classified 'code', just wrongly. The 5 real data files (SitesMassBins.Rd etc., each 0.9-1.7MB, MIME application/gzip per the Figshare API, i.e. genuine serialized data objects, not R documentation) were saved with extension '.Rd', colliding with R's own .Rd package-documentation-markup extension. .ext_registry has no Tier-1 entry for 'rd'; Tier 3's coarse crosswalk hardcodes ext=='rd' -> type 'code' (confirmed directly), with no check against file size or binary content that would flag these multi-megabyte gzip blobs as implausible R documentation source.
10.1111/1365-2435.13683	Data or code not shared in a FAIR format	CHECKED 2026-10-17 against metacheck 610cd4f2 by a full rerun. repo_check finds the Dryad deposit 10.5061/dryad.m37pvmd0h cited in the statement, and the Dryad API (version 81320) lists three Word documents, Data_Table_1.docx, Data_Table_2.docx and Data_Table_3.docx (35-66 KB) and nothing else. The data are tables inside these Word documents, which Cooper et al. also recorded (data format .doc(x)). Before the documentation fix (#487) data_check counted such a document as data; it now counts it as documentation, which is correct for the file type, so Metacheck finds no data file. The data exist, but not in a machine-readable format.
10.1111/1365-2435.13735	Data or code not shared in a FAIR format	CHECKED 2026-10-13 against metacheck 109a9aa by comparison, not by a full rerun: metacheck's changes since this row was written (data/code extension list and archive download fix, 4 Oct; GROBID DOI-space removal, 4 Oct; new registered hosts and Figshare institutional ids, 6 Oct; Dryad zip members, 10 Oct) were checked against this paper. 109a9aa's link detectors find the same repository links in the paper as the stored run, and none of the changes applies to this paper's files, so this explanation still holds. RECLASSIFIED 2026-10-08 (DATA_OR_CODE_NOT_IN_FAIR_FORMAT): The simulation outputs are bare .txt files (inside a zip) whose names do not label them as data; a .txt file needs to be labelled as data to be findable. File content not checked directly in this pass. Earlier explanation: 18 genuine simulation-output .txt files (e.g. Coord_row_sim_s0000.txt, Trait_sim_s0000.txt) inside the deposited ZIP are simulation-generated DATA, confirmed by the paper's text and Dryad's own listing, but none were ever assigned data_type=='data'. .txt is deliberately not format-locked in .ext_registry since it can be prose or data; Tier 2 keyword rules require tokens like 'data'/'output'/'results', none of which appear in these filenames; Tier 3's crosswalk routes generic 'text' filetype to 'documentation'. NOTABLY, the RDS snapshot actually used in the manuscript run shows these files as 'code' (not 'documentation' as today's vendored code produces when rerun), indicating the classifier code changed between when the snapshot was generated and now -- a reproducibility concern flagged separately from the root classification gap. Either way, genuine simulation-result data is mistyped into a non-data bucket.

Paper (DOI)	Category	Why Metacheck misses it
10.1111/1365-2435.14060	Data or code not shared in a FAIR format	CHECKED 2026-10-13 against metacheck 109a9aa by comparison, not by a full rerun: metacheck's changes since this row was written (data/code extension list and archive download fix, 4 Oct; GROBID DOI-space removal, 4 Oct; new registered hosts and Figshare institutional ids, 6 Oct; Dryad zip members, 10 Oct) were checked against this paper. 109a9aa's link detectors find the same repository links in the paper as the stored run, and none of the changes applies to this paper's files, so this explanation still holds. RECLASSIFIED 2026-10-08 (DATA_OR_CODE_NOT_IN_FAIR_FORMAT): The files are space-separated .txt tables of analysis outputs, named only by variable (e.g. aggression_rate.txt), and not labelled as data. Earlier explanation: 8 of 9 files in the Dryad repo (aggression_rate.txt, group_size.txt, N_groups_day.txt, N_individuals_day.txt, steepness.txt, strength_associations.txt, time_groups.txt, time_individuals.txt) are genuine tabular research data but all classified data_type='documentation'. .ext_registry leaves .txt's data_type as NA (deliberately not format-locked). Tier 2 keyword rules only override for bounded name tokens 'data'/'code'/etc; none of these 8 filenames contain such a token. All 8 fall to Tier 3's coarse crosswalk: filetype() returns 'text', and .file_type_crosswalk['text'] resolves to 'documentation'.
10.1111/1365-2435.14337	Data or code not shared in a FAIR format	CHECKED 2026-10-13 against metacheck 109a9aa by comparison, not by a full rerun: metacheck's changes since this row was written (data/code extension list and archive download fix, 4 Oct; GROBID DOI-space removal, 4 Oct; new registered hosts and Figshare institutional ids, 6 Oct; Dryad zip members, 10 Oct) were checked against this paper. 109a9aa's link detectors find the same repository links in the paper as the stored run, and none of the changes applies to this paper's files, so this explanation still holds. RECLASSIFIED 2026-10-08 (DATA_OR_CODE_NOT_IN_FAIR_FORMAT): Chen_2023_gross.txt is a tab-separated table preceded by a quoted title line (checked directly), shared as a bare .txt file not labelled as data. Metacheck's after-download content check also misses it (metacheck issue #470). Earlier explanation: Chen_2023_gross.txt is almost certainly the paper's core numeric dataset (gross soil N mineralization/nitrification rates). .txt is deliberately unlocked at Tier 1. The content-based txt_classify_content() upgrade path only recognizes comma/semicolon/tab/pipe delimiters and explicitly, by documented design, does not detect space/whitespace-delimited tables -- plausibly the file's actual format, so it never upgrades past the default 'documentation' guess. README_Chen_2023_gross.txt correctly resolves via the 'readme' keyword.
10.1111/1365-2656.12671	Data or code not shared in a FAIR format	CHECKED 2026-10-13 against metacheck 109a9aa by comparison, not by a full rerun: metacheck's changes since this row was written (data/code extension list and archive download fix, 4 Oct; GROBID DOI-space removal, 4 Oct; new registered hosts and Figshare institutional ids, 6 Oct; Dryad zip members, 10 Oct) were checked against this paper. 109a9aa's link detectors find the same repository links in the paper as the stored run, and none of the changes applies to this paper's files, so this explanation still holds. RECLASSIFIED 2026-10-08 (DATA_OR_CODE_NOT_IN_FAIR_FORMAT): The data are shared as tables inside Word .doc files rather than as machine-readable data files. Earlier explanation: .doc files on Dryad containing genuine research data were classified under a non-data category by the generic document-extension handling.

Paper (DOI)	Category	Why Metacheck misses it
10.1111/1365-2656.12732	Data or code not shared in a FAIR format	CHECKED 2026-10-13 against metacheck 109a9aa by comparison, not by a full rerun: metacheck's changes since this row was written (data/code extension list and archive download fix, 4 Oct; GROBID DOI-space removal, 4 Oct; new registered hosts and Figshare institutional ids, 6 Oct; Dryad zip members, 10 Oct) were checked against this paper. 109a9aa's link detectors find the same repository links in the paper as the stored run, and none of the changes applies to this paper's files, so this explanation still holds. RECLASSIFIED 2026-10-08 (DATA_OR_CODE_NOT_IN_FAIR_FORMAT): Dataset_Outomuro&Johansson.txt is a tab-separated table with a header row (checked directly), shared as a bare .txt file. Metacheck's name rule does not treat 'Dataset' as a data label, and its after-download content check misses the table because of trailing empty cells (metacheck issue #470). Earlier explanation: A genuine data file named with the word 'dataset' (not the bounded token 'data') was missed by Tier 2's keyword regex, which requires 'data' bounded by separators and does not match 'dataset' or similar compound words.
10.1111/1365-2656.13145	Data or code not shared in a FAIR format	CHECKED 2026-10-13 against metacheck 109a9aa by comparison, not by a full rerun: metacheck's changes since this row was written (data/code extension list and archive download fix, 4 Oct; GROBID DOI-space removal, 4 Oct; new registered hosts and Figshare institutional ids, 6 Oct; Dryad zip members, 10 Oct) were checked against this paper. 109a9aa's link detectors find the same repository links in the paper as the stored run, and none of the changes applies to this paper's files, so this explanation still holds. RECLASSIFIED 2026-10-08 (DATA_OR_CODE_NOT_IN_FAIR_FORMAT): Male_SES_encounter_history_matrix.txt is a space-separated .txt matrix (checked directly) whose name does not label it as data. Earlier explanation: Single file Male_SES_encounter_history_matrix.txt is the paper's primary mark-recapture dataset (34-yr encounter history matrix, 6,245 individuals, 58,177 records), explicitly titled 'Data from: Encounter history matrix' in the paper's own reference citation. .ext_registry leaves .txt's data_type as NA. Tier 2 keyword rules require a bounded 'data' token; the filename contains none. Tier 3 crosswalks the coarse 'text' filetype to 'documentation'.
10.1111/1365-2745.13026	Data or code not shared in a FAIR format	CHECKED 2026-10-13 against metacheck 109a9aa by comparison, not by a full rerun: metacheck's changes since this row was written (data/code extension list and archive download fix, 4 Oct; GROBID DOI-space removal, 4 Oct; new registered hosts and Figshare institutional ids, 6 Oct; Dryad zip members, 10 Oct) were checked against this paper. 109a9aa's link detectors find the same repository links in the paper as the stored run, and none of the changes applies to this paper's files, so this explanation still holds. RECLASSIFIED 2026-10-08 (DATA_OR_CODE_NOT_IN_FAIR_FORMAT): The cited Zenodo deposit (https://zenodo.org/records/1291377) contains LICENSE.md, README.md and a supplement PDF. The PDF contains a table with data. The data in this table should have been shared in a FAIR format (e.g., a csv file) for Metacheck to classify the repository as containing data. So there is data, but it is not in a FAIR format; Metacheck's classification of the PDF as documentation is as designed.
10.1111/1365-2745.13069	Data or code not shared in a FAIR format	CHECKED 2026-10-17 against metacheck 610cd4f2 by a full rerun. repo_check finds the Dryad deposit 10.5061/dryad.f4j738r cited in the statement, and the Dryad API (version 19446) lists one Word document, 'Data Archive for Arroyo et al 06,07.18.docx' (46 KB) and nothing else. The data are tables inside these Word documents, which Cooper et al. also recorded (data format .doc(x)). Before the documentation fix (#487) data_check counted such a document as data; it now counts it as documentation, which is correct for the file type, so Metacheck finds no data file. The data exist, but not in a machine-readable format.

Paper (DOI)	Category	Why Metacheck misses it
10.1111/2041-210X.13332	Data or code not shared in a FAIR format	CHECKED 2026-10-13 against metacheck 109a9aa by comparison, not by a full rerun: metacheck's changes since this row was written (data/code extension list and archive download fix, 4 Oct; GROBID DOI-space removal, 4 Oct; new registered hosts and Figshare institutional ids, 6 Oct; Dryad zip members, 10 Oct) were checked against this paper. 109a9aa's link detectors find one link more than the stored run, but the extra link is a Zenodo badge image (zenodo.org/badge/...svg), not a deposit, and none of the changes applies to this paper's files, so this explanation still holds. RECLASSIFIED 2026-10-08 (DATA_OR_CODE_NOT_IN_FAIR_FORMAT): The C++ source is committed without any file extension ('C++ code'), so its language cannot be determined from the file type. Earlier explanation: The repo's sole source file is committed with no extension at all, literally named 'C++ code' (confirmed via live GitHub fetch to be genuine Arduino/C++ source). data_check's own classifier already gets this right (Tier 2 keyword match on the bare word 'code', data_type='code'). The actual miss is downstream in code_check.R's code_lang(), which determines language purely via tools::file_ext() matched against .ext_registry$code_lang; an empty extension matches nothing, so code_lang() returns NA and the file never enters code_check's candidate set, leaving code_check_repo_urls empty despite data_check already having typed the file correctly.
10.1111/2041-210X.13396	Data or code not shared in a FAIR format	CHECKED 2026-10-13 against metacheck 109a9aa by comparison, not by a full rerun: metacheck's changes since this row was written (data/code extension list and archive download fix, 4 Oct; GROBID DOI-space removal, 4 Oct; new registered hosts and Figshare institutional ids, 6 Oct; Dryad zip members, 10 Oct) were checked against this paper. 109a9aa's link detectors find the same repository links in the paper as the stored run, and none of the changes applies to this paper's files, so this explanation still holds. RECLASSIFIED 2026-10-08 (DATA_OR_CODE_NOT_IN_FAIR_FORMAT): The repository's only file is a Word supporting-information document, Nishida_supplementary.docx (https://datadryad.org/downloads/file_stream/292473), opened and read directly. Its Tables S1-S7 report means and standard deviations, and Tables S2-S4 and S6 also give specimen-level values. These are data, but shared as tables in a Word document rather than as a machine-readable file (e.g., a csv file), so Metacheck classifies the file as documentation.
10.1111/2041-210X.13979	Data or code not shared in a FAIR format	CHECKED 2026-10-13 against metacheck 109a9aa by comparison, not by a full rerun: metacheck's changes since this row was written (data/code extension list and archive download fix, 4 Oct; GROBID DOI-space removal, 4 Oct; new registered hosts and Figshare institutional ids, 6 Oct; Dryad zip members, 10 Oct) were checked against this paper. 109a9aa's link detectors find the same repository links in the paper as the stored run, and none of the changes applies to this paper's files, so this explanation still holds. RECLASSIFIED 2026-10-08 (DATA_OR_CODE_NOT_IN_FAIR_FORMAT): The R code is saved as 'R code.txt' rather than with a .R extension, so its language cannot be determined from the file type. Earlier explanation: Sole file 'R code.txt' genuinely is R source code (confirmed via Zenodo's own title/description: implements five richness estimators). data_check's data_classify_files() already correctly tags it 'code' via a Tier-2 keyword match on 'code' bounded by space/period. But code_check()'s separate code_lang() function keys purely on file extension via .ext_registry$code_lang; .txt's code_lang is NA by design, so code_lang('R code.txt') returns NA, the file is filtered out of code_check's relevant-languages set, and code_check_repo_urls comes back empty -- a cross-module inconsistency between data_check's keyword logic and code_check's extension-only logic.

Paper (DOI)	Category	Why Metacheck misses it
10.1111/1365-2435.13183	Format Metacheck does not yet support	CHECKED 2026-10-13 against metacheck 109a9aa by comparison, not by a full rerun: metacheck's changes since this row was written (data/code extension list and archive download fix, 4 Oct; GROBID DOI-space removal, 4 Oct; new registered hosts and Figshare institutional ids, 6 Oct; Dryad zip members, 10 Oct) were checked against this paper. 109a9aa's link detectors find the same repository links in the paper as the stored run, and none of the changes applies to this paper's files, so this explanation still holds. data_check correctly typed the repo's one Julia source file (wns-simple.jl) as 'code' via metacheck::file_types, where jl->code is registered. However, code_check (whose output is what the manuscript's n_mc_code aggregate actually counts) gates everything through code_lang(), which looks up language ONLY via .ext_registry$code_lang -- and that table has no row for extension 'jl' at all (Julia appears only as a possible notebook kernel, never as its own .jl entry). So code_lang() returns NA, the file is never flagged as a code-check candidate, and code_check_repo_urls is empty despite data_check correctly flagging the same file. This is an internal inconsistency between two classifier tables that should agree but don't. CORRECTION 2026-10-08: a live rerun confirmed .ext_registry$code_lang now HAS a row for extension 'jl' (added 2026-10-04), and data_classify_files() correctly tags wns-simple.jl as data_type = 'code'. But this row's original diagnosis was already about a DIFFERENT, downstream gate: code_check.R's own `listed_langs` whitelist (R, Python, SAS, SPSS, Stata, Mplus, MATLAB, JASP) never included 'Julia' as a possible language at all, so code_check's own `table` (what the manuscript's Cooper-vs-Metacheck comparison counts, via code_table_slim_obj) still has zero rows for this paper even though data_check's classification is now correct. Confirmed directly: a fresh rerun through module_run() produced res_code_check$table with 0 rows for this paper_id. The extension-registry fix and the code_check language-whitelist gap are two separate, independent defects; only the first has been fixed.
10.1111/1365-2435.13315	Format Metacheck does not yet support	CHECKED 2026-10-13 against metacheck 109a9aa by comparison, not by a full rerun: metacheck's changes since this row was written (data/code extension list and archive download fix, 4 Oct; GROBID DOI-space removal, 4 Oct; new registered hosts and Figshare institutional ids, 6 Oct; Dryad zip members, 10 Oct) were checked against this paper. 109a9aa's link detectors find the same repository links in the paper as the stored run, and none of the changes applies to this paper's files, so this explanation still holds. The deposit's only file, DataDryad.7z, is a 7-Zip archive. Metacheck's zip-peek.R explicitly documents that only .zip, the .tar family, and single-file .gz/.bz2/.xz can be opened with base R -- .7z is deliberately left unextracted ('left as other, never fetched') since Metacheck has no libarchive/7z dependency. Since the archive is never opened, data_classify_files() only ever sees the archive's own filename. A second, independent factor compounds this: even the Tier-2 keyword fallback that would normally rescue a misnamed data file fails, since the bounded-token regex requires a separator before 'data' and 'DataDryad.7z' has 'data' glued directly to 'dryad' with no boundary character (confirmed by running the exact regex).
10.1111/1365-2435.13848	Format Metacheck does not yet support	RECHECKED 2026-10-17 by a full rerun (metacheck 610cd4f2), which lists all 27 files of the Dryad deposit since the paging fix (before: the first 20): still no file classified as data. The 24 ANSYS Fluent simulation files (12 .cas.gz and 12 .dat.gz) are typed 'unknown', the laser scan (.stl) 'materials', README.txt documentation and one report file (.out) output. ANSYS Fluent simulation files are genuine simulation data with no entry in .ext_registry, a domain-specific engineering/CFD format gap.

Paper (DOI)	Category	Why Metacheck misses it
10.1111/1365-2664.13096	Format Metacheck does not yet support	CHECKED 2026-10-13 against metacheck 109a9aa by comparison, not by a full rerun: metacheck's changes since this row was written (data/code extension list and archive download fix, 4 Oct; GROBID DOI-space removal, 4 Oct; new registered hosts and Figshare institutional ids, 6 Oct; Dryad zip members, 10 Oct) were checked against this paper. 109a9aa's link detectors find the same repository links in the paper as the stored run, and none of the changes applies to this paper's files, so this explanation still holds. Sole file 'EEEworm Model V1.0.0 Johnston et al..nlogo' is NetLogo model source code (confirmed as 'programme code' by the paper's own text). .ext_registry has no 'nlogo' entry anywhere. Tier 2 keyword rules find no match in the filename. Tier 3 coarse crosswalk also has no NetLogo mapping. Directly executed: data_classify_files() on this filename returns 'unknown'. CORRECTION 2026-10-08: a live rerun confirmed .ext_registry now HAS an 'nlogo' entry (added 2026-10-04), and data_classify_files() correctly tags the .nlogo model file as data_type = 'code' (no longer 'unknown' as this row originally found). But code_check.R's own `listed_langs` whitelist (R, Python, SAS, SPSS, Stata, Mplus, MATLAB, JASP) has never included 'NetLogo', so code_check's own `table` still has zero rows for this paper -- confirmed directly via a fresh rerun. Same underlying class of gap as the Julia case above: the extension-registry fix and code_check's separate language-whitelist gate are independent defects, and only the registry side has been addressed.
10.1111/1365-2664.13538	Format Metacheck does not yet support	CHECKED 2026-10-13 against metacheck 109a9aa by comparison, not by a full rerun: metacheck's changes since this row was written (data/code extension list and archive download fix, 4 Oct; GROBID DOI-space removal, 4 Oct; new registered hosts and Figshare institutional ids, 6 Oct; Dryad zip members, 10 Oct) were checked against this paper. 109a9aa's link detectors find the same repository links in the paper as the stored run, and none of the changes applies to this paper's files, so this explanation still holds. Single .rar archive genuinely contains R scripts and tabular data files (confirmed via Dryad's own file-content description). .rar is absent from .ext_registry entirely. zip-peek.R's own comment states .rar/.7z 'need an external binary or the libarchive-backed archive package, which metacheck does NOT depend on' -- a deliberately-documented limitation, not an oversight. .is_readable_archive() returns FALSE for 'rar', so it is never downloaded or opened.
10.1111/1365-2664.14746	Format Metacheck does not yet support	CHECKED 2026-10-13 against metacheck 109a9aa by comparison, not by a full rerun: metacheck's changes since this row was written (data/code extension list and archive download fix, 4 Oct; GROBID DOI-space removal, 4 Oct; new registered hosts and Figshare institutional ids, 6 Oct; Dryad zip members, 10 Oct) were checked against this paper. 109a9aa's link detectors find one link more than the stored run, but the extra link is the same OSF project written a second way (OSF.IO/AWSXG), and none of the changes applies to this paper's files, so this explanation still holds. Three GeoTIFF spatial-raster data layers (microcorrs500m.tif, PAconns500m.tif, betweenness500m.tif) are exactly what the paper's Data Availability Statement names as shared data. .ext_registry has no entry for 'tif'/'tiff'. Tier 2 finds no keyword match. Tier 3 resolves via metacheck::file_types to coarse type 'image', and .file_type_crosswalk hard-maps image->'materials' -- a rule for ordinary photographs, not disambiguated for georeferenced raster data.

Paper (DOI)	Category	Why Metacheck misses it
10.1111/1365-2745.13006	Format Metacheck does not yet support	CHECKED 2026-10-13 against metacheck 109a9aa by comparison, not by a full rerun: metacheck's changes since this row was written (data/code extension list and archive download fix, 4 Oct; GROBID DOI-space removal, 4 Oct; new registered hosts and Figshare institutional ids, 6 Oct; Dryad zip members, 10 Oct) were checked against this paper. 109a9aa's link detectors find the same repository links in the paper as the stored run, and none of the changes applies to this paper's files, so this explanation still holds. Sole file 'To submission.rar' genuinely contains research data and R code (Figshare's own description: 'contains research materials including a dataset and analytical code'). .ext_registry has no entry for 'rar'. Archive-expansion in data_check.R only unpacks zip/tar-family/gz/bz2/xz; code comments explicitly state .7z/.rar are not carved out and stay excluded, with repo_check instead warning the author to re-upload as .zip.
10.1111/2041-210X.13124	Format Metacheck does not yet support	CHECKED 2026-10-13 against metacheck 109a9aa by comparison, not by a full rerun: metacheck's changes since this row was written (data/code extension list and archive download fix, 4 Oct; GROBID DOI-space removal, 4 Oct; new registered hosts and Figshare institutional ids, 6 Oct; Dryad zip members, 10 Oct) were checked against this paper. 109a9aa's link detectors find the same repository links in the paper as the stored run, and none of the changes applies to this paper's files, so this explanation still holds. Sole .rar file genuinely contains two GIS raster data files (congruence.grd, pr_Litocranius_walleri.grd), confirmed via Dryad's usage notes. .ext_registry has no entry for 'rar' (or '7z'). data_check.R's archive-expansion comment explicitly states '.7z/.rar are NOT here -- base R cannot read them (repo_check warns instead)' -- a deliberate design limitation.
10.1111/1365-2664.14504	Format Metacheck does not yet support (read as image material)	CHECKED 2026-10-13 against metacheck 109a9aa by comparison, not by a full rerun: metacheck's changes since this row was written (data/code extension list and archive download fix, 4 Oct; GROBID DOI-space removal, 4 Oct; new registered hosts and Figshare institutional ids, 6 Oct; Dryad zip members, 10 Oct) were checked against this paper. 109a9aa's link detectors find the same repository links in the paper as the stored run, and none of the changes applies to this paper's files, so this explanation still holds. Four GeoTIFF raster layers (DailyAbsorpWinter_ AllConflict.tif, DailyAbsorpWinter_Livestock.tif, FidelityWinter.tif, ResistanceWinter.tif) are genuine SAMC model input layers for a wolf-recolonization connectivity analysis. .ext_registry has no row for '.tif'/'.tiff'. Tier 2 finds no keyword match. Tier 3's coarse crosswalk buckets .tif under coarse type 'image', and .file_type_crosswalk hardcodes image->'materials' unconditionally -- a rule built for photographs/figures, not GIS raster data (unlike .nii/.dcm/.mnc/.mgz, which the registry already special-cases to 'data').
10.1111/2041-210X.14457	Format Metacheck does not yet support (read as image material)	CHECKED 2026-10-13 against metacheck 109a9aa by comparison, not by a full rerun: metacheck's changes since this row was written (data/code extension list and archive download fix, 4 Oct; GROBID DOI-space removal, 4 Oct; new registered hosts and Figshare institutional ids, 6 Oct; Dryad zip members, 10 Oct) were checked against this paper. 109a9aa's link detectors find the same repository links in the paper as the stored run, and none of the changes applies to this paper's files, so this explanation still holds. 64 .JPG drone photographs are the paper's primary measurement data for 3D structure-from-motion photogrammetry (explicitly: 'Data required to recreate analysis with ground truthing images'), not participant-facing stimuli. .ext_registry has no entry for 'jpg'/'jpeg'; classification falls to the coarse crosswalk, which hard-codes image->'materials' -- a rule correct for the common case (a stimulus photo) but wrong for this domain where photographs are literally the quantitative primary data.

A.11 What changed in the final run, why, and the consequences for the results

The results in this manuscript come from two runs. The `repo_check`, `data_check`, `code_check`, `open_practices` and ODDPub results come from one run on all 1861 papers with Metacheck commit e62da2e (version 0.4.2.1), after which 22 papers were run again with commit f7949c0, which holds fixes for faults found during that run. The `reproducibility_check` results come from a separate run, on the 435 of those papers that have runnable code, with commit f6aec07c (version 0.4.3.6) — described in A.7 — together with its two follow-up runs: 58 papers run again on an image with system header files added, and one paper run on the `dev` branch (0.4.3.13) after a fix for a fault that had stopped its check. Each paper's reproducibility result records which of these runs and which commit produced it. The repository, data and code results are unchanged from the earlier pass of this project described below; the reproducibility results are new to the final run.

The results of an earlier version of this project were a combination of several partial runs with older versions, the last of them with commit 610cd4f (version 0.3.3.11). This appendix compares the last of those earlier results, dated 18 October 2026, with the final results, for the columns that can be compared with Cooper et al.'s coding (the repository, data and code columns), to show whether the conclusions depend on the version of Metacheck. Table 5 shows they do not: no agreement percentage moved by more than 0.2 percentage points.

Table 5: Percentage agreement between Cooper et al. and Metacheck in the earlier and the final results, and the number of papers both coded.

Comparison	Earlier %	Final %	Earlier n	Final n
Data availability	85.5	85.5	1,735	1,735
Data archive platform (exact match of platform sets)	90.4	90.3	1,403	1,413
Data licence present	95.7	95.7	1,323	1,333
Data downloadable and usable	99.2	99.2	1,337	1,349
Code archived	88.0	87.9	1,669	1,669
Code archive platform (exact match of platform sets)	77.8	78.0	410	410
Code downloadable	99.8	99.8	404	404
Any README present (data or code, collapsed)	72.9	72.9	1,726	1,726

In the columns compared with Cooper et al., 62 values changed, in 30 of the 1861 papers. Of these values, 10 moved from disagreement to agreement with Cooper et al. and 5 moved from agreement to disagreement. A further 32 could not be compared before, because one side had no answer, and can be now: 25 agree and 7 disagree. Of the rest, 6 still disagree with a

different value, and 9 cannot be compared in either version. Table 6 gives the cause of each kind of change: the change in Metacheck, the commit that made it, and what it did to the results. Table 7 lists every changed value.

Two causes change results for a reason other than a correction of Metacheck. `NOT_DOWNLOADED_ON_PURPOSE` is a decision for this run. For `NOW_DOWNLOADED`, the final run holds a saved copy of files that the earlier results do not, and we did not establish why the earlier runs had none.

Table 6: Causes of the changed values: the change in Metacheck, its commit, its consequence, and the number of papers (P) and values (V). T and A: values that moved to or away from agreement with Cooper et al. N+ and N-: values that could not be compared before and now agree or disagree.

Cause	Commit	Mechanism	Consequence	P	V	T	A	N+	N-
FILE_TYPE_TABLE	d52b3fd9; 9509af14	metacheck's table of file types (file_types.rda) changed: .json counts as data only (it was code and data), .js, .ts and .tsx as code only, .html as a web page and no longer as code; shapefile parts and .ply count as data. The earlier results of these papers were made before this change.	Pages of a documentation website no longer count as code, and JavaScript no longer counts as data. JSON files count as data, also configuration files such as package.json, so software repositories can now be listed as holding data.	6	9	0	1	1	3
ADDRESS_ PUNCTUATION	5c4feb1e	repo_check removes a final . , ; or : from a repository address. With the full stop that ended a sentence the address led nowhere, and the deposit was listed without files.	The deposit is found and its files are listed.	2	4	0	0	1	2
DATAONE_ IDENTIFIERS	5c4feb1e	A DataONE file identifier with slashes is kept whole, it was cut at its last slash; a file with no usable identifier is found by name in the node's search, which works again on KNB.	Files that failed to download are fetched and read.	2	5	0	1	3	1
DRYAD_SHARING_ LINK	f7826893	repo_check follows a Dryad sharing link (datadryad.org/stash/share/...), the link made for reviewers, to the dataset it leads to. It carries no DOI, so the paper had no repository.	The deposit is found, so data, license, download and README all change.	1	5	2	0	2	1
ROOT_R_PACKAGE_ KEPT	9509af14	repo_check keeps the files of an R package at a repository's root when the only files outside it configure services such as Travis, codecov or GitHub Actions. These files have the extension .yml, which counts as code, so the package looked like a bundled dependency and its files were removed.	The GitHub repository's R files are listed, so GitHub joins the platforms for code, and for data when the package holds data files.	3	5	0	1	0	0

Cause	Commit	Mechanism	Consequence	P	V	T	A	N+	N-
COMPRESSED_TAR_UNPACKED	bd44cc56; not separated from other changes to archives	data_check unpacks a compressed tar file (.tar.gz) inside a deposit and classifies the files in it, also when it lies inside a zip file. The earlier results list it as one file of unknown type.	R files and data files inside the archive are found.	3	7	1	1	3	0
OSF_LISTING	d52b3fd9	osf_info() lists a project's whole tree of components and folders. The earlier listing of this project held 96 files, the final listing 131.	Scripts and a README in folders that were not listed are found.	1	4	1	1	0	0
OSF_LICENSE	f7826893	With osf_license = TRUE, repo_check reads the license of an OSF project through its registration when the project address does not answer. The final run sets this argument.	The license is found.	1	1	1	0	0	0
DSPACE_LOOKUP	d52b3fd9; 3bfb2ed7; not separated	DSpace repositories are recognised from doi.org's handle service and by host name, and a repository's former address is recognised.	The deposit, or its license, is found.	3	8	5	0	3	0
DUPLICATE_OF_OSF_LISTING	d52b3fd9; abc2e2a8	An OSF project with a linked GitHub repository is listed through GitHub, so its rows have the same address and path as the GitHub repository's own rows. repo_check keeps each file once, under OSF, which comes first.	No file is lost, but GitHub leaves the list of platforms although the paper names it.	1	1	0	0	0	0
NOW_DOWNLOADED	not established	The final run holds a saved copy of these files, and the earlier results do not. For a .txt file, data_check reads the content of a saved copy and calls it data when it is a table (txt_classify_content(), since e1b5aa17, 2026-08-12), which explains the three papers with .txt data. Why the earlier runs have no copy was not established; no change in metacheck was found that explains it.	Data files are read, and data_download becomes TRUE.	6	12	0	0	12	0
NOT_DOWNLOADED_ON_PURPOSE	none; setting of this run	The final run does not download the SAPFLUXNET deposit of 10.1111/1365-2664.14383, on the authors' decision, because of its size; code/run_metacheck.R lists the paper in no_download.	data_download is missing for this paper instead of TRUE.	1	1	0	0	0	0

Table 7: Every changed value in a column compared with Cooper et al.

Paper	Column	Earlier	Final	Effect	Cause
10_1002_2688_8319_12310	data_archive	Figshare;GitHub;Zenodo	Figshare	still_disagrees	FILE_TYPE_TABLE
10_1002_2688_8319_12402	data_license	FALSE	TRUE	toward	OSF_LICENSE
10_1002_pan3_10551	data_archive	GitHub;Zenodo	GitHub;Other;Zenodo	still_disagrees	ADDRESS_PUNCTUATION
10_1111_1365_2435_13192	data_download	missing	TRUE	new_agreement	NOW_DOWNLOADED
10_1111_1365_2435_13921	data_archive	missing	Zenodo	new_agreement	NOW_DOWNLOADED
10_1111_1365_2435_13921	data_license	missing	TRUE	new_agreement	NOW_DOWNLOADED
10_1111_1365_2435_13921	data_download	missing	TRUE	new_agreement	NOW_DOWNLOADED
10_1111_1365_2435_14022	data_availability	FALSE	TRUE	toward	DRYAD_SHARING_LINK
10_1111_1365_2435_14022	data_archive	missing	Dryad	new_agreement	DRYAD_SHARING_LINK
10_1111_1365_2435_14022	data_license	missing	TRUE	new_disagreement	DRYAD_SHARING_LINK
10_1111_1365_2435_14022	data_download	missing	TRUE	new_agreement	DRYAD_SHARING_LINK
10_1111_1365_2435_14022	any_readme	FALSE	TRUE	toward	DRYAD_SHARING_LINK
10_1111_1365_2435_14173	data_archive	GitHub;OSF;Zenodo	OSF;Zenodo	not_comparable	DUPLICATE_OF_OSF_LISTING
10_1111_1365_2435_14305	code_archived	FALSE	TRUE	away	COMPRESSED_TAR_ UNPACKED
10_1111_1365_2435_14305	code_archive	missing	Zenodo	not_comparable	COMPRESSED_TAR_ UNPACKED
10_1111_1365_2435_14305	code_download	missing	TRUE	not_comparable	COMPRESSED_TAR_ UNPACKED
10_1111_1365_2435_14431	code_archived	FALSE	TRUE	away	OSF_LISTING
10_1111_1365_2435_14431	code_archive	missing	OSF	not_comparable	OSF_LISTING
10_1111_1365_2435_14431	code_download	missing	TRUE	not_comparable	OSF_LISTING
10_1111_1365_2435_14431	any_readme	FALSE	TRUE	toward	OSF_LISTING
10_1111_1365_2435_14492	data_download	missing	TRUE	new_agreement	NOW_DOWNLOADED
10_1111_1365_2656_12978	data_archive	missing	Dryad	new_agreement	NOW_DOWNLOADED
10_1111_1365_2656_12978	data_license	missing	TRUE	new_agreement	NOW_DOWNLOADED
10_1111_1365_2656_12978	data_download	missing	TRUE	new_agreement	NOW_DOWNLOADED

Paper	Column	Earlier	Final	Effect	Cause
10_1111_1365_2656_13159	data_archive	missing	Other	new_agreement	DSPACE_LOOKUP
10_1111_1365_2656_13159	data_license	missing	TRUE	new_agreement	DSPACE_LOOKUP
10_1111_1365_2656_13159	data_download	missing	TRUE	new_agreement	DSPACE_LOOKUP
10_1111_1365_2656_13159	code_archive	GitHub	GitHub;Other	toward	DSPACE_LOOKUP
10_1111_1365_2656_13222	data_archive	missing	Dryad	new_agreement	NOW_DOWNLOADED
10_1111_1365_2656_13222	data_license	missing	TRUE	new_agreement	NOW_DOWNLOADED
10_1111_1365_2656_13222	data_download	missing	TRUE	new_agreement	NOW_DOWNLOADED
10_1111_1365_2656_13504	data_download	missing	TRUE	new_agreement	DATAONE_IDENTIFIERS
10_1111_1365_2664_14383	data_download	TRUE	missing	not_comparable	NOT_DOWNLOADED_ON_PURPOSE
10_1111_1365_2664_14751	data_archive	GitHub	GitHub;Other	toward	DSPACE_LOOKUP
10_1111_1365_2664_14751	data_license	FALSE	TRUE	toward	DSPACE_LOOKUP
10_1111_1365_2664_14751	code_archive	GitHub	GitHub;Other	toward	DSPACE_LOOKUP
10_1111_1365_2745_14312	data_download	missing	TRUE	new_agreement	NOW_DOWNLOADED
10_1111_1365_2745_14340	code_archive	Dryad;Zenodo	Dryad;GitHub;Zenodo	still_disagrees	ROOT_R_PACKAGE_KEPT
10_1111_2041_210x_12774	data_archive	Dryad	Dryad;GitHub	not_comparable	ROOT_R_PACKAGE_KEPT
10_1111_2041_210x_12774	code_archive	Dryad	Dryad;GitHub	away	ROOT_R_PACKAGE_KEPT
10_1111_2041_210x_13114	data_archive	missing	Figshare;GitHub;Zenodo	new_disagreement	ADDRESS_PUNCTUATION
10_1111_2041_210x_13114	data_license	missing	TRUE	new_disagreement	ADDRESS_PUNCTUATION
10_1111_2041_210x_13114	data_download	missing	TRUE	new_agreement	ADDRESS_PUNCTUATION
10_1111_2041_210x_13120	code_archive	Zenodo	GitHub;Zenodo	toward	COMPRESSED_TAR_UNPACKED
10_1111_2041_210x_13308	data_license	FALSE	TRUE	toward	DSPACE_LOOKUP
10_1111_2041_210x_13437	code_archive	GitHub;Zenodo	Zenodo	away	FILE_TYPE_TABLE
10_1111_2041_210x_13669	data_archive	missing	GitHub;Zenodo	new_disagreement	FILE_TYPE_TABLE
10_1111_2041_210x_13669	data_license	missing	TRUE	new_disagreement	FILE_TYPE_TABLE
10_1111_2041_210x_13709	data_archive	missing	GitHub	not_comparable	FILE_TYPE_TABLE
10_1111_2041_210x_13709	data_license	missing	TRUE	not_comparable	FILE_TYPE_TABLE

Paper	Column	Earlier	Final	Effect	Cause
10_1111_2041_210x_13748	data_archive	missing	Zenodo	new_agreement	COMPRESSED_TAR_UNPACKED
10_1111_2041_210x_13748	data_license	missing	TRUE	new_agreement	COMPRESSED_TAR_UNPACKED
10_1111_2041_210x_13748	data_download	missing	TRUE	new_agreement	COMPRESSED_TAR_UNPACKED
10_1111_2041_210x_13787	data_archive	GitHub	GitHub;Zenodo	still_disagrees	FILE_TYPE_TABLE
10_1111_2041_210x_14062	data_archive	missing	GitHub;Zenodo	new_disagreement	FILE_TYPE_TABLE
10_1111_2041_210x_14062	data_license	missing	TRUE	new_agreement	FILE_TYPE_TABLE
10_1111_2041_210x_14387	data_archive	missing	Other	new_disagreement	DATAONE_IDENTIFIERS
10_1111_2041_210x_14387	data_license	missing	TRUE	new_agreement	DATAONE_IDENTIFIERS
10_1111_2041_210x_14387	data_download	missing	TRUE	new_agreement	DATAONE_IDENTIFIERS
10_1111_2041_210x_14387	any_readme	FALSE	TRUE	away	DATAONE_IDENTIFIERS
10_1111_2041_210x_14427	data_archive	Zenodo	GitHub;Zenodo	still_disagrees	ROOT_R_PACKAGE_KEPT
10_1111_2041_210x_14427	code_archive	Zenodo	GitHub;Zenodo	still_disagrees	ROOT_R_PACKAGE_KEPT

Columns that are not compared with Cooper et al. changed for more papers. The largest changes do not reflect a change in what was found: `mc_on_request` went from missing to FALSE for 167 papers, because every paper now gets a row in `open_practices`. `mc_code_open` went from TRUE to FALSE for 30 papers, because only a capital R now counts as a word for code, so a statistic such as r(68) = .58 no longer makes a sentence a statement about code. Table 8 lists these columns.

Table 8: Changes in columns that are not compared with Cooper et al.

Column	Papers	Commit	Mechanism
mc_on_request	167	5deacb37	Every paper now gets a row in open_practices, so a paper with no flagged sentence reads FALSE instead of missing. No answer changed.
mc_code_open	30	5deacb37	Only a capital R counts as a word for code, so a statistic such as r(68) = .58 no longer makes a sentence a code statement.
mc_data_open	3	not established	The statements of these papers offer data only on request, and they no longer count as open data.

Column	Papers	Commit	Mechanism
mc_data_format	94	e985e2af; 358074ae; d52b3fd9	Operating system files such as .DS_Store and __MACOSX folders are left out, and the file types of cause FILE_TYPE_TABLE apply.
mc_data_doi	15	follows from the compared columns	The DOI of a deposit is read only for a repository that holds data, so it changes with data_archive.
mc_code_language	5	follows from the compared columns	The languages are read from the code files found.
mc_data_README	3	follows from the compared columns	Same value as any_readme.
mc_code_README	3	follows from the compared columns	Same value as any_readme.

The final results keep two limitations that these comparisons brought to light. First, `repo_check` removes the folders of an R package at a repository's root when any file classified as data or code lies outside them, on the assumption that the package is a dependency bundled into a study's repository. In the saved GitHub listings this removes files from 46 packages in 41 papers, among them the authors' own package in 10.1111/2041-210X.13437, where only the files of a documentation website lie outside it. Adding the removed files back to the final results, in a simulation that is not part of this pipeline and covers GitHub listings only, changes the answer on whether code is archived for one paper (agreement falls from 87.9% to 87.8%), the agreement on the code platform from 78.0% to 77.8%, and on the data platform from 90.3% to 90.2%. We therefore left the rule unchanged. Second, because `.json` files count as data, a configuration file such as `package.json` can make a software repository count as holding data: 11 papers have such a file among their data files, and in 2 papers all data files are JSON files.